INTERNATIONAL CYBERSECURITY AND DIGITAL FORENSICS ACADEMY

SBT-DF204 DIGITAL FORENSICS

Case Study 4: Investigating Rhion Possession Evidence

Item	Detail
Student name	Nebeuwa Ifeanyichukwu Raphael
Registration number	202520850LE
Course and case	SBT-DF204, Case Study 4
Evidence set	DFRWS 2005 "Rhino Hunt" case (NIST CFReDS): one USB disk image (RHINOUSB.dd) and three network captures (rhino.log, rhino2.log, rhino3.log)
Examination platform	Kali Linux (rolling) virtual machine; analysis performed offline on read-only working copies
Examination period	25 to 31 October 2026
Report date	31 October 2026

Every finding in this report separates **observation** (what a tool showed) from **interpretation** (what I conclude from it). Each screenshot sits directly under the finding it supports and carries a figure number. Evidence IDs (E01 onward) link the findings to the evidence log in Appendix A.

1. Executive summary

I examined a seized USB image and three network captures from a simulated investigation into the possession of rhinoceros images, where the scenario treats possession of nine or more unique rhinoceros images as a crime. My main findings are:

- **Integrity.** All four evidence files matched the published MD5 values. I recorded SHA-256 values, set the originals read-only and worked only on copies (E01 to E05).
- **USB image.** The image is one FAT16 volume with no partition table. It holds two allocated recipe files, no deleted directory entries, and 259,227,136 bytes of unallocated space, of which 201,794,898 bytes are not zero (E06 to E10).
- **Recovery.** PhotoRec carved 132 files: 7 JPEG, 2 GIF, 1 Word document and 122 text files. The JPEGs are 6 unique images because one pair is identical. The start sectors of all nine image files are unallocated (E11 to E16).
- **Hidden content.** stegdetect flagged JPEG files for the jphide tool. Dictionary-recovered passphrases then extracted two further valid JPEG images from two of the carriers (E17 to E20).
- **FTP.** On 2004-04-26 between 22:21:49Z and 22:26:46Z, the account "gnome" uploaded rhino1.jpg, rhino3.jpg and an encrypted archive from 137.30.122.253 to the FTP server 137.30.120.40. The archive holds rhino2.jpg, which is byte-identical to the file I carved from the USB image (E21 to E25).
- **HTTP.** On 2004-04-28, client 137.30.123.234 retrieved rhino4.jpg and rhino5.gif from www.cs.uno.edu/~gnome, searched Google for "rhino.exe", and then downloaded /~gnome/rhino.exe (145,920 bytes). I examined that program statically only (E26 to E27).
- **Image count.** The evidence contains 14 unique image files by SHA-256. By my visual review, **10 depict a rhinoceros**, confirmed by directly viewing every file. The USB image alone holds 6 unique rhinoceros images. The threshold of nine is reached only when images seen in the network captures are added (E28).

Confidence. I have high confidence in the technical facts, because hashes, headers, trailers and byte counts were verified. I have moderate confidence that the FTP and HTTP activity relate to the same "gnome" account, because the link is a name only. I have low confidence in any statement about which person used which device, because nothing in the evidence establishes that.

2. Scope, handling rules and method

2.1 Rules I followed

- I used only the course-supplied case materials and the tools named in the teaching decks, and I worked offline on the evidence.
- I did not execute rhino.exe, did not submit any hash to an online service, and did not contact any host named in the evidence.
- I treat the FTP login credentials, the archive password and the steganography passphrases as sensitive. They are blacked out in the screenshots and withheld from the text.

- Screenshots that would show the picture content are masked (pixelated), so the report contains no media files.
- The recovered images remain inside my lab virtual machine. I will shred them only after assessment, using `shred -vzn 3`, as the brief requires.

2.2 Tools and versions

Purpose	Tool and version
Platform	Kali Linux (rolling) virtual machine
Hashing and copying	`sha256sum`, `md5sum`, `unzip`, `dd`, `xxd`
Disk analysis	The Sleuth Kit: `mmls`, `img_stat`, `fsstat`, `fls`, `istat`, `icat`, `blkls`, `blkstat` ; `fdisk`
File recovery	PhotoRec 7.2 (February 2024)
Metadata and viewing	ExifTool 13.50, `file`, `jpeginfo`, ImageMagick `montage`, Eye of GNOME
Steganography	stegdetect and stegbreak 0.6.1 built from commit 46a09e93320cd63b6731e0bc5b22c11c588a0e81; jpseek 0.51 (jphs 0.5) under Wine
Password recovery (course use only)	Kali wordlist `rockyou.txt` for stegbreak; `fcrackzip` for the archive
Network analysis	Wireshark and `tshark`

2.3 Time handling

All packet times are shown in UTC. The FAT file-system times, the zip entry times and the dates inside Office and JPEG files carry **no time zone**. My virtual machine labels local times as "-0400" or "EDT", but that label comes from my machine and not from the evidence. I therefore report the raw values and state any assumption where I make one.

2.4 Method in outline

1. Preserve: hash the package, set it read-only, extract to working copies and hash them again.
2. Examine the disk image with the Sleuth Kit before recovering anything.
3. Recover files from unallocated space with PhotoRec, verify the carved files and review their content and metadata.
4. Test the JPEG files for hidden content with stegdetect, stegbreak and jpseek.
5. Rebuild the files transferred in the FTP capture, validate them and examine the encrypted archive.
6. Export and validate the objects in the two HTTP captures.
7. Correlate all artefacts by hash, build a UTC timeline, and state conclusions and limitations.

3. Evidence acquisition and integrity (E01 to E05)

Observation. I created a case folder structure and recorded the examination start time as **2026-10-25 10:35:21 UTC** (Figure 1). I first searched my home folder for any earlier copy of the case files and then downloaded the course package, 3,614,418 bytes, from the course repository link in the teaching deck (Figure 2). The file was saved at 06:40:44 -0400 on my machine, which is **10:40:44 UTC**.

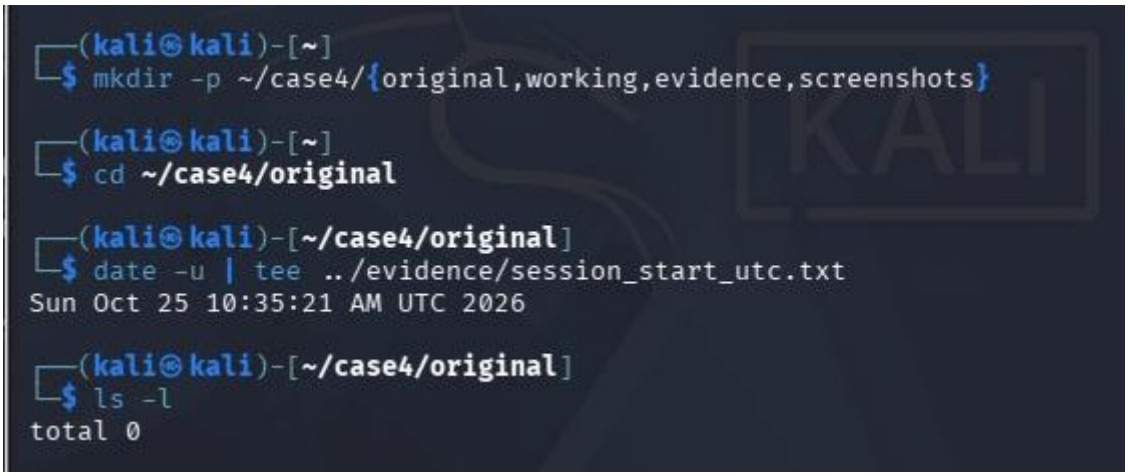


Figure 1. Working folders created and the examination start time recorded in UTC (2026-10-25 10:35:21 UTC).

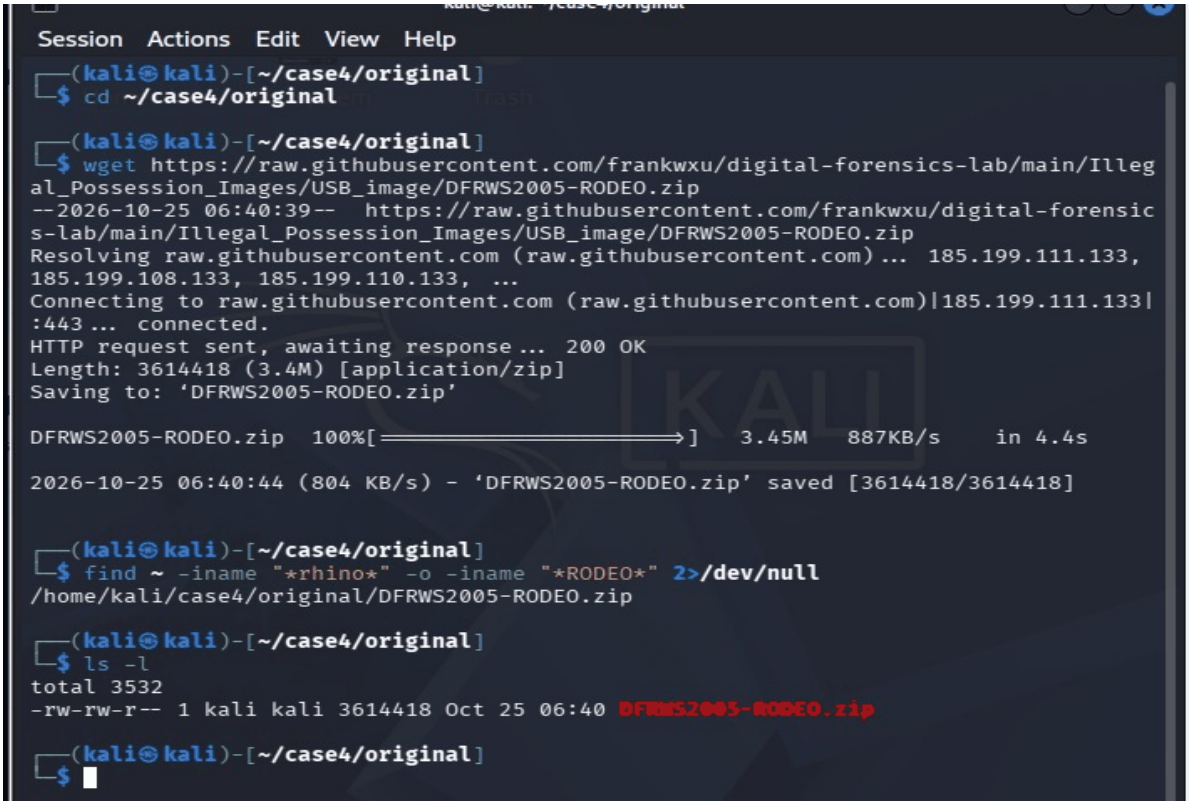


Figure 2. Course package downloaded (3,614,418 bytes) after a search for any pre-existing copies of the case files.

I computed SHA-256 and MD5 of the original zip before opening it, and set it read-only (Figure 4). I then extracted its four files into a separate working folder (Figure 3) and hashed each of them (Figure 5). I set the working files read-only as well (Figure 6).

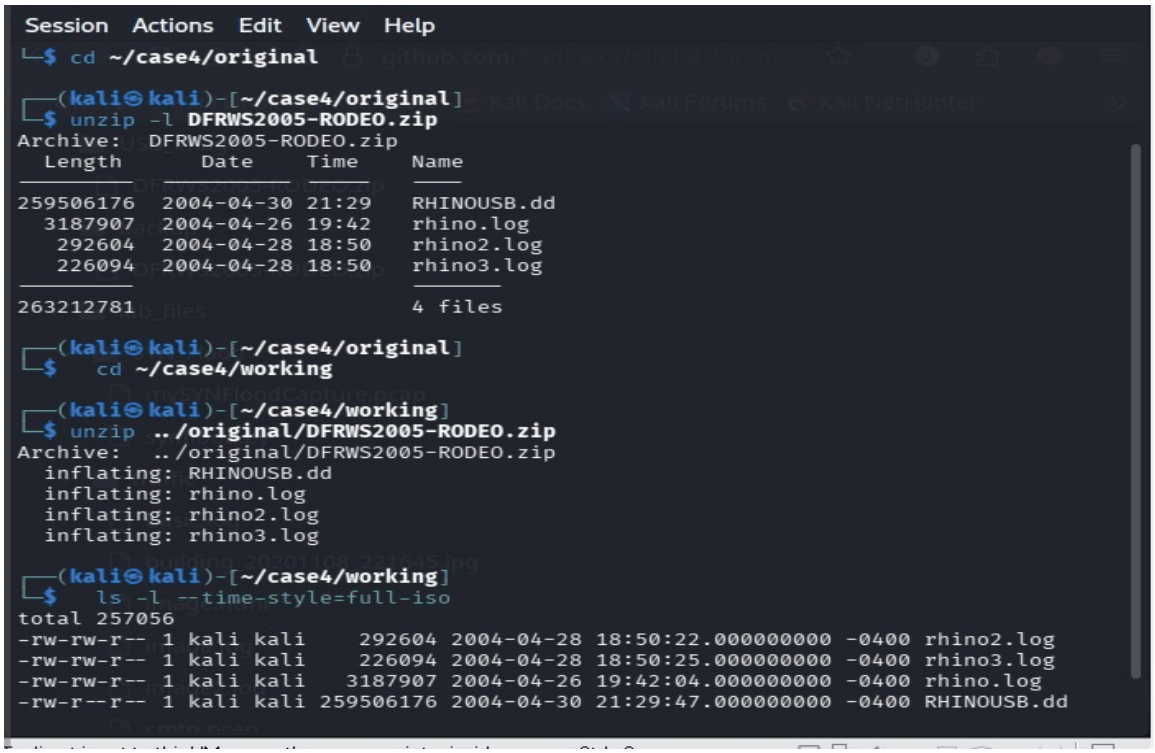


Figure 3. Package contents listed with unzip -l and extracted into the working folder; file times are shown with the VM offset (-0400).

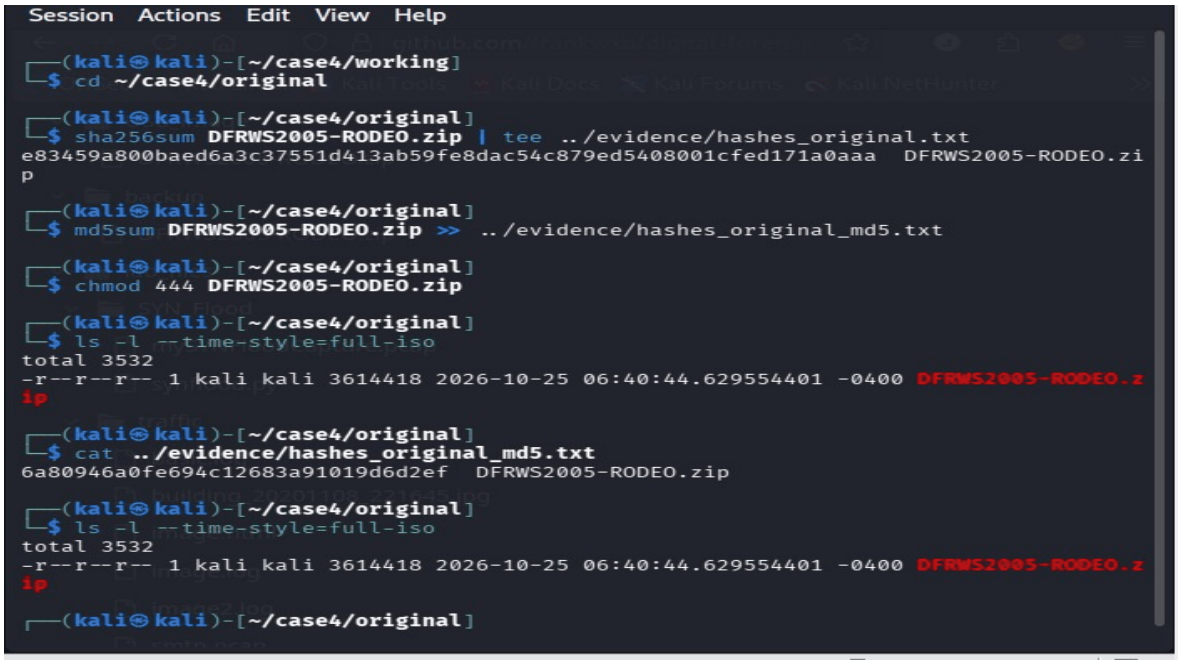


Figure 4. SHA-256 and MD5 of the original zip recorded before use; the original set read-only (chmod 444).

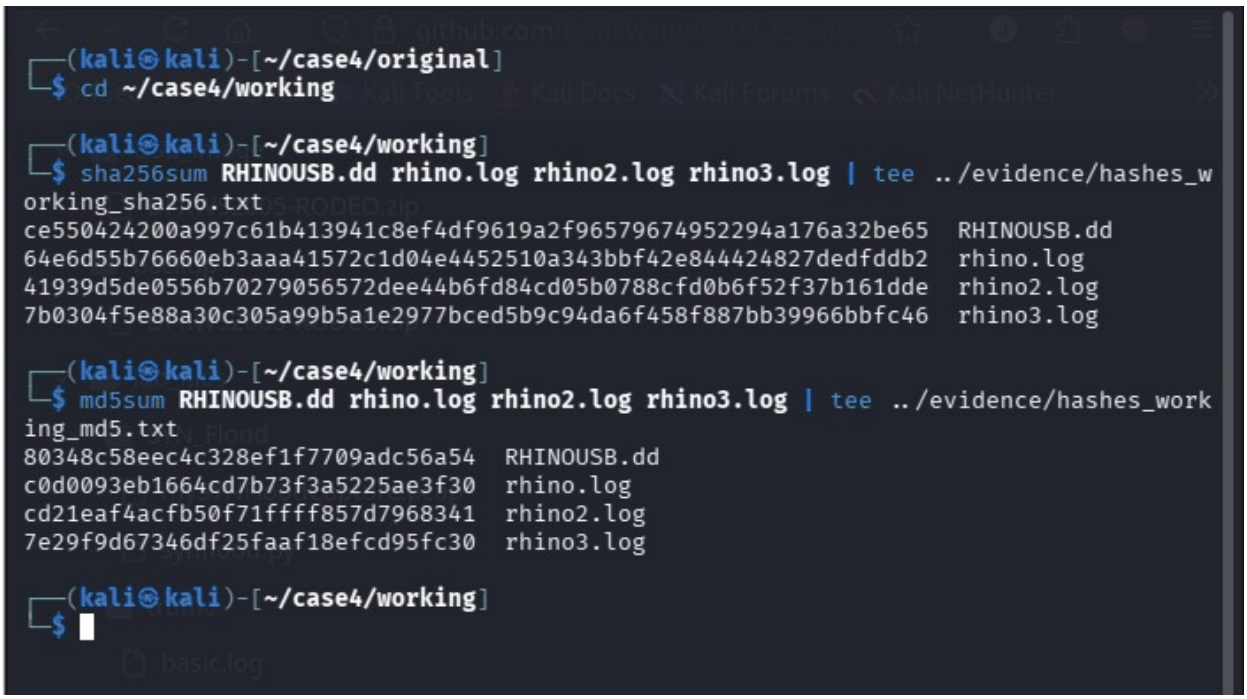


Figure 5. SHA-256 and MD5 of the four extracted working files.

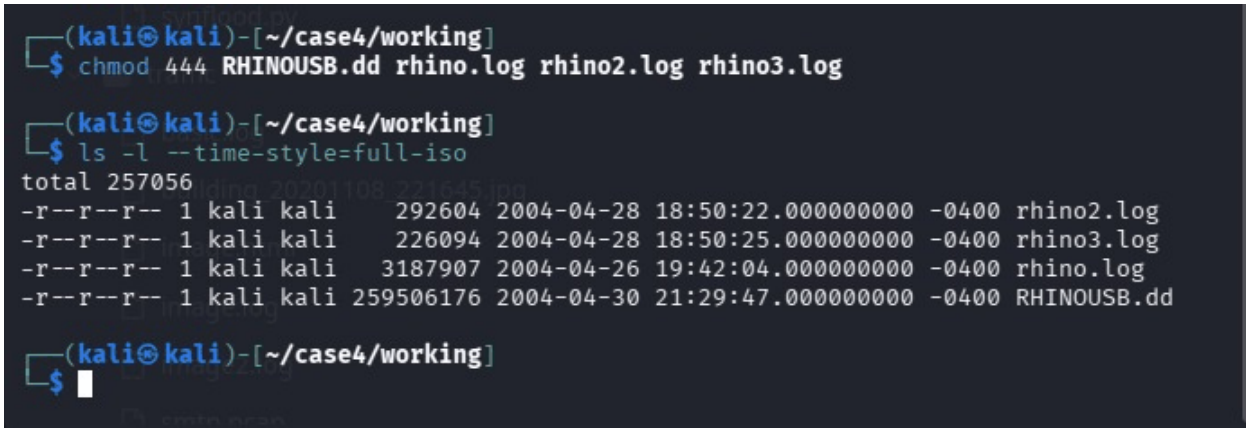


Figure 6. Working files set read-only (chmod 444); sizes and timestamps listed.

File	Size (bytes)	SHA-256	MD5
DFRWS2005-RODEO.zip (original)	3,614,418	e83459a800baed6a3c37551d413ab59fe8dac54c879ed5408001cfed171a0aaa	6a80946a0fe694c12683a91019d6d2ef
RHINOUSB.dd	259,506,176	ce550424200a997c61b413941c8ef4df9619a2f96579674952294a176a32be65	80348c58eec4c328ef1f7709adc56a54
rhino.log	3,187,907	64e6d55b76660eb3aaa41572c1d04e4452510a343bbf42e844424827dedfddb2	c0d0093eb1664cd7b73f3a5225ae3f30
rhino2.log	292,604	41939d5de0556b70279056572dee44b6fd84cd05b0788cfd0b6f52f37b161dde	cd21eaf4acfb50f71ffff857d7968341
rhino3.log	226,094	7b0304f5e88a30c305a99b5a1e2977bced5b9c94da6f458f887bb39966bbfc46	7e29f9d67346df25faaf18efcd95fc30

I compared the four working-file MD5 values with the published reference values and every file reported OK (Figure 7).

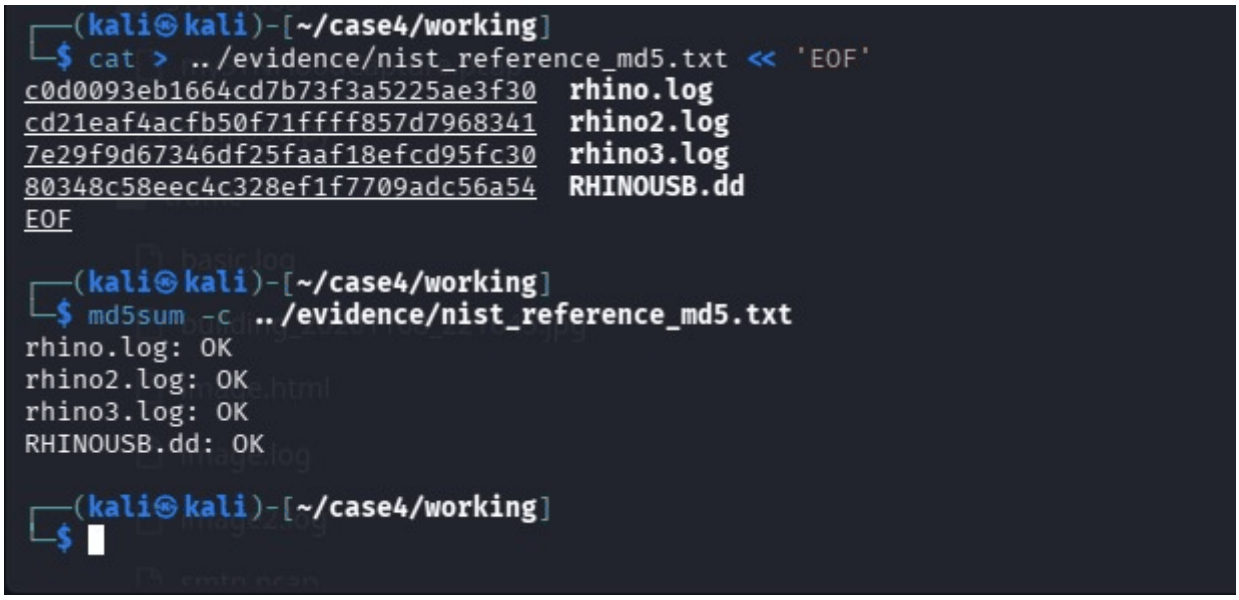


Figure 7. MD5 comparison of the working files with the published reference values: all four report OK.

Interpretation. The matching values show that my working copies are identical to the published case files and were unchanged when I started. Because the originals and working copies are read-only and every later tool ran on copies, the evidence was not altered during my examination. A final re-hash at the end of the work is recommended before disposal.

4. Disk image analysis before recovery (E06 to E10)

4.1 Image type and layout

Observation. `file` identifies RHINOUSB.dd as a FAT (16-bit) volume with 506,848 sectors and serial number 0x4092d9d1 (Figure 8). `mmls` printed nothing and returned **exit code 1** (Figure 9, Figure 10), and `fdisk -l` reports a "dos" label with identifier 0x00000000 but lists no partitions (Figure 11). `img_stat` reports a raw image of 259,506,176 bytes with 512-byte sectors, and `fsstat` reports a FAT16 file system with 0 sectors before it (Figure 12).

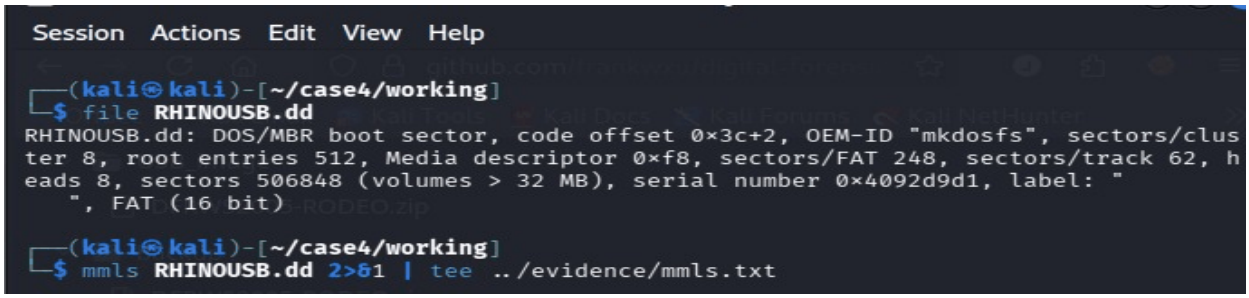


Figure 8. file identifies a FAT16 boot sector (506,848 sectors, serial 0x4092d9d1); mmls run on the image.

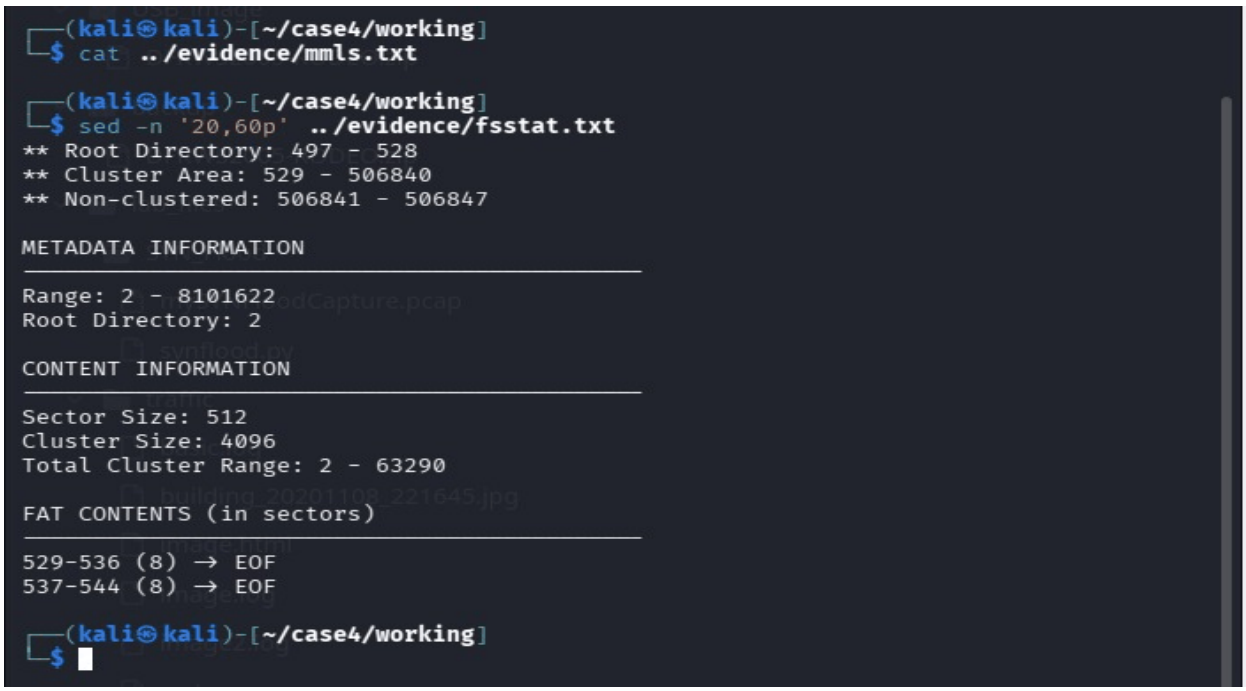


Figure 9. The saved mmls output is empty; fsstat metadata range and FAT chain (two one-cluster files).

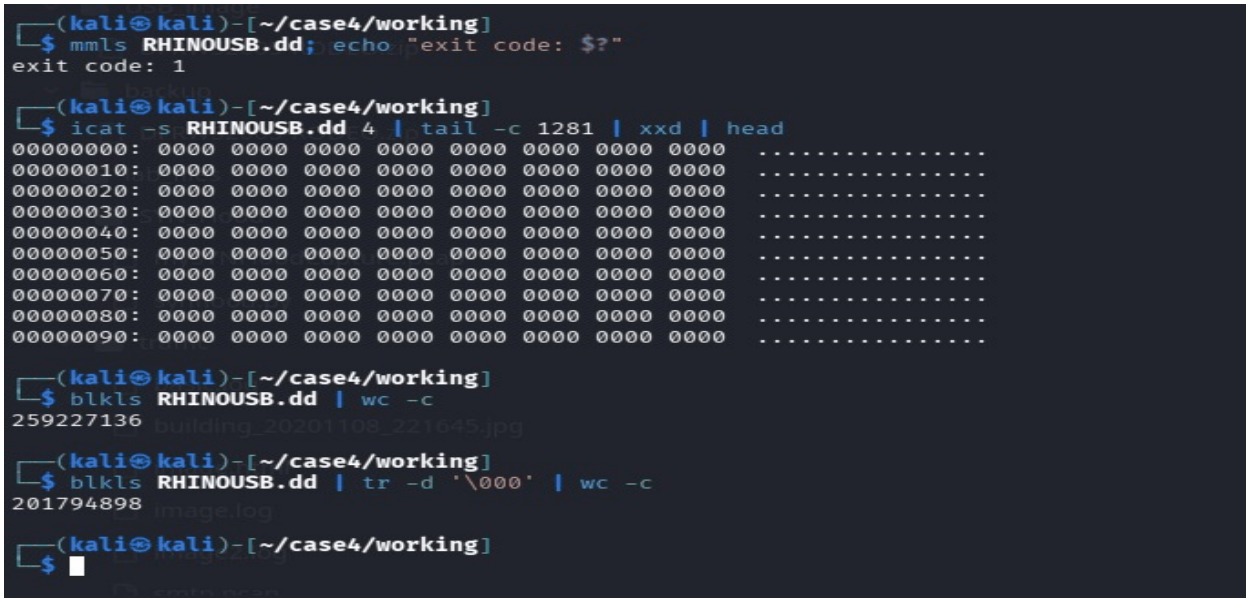


Figure 10. mmls exit code 1 (no partition table); slack check on GUMBO1.TXT; size of unallocated space and count of non-zero bytes.

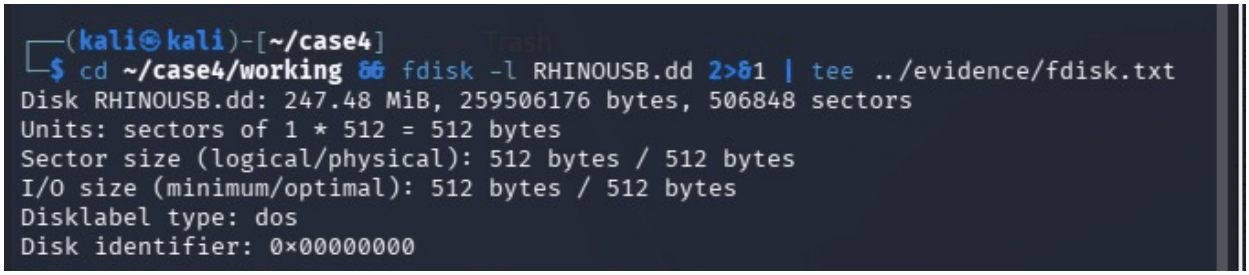


Figure 11. fdisk -l reports a dos disklabel with identifier 0x00000000 and lists no partitions.

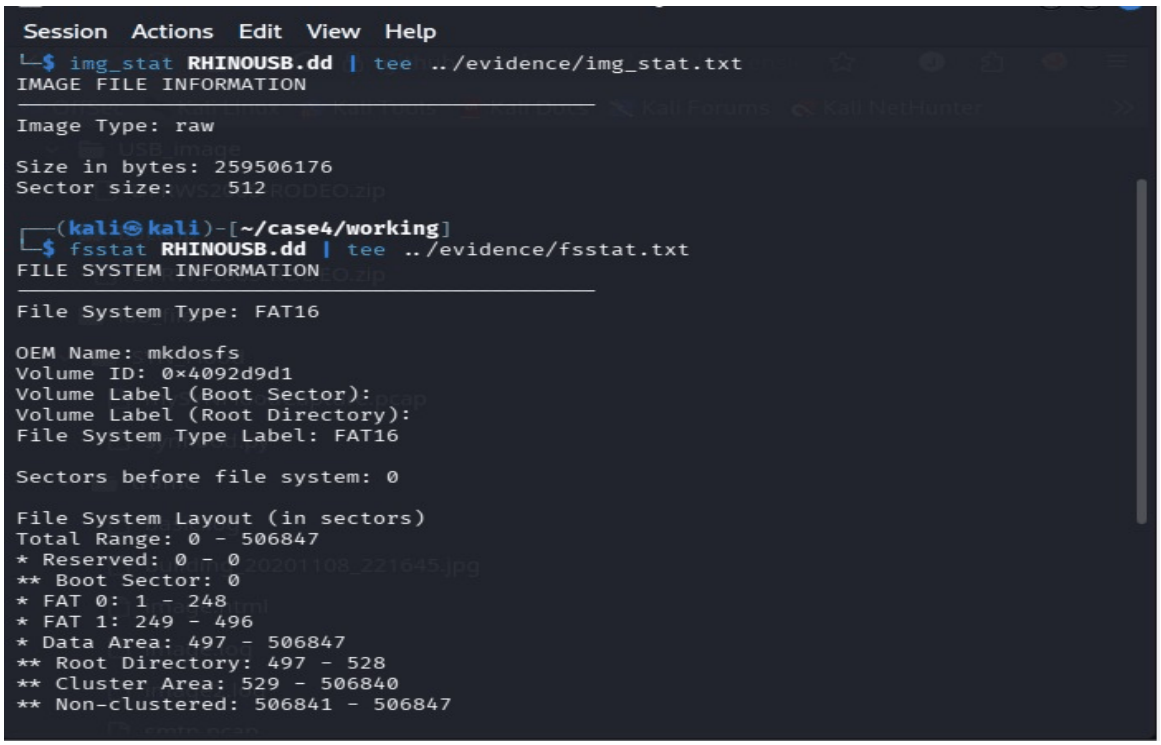


Figure 12. img_stat (raw image, 259,506,176 bytes, 512-byte sectors) and fsstat (FAT16 layout).

Interpretation. The file system starts at sector 0, so the image is one FAT16 volume without a partition table. No offset is needed for the Sleuth Kit tools. The size also checks out: 506,848 sectors × 512 bytes = 259,506,176 bytes.

Property	Value (from fsstat)
File system	FAT16, OEM name `mkdosfs`, volume ID 0x4092d9d1, no volume label
Sector size / cluster size	512 bytes / 4,096 bytes (8 sectors)
FAT 1 and FAT 2	Sectors 1 to 248 and 249 to 496
Root directory	Sectors 497 to 528
Cluster area	Sectors 529 to 506,840 (clusters 2 to 63,290)
Allocated chains	529 to 536 (EOF) and 537 to 544 (EOF): two one-cluster files

4.2 Files, deleted entries and timestamps

Observation. `fls` lists two allocated files, `gumbo1.txt` (directory entry 4, 2,815 bytes) and `gumbo2.txt` (entry 6, 1,293 bytes), plus the virtual entries `$MBR`, `$FAT1`, `$FAT2` and `$OrphanFiles`. The listing of deleted entries, `fls -d`, returned nothing (Figure 13). `istat` shows both files written on 2004-04-30 at 18:11:20 and 18:11:24, with an access date only (00:00:00) and the same creation times (Figure 14, Figure 15). `icat` shows that both files are Louisiana gumbo recipes (Figure 16, Figure 17). I also read the root directory sectors directly with `dd` and `xxd`: they

hold long-name and short-name entries for the two files only, and the decoded date and time fields match `istat` (Figure 18).

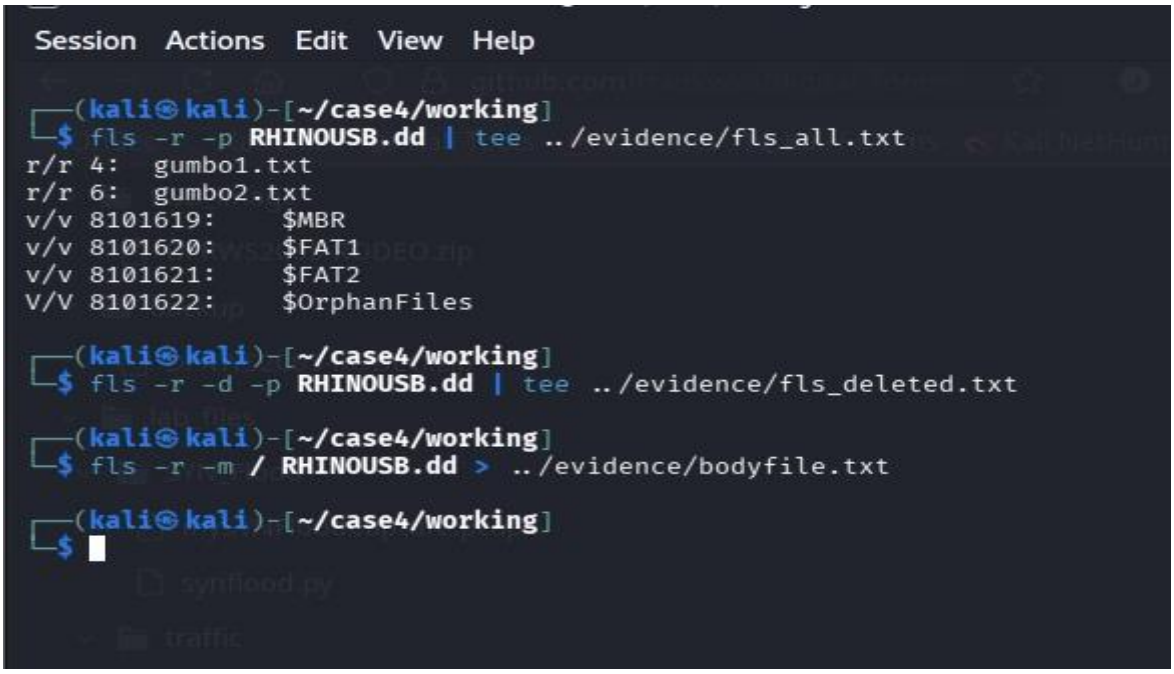


Figure 13. fls: two allocated files and virtual entries; the deleted-entry listing (fls -d) returned nothing.

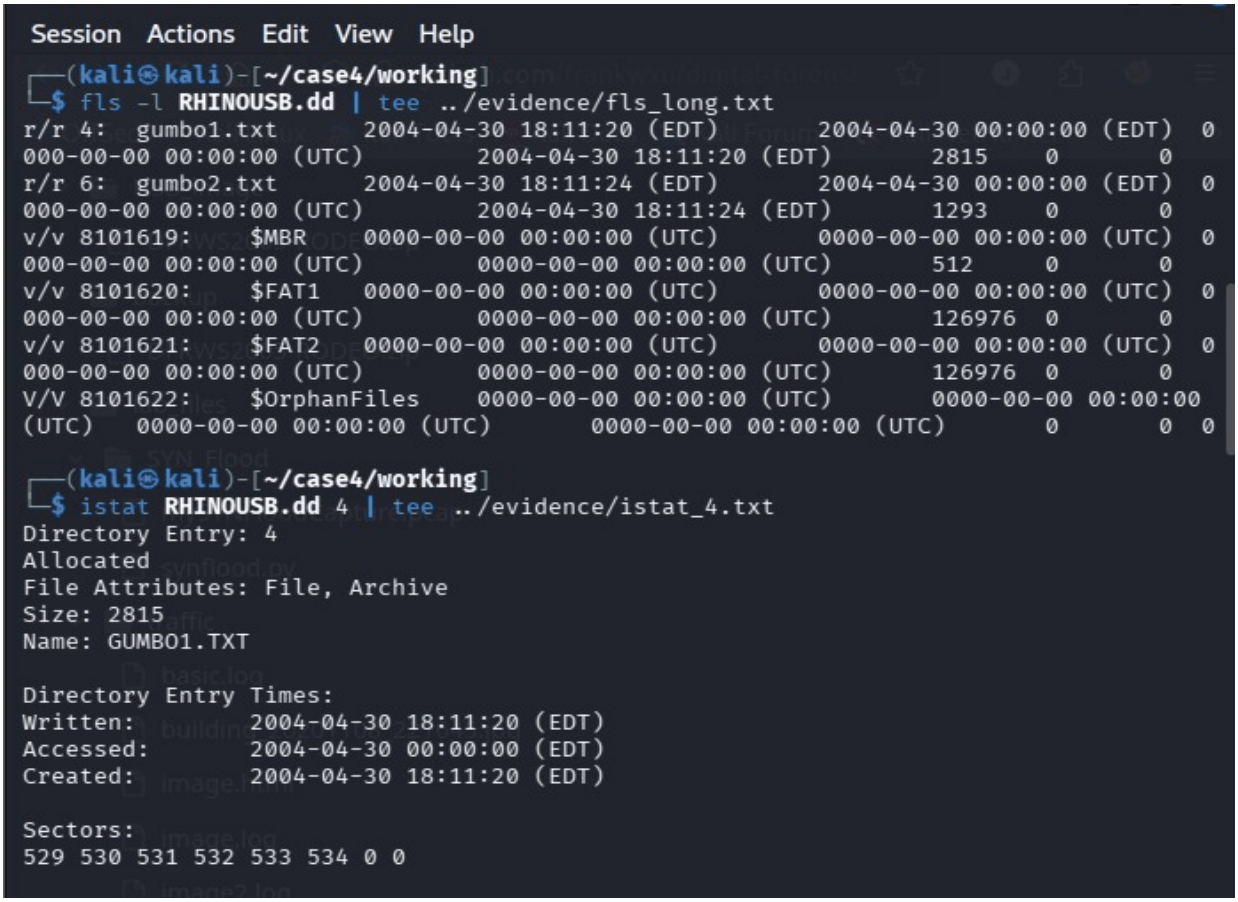


Figure 14. fls -l and istat for directory entry 4 (GUMBO1.TXT).

```
Session  Actions  Edit  View  Help
┌──(kali㉿kali)-[~/case4/working]
└─$ istat RHINOUSB.dd 6 | tee ../evidence/istat_6.txt
Directory Entry: 6
Allocated
File Attributes: File, Archive
Size: 1293
Name: GUMBO2.TXT

Directory Entry Times:
Written:        2004-04-30 18:11:24 (EDT)
Accessed:       2004-04-30 00:00:00 (EDT)
Created:        2004-04-30 18:11:24 (EDT)

Sectors:
537 538 539 0 0 0 0 0
```

Figure 15. istat for directory entry 6 (GUMBO2.TXT).

```
Session  Actions  Edit  View  Help
┌──(kali㉿kali)-[~/case4/working]
└─$ icat RHINOUSB.dd 4 | head -30
SHRIMP AND TASSO GUMBO

Associate Food Editor: Alexis Touchet
Father: Rodney Miller, Abbeville, LA

When I was a kid, our summer routine started with the opening day of
shrimp season ♦ my dad would get the boat ready and out we would go,
winding our way down the bayou until we reached Vermilion Bay. He
would check to make sure the trawl net was securely tied before
tossing it into the water, and would then let it be dragged down
slowly along with the trawl boards. When we painstakingly pulled the
net up onto the boat and untied the bag, it spilled out a catch of
shrimp, crabs, and fish. These fresh shrimp were the stars in Dad's
favorite gumbo recipe.

2 lb large shrimp in shell (21 to 25 per lb), peeled and shells reserved
14 cups water
1/4 cup vegetable oil
1/2 cup all-purpose flour
2 medium onions, chopped
2 celery ribs, chopped
1 large green bell pepper, chopped
1/2 lb fresh or frozen pork tasso* (Cajun-cured smoked meat), thawed if frozen, trim
med and cut into 1/4-inch pieces
1 1/2 teaspoons salt
1/2 teaspoon cayenne
10 oz fresh or frozen baby okra, thawed if frozen, trimmed, and cut into 1/4-inch-th
ick rounds (2 cups)
3/4 cup thinly sliced scallion greens
Accompaniments: cooked white rice; hot pepper sauce
```

Figure 16. icat of GUMBO1.TXT: a recipe text.

```
└─$ icat RHINOUSB.dd 6 | head -30
SHRIMP AND ANDOUILLE SAUSAGE GUMBO

1/2 cup vegetable oil
1/2 cup all purpose flour
4 celery stalks, coarsely chopped
2 medium onions, coarsely chopped
2 green bell peppers, chopped
2 bay leaves
2 teaspoons salt
2 teaspoons dried oregano, crumbled
1/2 teaspoon cayenne pepper
5 8-ounce bottles clam juice
1 28-ounce can plum tomatoes, drained, chopped
1 pound smoked andouille or kielbasa sausage, halved lengthwise,
 sliced 1/4 inch thick
1/2 pound okra, trimmed, cut crosswise into 1/4-inch-thick slices

2 pounds uncooked medium shrimp, peeled, deveined
Freshly cooked long-grain rice
2 tomatoes, seeded, diced

Heat oil heavy large Dutch oven over high heat until almost
smoking. Add flour and stir until dark red-brown, about 8
minutes. Immediately add celery, onions and bell peppers. Cook 5
minutes, stirring and scraping bottom of pan often. Mix in bay leaves,
salt, oregano and cayenne. Add clam juice, canned tomatoes and
sausage. Boil 15 minutes. Add okra, reduce heat and simmer until okra
is tender, about 15 minutes. (Can be made 1 day ahead. Cover and
chill. Bring to simmer before continuing.)
```

Figure 17. icat of GUMBO2.TXT: a recipe text.

```
┌──(kali㉿kali)-[~/case4/working]
└─$ dd if=RHINOUSB.dd bs=512 skip=497 count=4 2>/dev/null | xxd | head -40
00000000: 4167 0075 006d 0062 006f 000f 00ae 3100  Ag.u.m.b.o....1.
00000010: 2e00 7400 7800 7400 0000 0000 ffff ffff  ..t.x.t.........
00000020: 4755 4d42 4f31 2020 5458 5420 0000 6a91  GUMBO1  TXT ..j.
00000030: 9e30 9e30 0000 6a91 9e30 0200 ff0a 0000  .0.0..j..0......
00000040: 4167 0075 006d 0062 006f 000f 0096 3200  Ag.u.m.b.o....2.
00000050: 2e00 7400 7800 7400 0000 0000 ffff ffff  ..t.x.t.........
00000060: 4755 4d42 4f32 2020 5458 5420 0000 6c91  GUMBO2  TXT ..l.
00000070: 9e30 9e30 0000 6c91 9e30 0300 0d05 0000  .0.0..l..0......
00000080: 0000 0000 0000 0000 0000 0000 0000 0000  ................
00000090: 0000 0000 0000 0000 0000 0000 0000 0000  ................
000000a0: 0000 0000 0000 0000 0000 0000 0000 0000  ................
000000b0: 0000 0000 0000 0000 0000 0000 0000 0000  ................
000000c0: 0000 0000 0000 0000 0000 0000 0000 0000  ................
000000d0: 0000 0000 0000 0000 0000 0000 0000 0000  ................
000000e0: 0000 0000 0000 0000 0000 0000 0000 0000  ................
000000f0: 0000 0000 0000 0000 0000 0000 0000 0000  ................
00000100: 0000 0000 0000 0000 0000 0000 0000 0000  ................
00000110: 0000 0000 0000 0000 0000 0000 0000 0000  ................
00000120: 0000 0000 0000 0000 0000 0000 0000 0000  ................
00000130: 0000 0000 0000 0000 0000 0000 0000 0000  ................
00000140: 0000 0000 0000 0000 0000 0000 0000 0000  ................
00000150: 0000 0000 0000 0000 0000 0000 0000 0000  ................
00000160: 0000 0000 0000 0000 0000 0000 0000 0000  ................
00000170: 0000 0000 0000 0000 0000 0000 0000 0000  ................
00000180: 0000 0000 0000 0000 0000 0000 0000 0000  ................
00000190: 0000 0000 0000 0000 0000 0000 0000 0000  ................
000001a0: 0000 0000 0000 0000 0000 0000 0000 0000  ................
000001b0: 0000 0000 0000 0000 0000 0000 0000 0000  ................
000001c0: 0000 0000 0000 0000 0000 0000 0000 0000  ................
000001d0: 0000 0000 0000 0000 0000 0000 0000 0000  ................
```

Figure 18. Raw root-directory sectors (sector 497 onward) read with dd and xxd: entries for GUMBO1.TXT and GUMBO2.TXT only.

Interpretation. The visible file system holds only two unrelated text files and no deleted entries. The FAT "access" time has date resolution only, so 00:00:00 is normal and is not a clue. FAT times have no time zone and 2-second resolution, so they are raw local times of an unknown zone.

4.3 Unallocated space

Observation. `blkls` outputs 259,227,136 bytes of unallocated space. After deleting NUL bytes, 201,794,898 bytes remain (Figure 10). The unallocated blocks are 506,303 sectors, and the difference from the 506,848 total equals the boot sector, both FATs, the root directory and the two allocated clusters (545 sectors). My slack check on `gumbo1.txt` showed zeros in the part I displayed.

Interpretation. The unallocated area was **not wiped with zeros**: about 78% of it is non-zero data, so recovery by carving is worthwhile. The OEM name `mkdosfs` is an observation about the tool that wrote the boot sector. I draw no conclusion from it alone about when or why the volume was formatted.

5. Recovery of deleted and unallocated files (E11 to E16)

5.1 PhotoRec runs

Observation. I ran PhotoRec 7.2 on the read-only working copy with logging turned on (Figure 19). It listed the image as read-only and showed one FAT16 partition covering the whole disk, 506,848 sectors (Figure 20). I chose the file-system type "Other" (Figure 21) PhotoRec reported **132 files saved** (Figure 22).

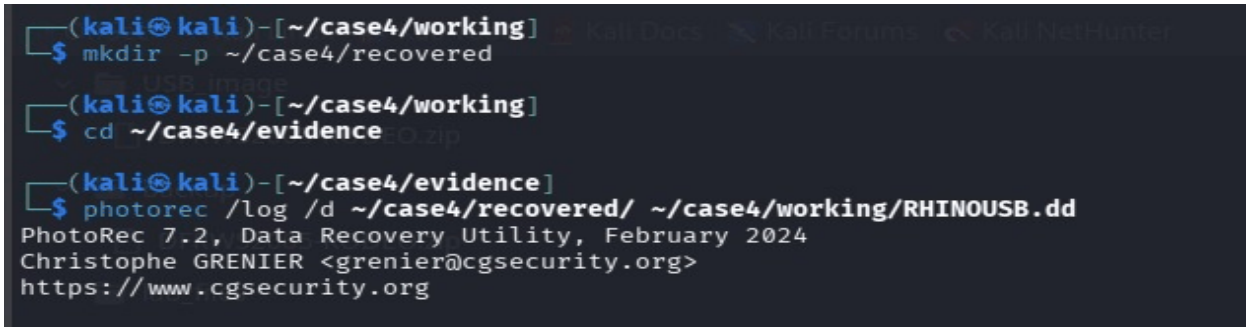


Figure 19. PhotoRec started on the working copy with logging and an output folder.

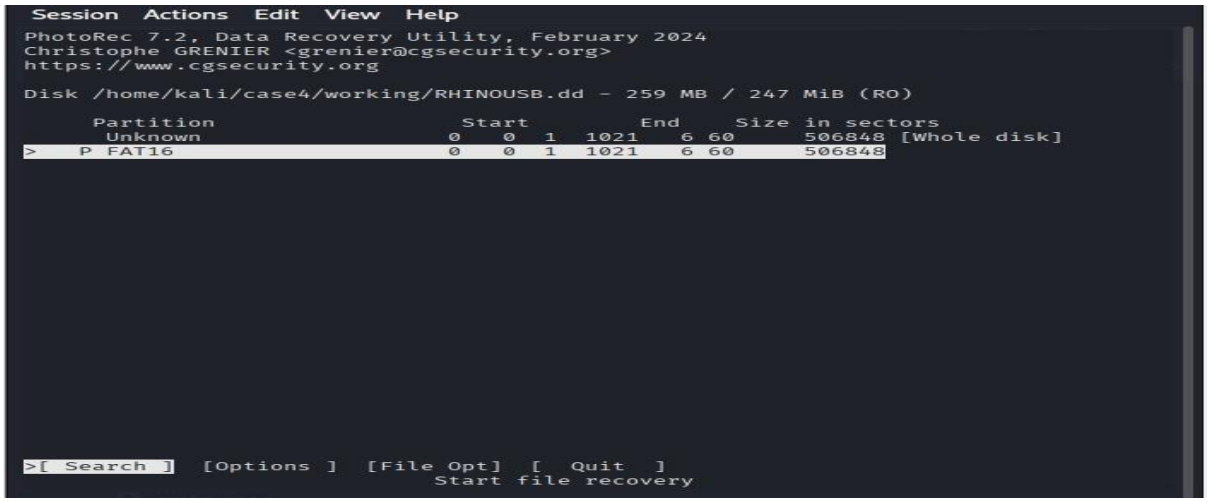


Figure 20. PhotoRec partition selection: FAT16, whole disk of 506,848 sectors.

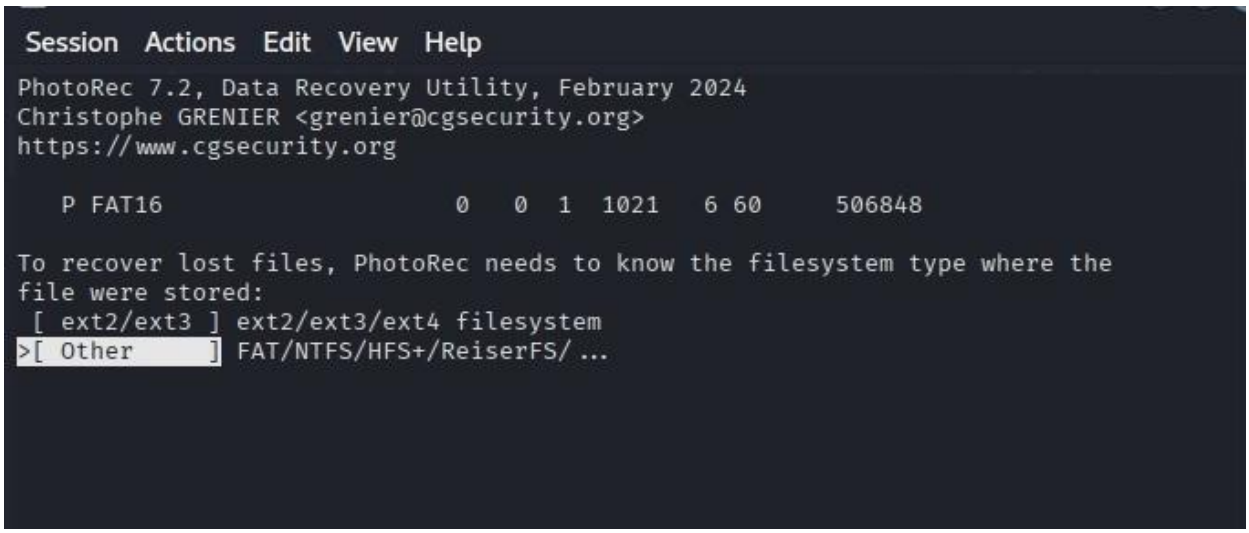


Figure 21. File-system type "Other" (FAT/NTFS/HFS+/ReiserFS) selected.

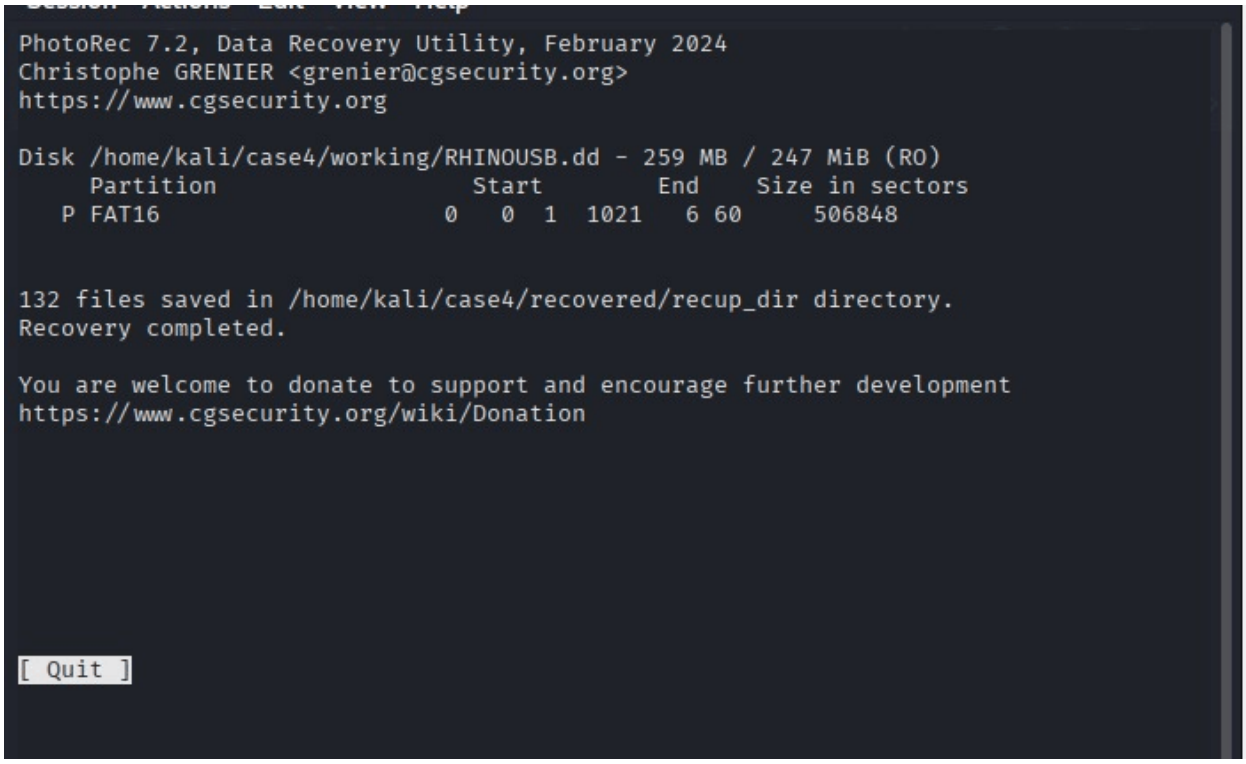


Figure 22. PhotoRec completed: 132 files saved.

I ran PhotoRec twice, which created `recup_dir.1` and `recup_dir.2`. Each folder holds 133 files: 132 carved files plus PhotoRec's own `report.xml`. Across both folders there are 266 files: 244 txt, 14 jpg, 4 gif, 2 doc and 2 xml (Figure 23). A comparison of the SHA-256 listing of the two folders shows that **only `report.xml` differs** (Figure 25).

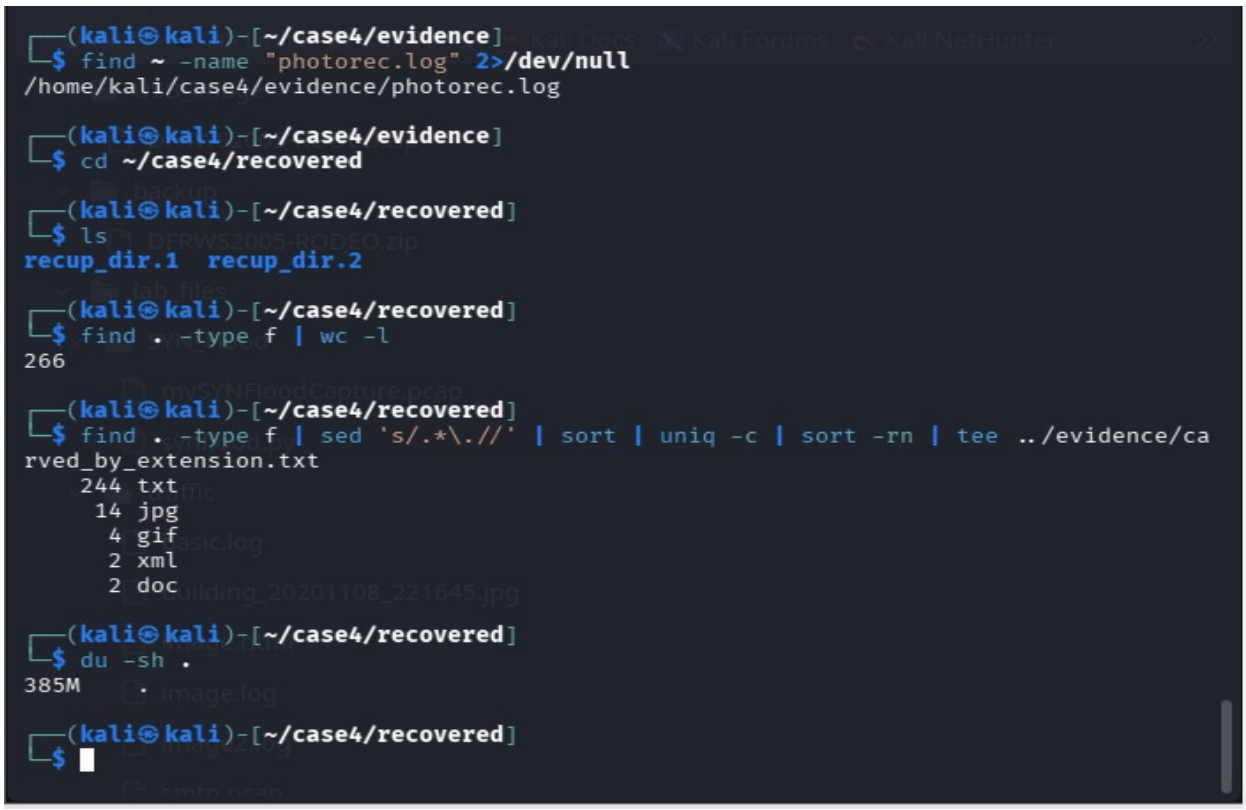


Figure 23. PhotoRec log located; two output folders; 266 files in total; counts by extension and output size.

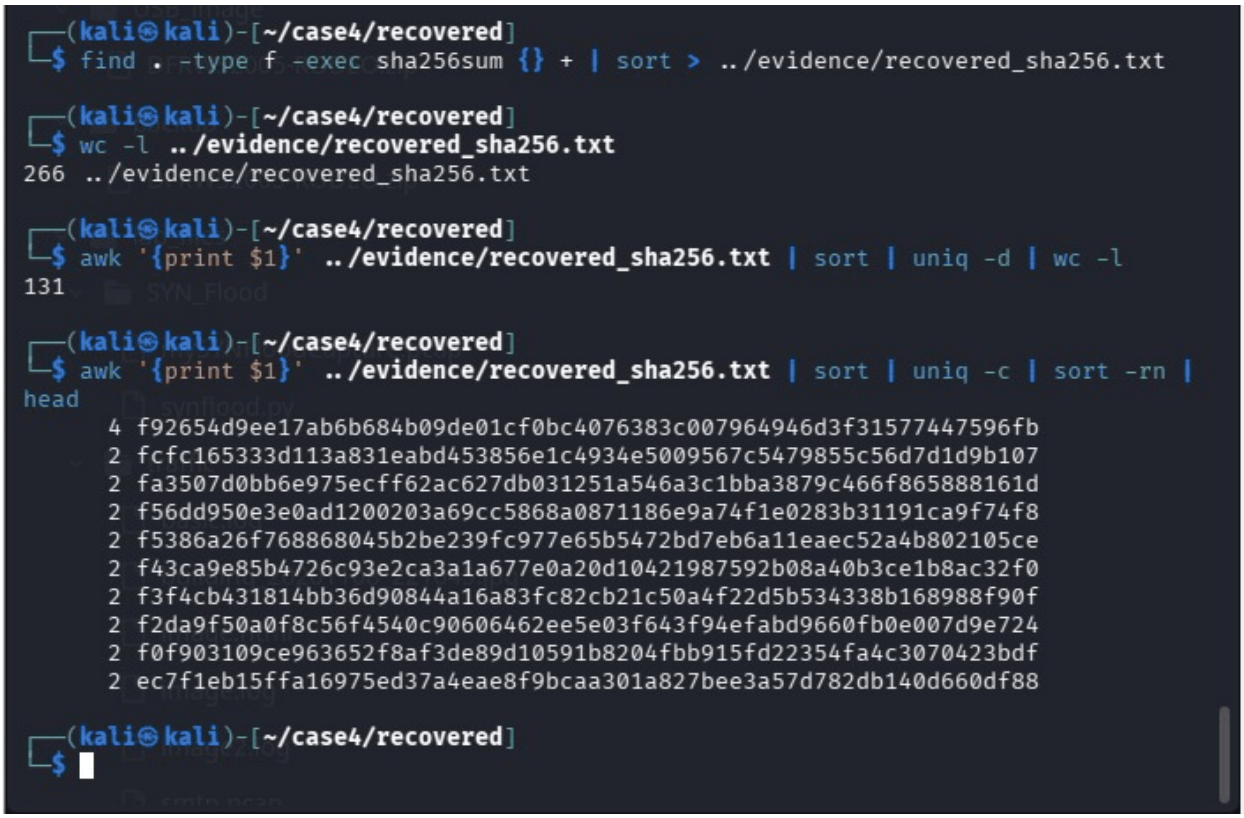


Figure 24. SHA-256 of every recovered file and duplicate-hash analysis.



Figure 25. 133 files in each output folder; the two folders differ only in report.xml; 133 unique hashes overall.

Interpretation. Two runs on the same input gave byte-identical carved content, so the recovery is reproducible. I treat `recup_dir.1` as the primary set. Per run, the 132 carved files are 122 txt, 7 jpg, 2 gif and 1 doc. Across both folders there are 133 unique hashes: 131 carved contents plus the two different `report.xml` files. Within one run, 132 carved files reduce to 131 unique contents, because exactly one pair is identical: `f0105873.jpg` and `f0335081.jpg` (same SHA-256, 264,600 bytes, found at sectors 105873 and 335081).

5.2 Carved image files

Observation. PhotoRec names each carved file after the sector where it starts. I converted each start sector to a byte offset (sector × 512) and read the original image at that offset with `xxd`. Every offset holds a JPEG signature (`FFD8FFE0`) or a GIF signature (`GIF8`) (Figure 29). `blkstat` reports every one of the nine start sectors as **Not Allocated** (Figure 30). I copied the nine images to an analysis folder and hashed them (Figure 28). `file` gives their type, dimensions and any comment (Figure 26, Figure 27).

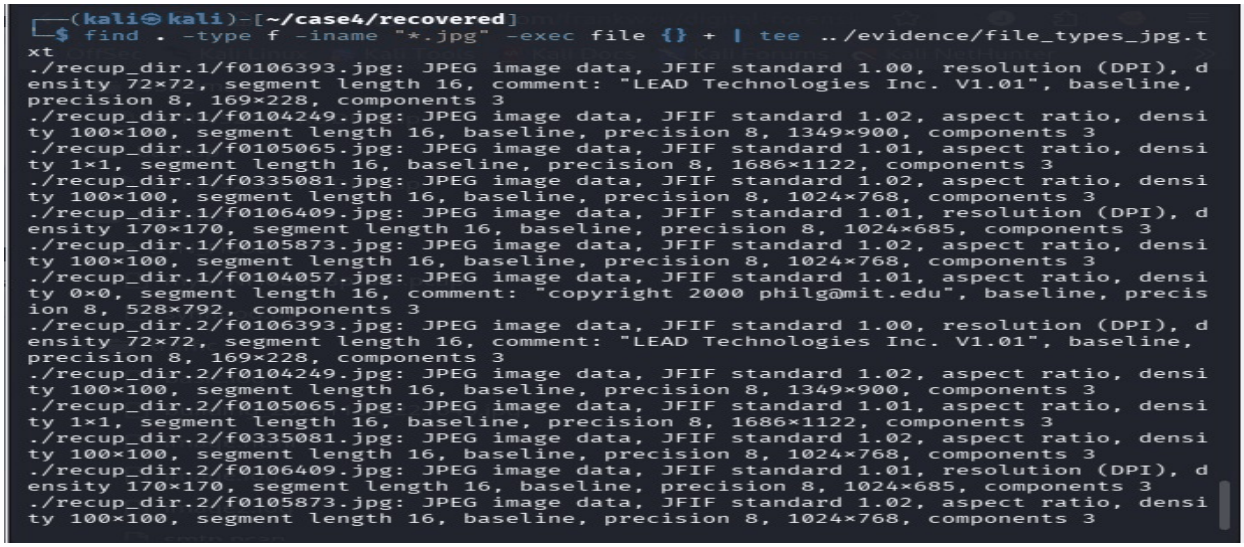


Figure 26. file output for the recovered JPEG files (dimensions, JFIF versions, comments).



Figure 27. file output for the recovered GIF files.

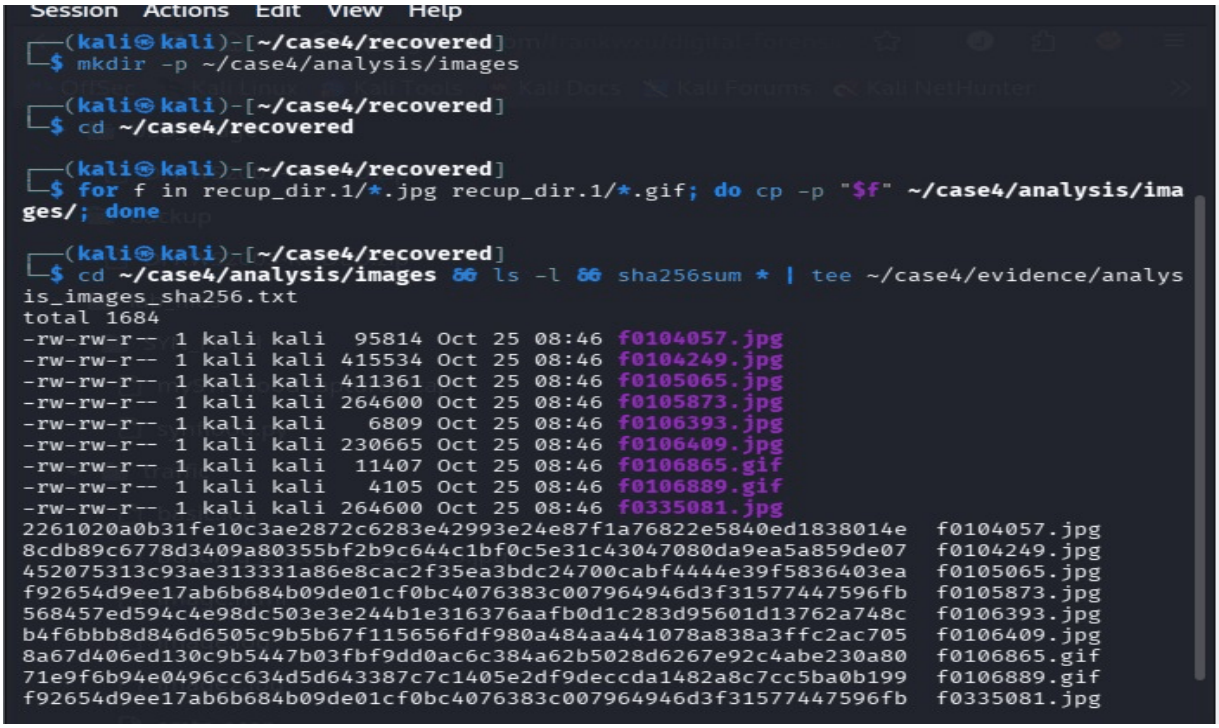


Figure 28. Recovered images copied for analysis and hashed with SHA-256.

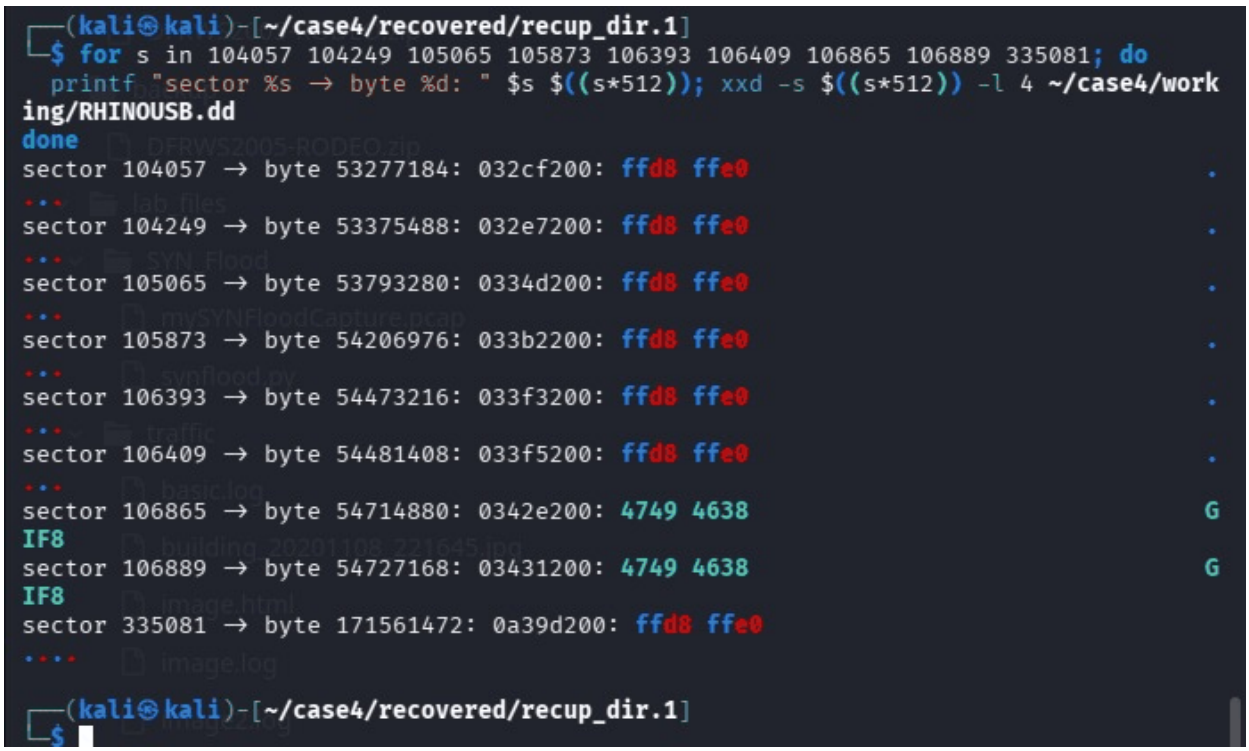


Figure 29. Carved-file start sectors converted to byte offsets in the original image; JPEG (FFD8FFE0) or GIF (GIF8) headers found at each offset.

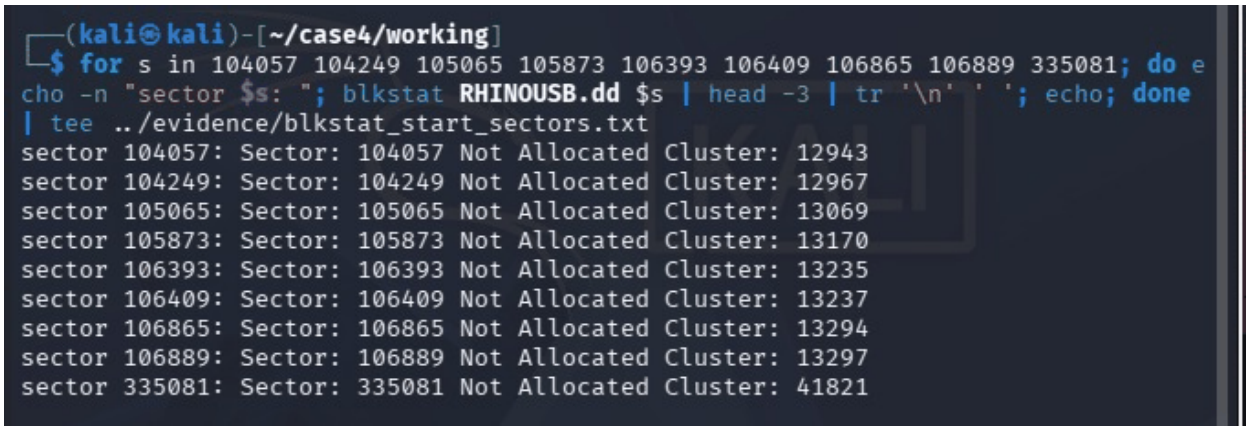


Figure 30. blkstat on each start sector: all nine are "Not Allocated" (cluster numbers shown).

File	Start sector	Byte offset	Size (bytes)	Type and dimensions	SHA-256 (first 16)
f0104057.jpg	104057	53,277,184	95,814	JPEG 528×792, comment "copyright 2000 philg@mit.edu"	2261020a0b31fe10
f0104249.jpg	104249	53,375,488	415,534	JPEG 1349×900	8cdb89c6778d3409
f0105065.jpg	105065	53,793,280	411,361	JPEG 1686×1122	452075313c93ae31
f0105873.jpg	105873	54,206,976	264,600	JPEG 1024×768	f92654d9ee17ab6b
f0106393.jpg	106393	54,473,216	6,809	JPEG 169×228, comment "LEAD Technologies Inc. V1.01"	568457ed594c4e98

f0106409.jpg	106409	54,481,408	230,665	JPEG 1024×685	b4f6bbb8d846d650
f0106865.gif	106865	54,714,880	11,407	GIF89a 290×246	8a67d406ed130c9b
f0106889.gif	106889	54,727,168	4,105	GIF89a 150×87	71e9f6b94e0496cc
f0335081.jpg	335081	171,561,472	264,600	JPEG 1024×768 (identical to f0105873.jpg)	f92654d9ee17ab6b

Interpretation. The nine image files come from space that no directory entry points to. That is consistent with content that was deleted or that remained after the volume was recreated. I cannot say when it was written or removed, or by whom.

5.3 Carved Word document

Observation. PhotoRec carved one Word document, whose name PhotoRec took from its title (Figure 31). ExifTool shows the title "She died in February at the age of 74", the author and last-saved-by fields filled with a repeated joke phrase, the company "University of New Orleans", 3 pages, 1,022 words, 9 revisions and 19 minutes of editing time (Figure 32, Figure 33). The creation and save times are in **August 2005**. `file` shows 03:17 and 03:40, while ExifTool shows 02:17 and 02:40 for the same fields. The two tools handle the time zone differently, so I cite the tool whenever I give these values. PhotoRec also set the carved file's own modification time from the document's internal date, whereas the other carved files carry the time of my recovery run.

The body text is mostly personal diary-style writing. Near the end, the writer comments on rhino pictures and says that fewer than ten photos is "no big deal". They also say they "zapped the hard drive" and threw it into the Mississippi River, that they plan to reformat a USB key after the entry while trying to keep "the good stuff", and that they need to change the password on a "gnome" account that someone named Jeremy gave them (Figure 34). A search for "rhino" in the first recovery folder matched only this document and `report.xml`; the `report.xml` match is a false positive, because that file records the image path containing "RHINOUSB".

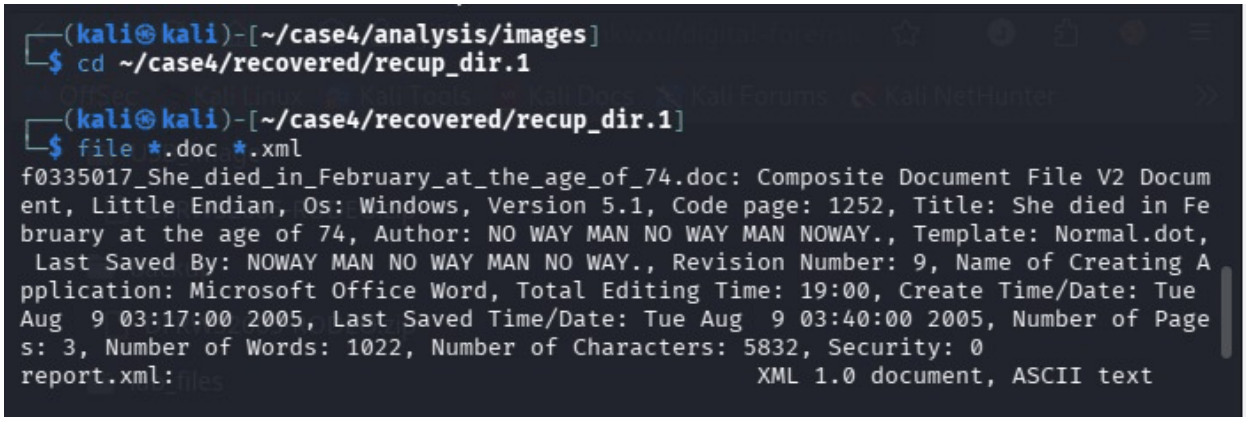


Figure 31. file output for the carved Word document and report.xml.

```
┌──(kali㉿kali)-[~/case4/recovered/recup_dir.1]
└─$ exiftool *.doc *.xml
======== f0335017_She_died_in_February_at_the_age_of_74.doc
ExifTool Version Number         : 13.50
File Name                       : f0335017_She_died_in_February_at_the_age_of_74.doc
Directory                       : .
File Size                       : 31 kB
File Modification Date/Time     : 2005:08:08 22:40:00-04:00
File Access Date/Time           : 2026:10:25 08:55:19-04:00
File Inode Change Date/Time     : 2026:10:25 08:46:53-04:00
File Permissions                : -rw-rw-r--
File Type                       : DOC
File Type Extension             : doc
MIME Type                       : application/msword
Identification                  : Word 8.0
Language Code                   : English (US)
Doc Flags                       : 1Table, ExtChar
System                          : Windows
Word 97                         : No
Title                           : She died in February at the age of 74
Subject                         :
Author                          : NO WAY MAN NO WAY MAN NOWAY.
Keywords                        :
Comments                        :
Template                        : Normal.dot
Last Modified By                : NOWAY MAN NO WAY MAN NO WAY.
Software                        : Microsoft Office Word
Create Date                     : 2005:08:09 02:17:00
Modify Date                     : 2005:08:09 02:40:00
Security                        : None
Code Page                       : Windows Latin 1 (Western European)
Company                         : University of New Orleans
```

Figure 32. ExifTool: metadata of the carved Word document (first part).

```
Session  Actions  Edit  View  Help
Company                         : University of New Orleans
Char Count With Spaces          : 6841
App Version                     : 11.6360
Scale Crop                      : No
Links Up To Date                : No
Shared Doc                      : No
Hyperlinks Changed              : No
Title Of Parts                  : She died in February at the age of 74
Heading Pairs                   : Title, 1
Comp Obj User Type Len          : 31
Comp Obj User Type              : Microsoft Office Word Document
Last Printed                    : 0000:00:00 00:00:00
Revision Number                 : 9
Total Edit Time                 : 19 minutes
Words                           : 1022
Characters                      : 5832
Pages                           : 3
Paragraphs                      : 13
Lines                           : 48
======== report.xml
ExifTool Version Number         : 13.50
File Name                       : report.xml
Directory                       : .
File Size                       : 28 kB
File Modification Date/Time     : 2026:10:25 08:46:53-04:00
File Access Date/Time           : 2026:10:25 08:55:19-04:00
File Inode Change Date/Time     : 2026:10:25 08:46:53-04:00
File Permissions                : -rw-rw-r--
File Type                       : XML
File Type Extension             : xml
MIME Type                       : application/xml
Dfxml Xmloutputversion          : 1.0
```

Figure 33. ExifTool: remainder of the Word document metadata, and the PhotoRec report.xml.

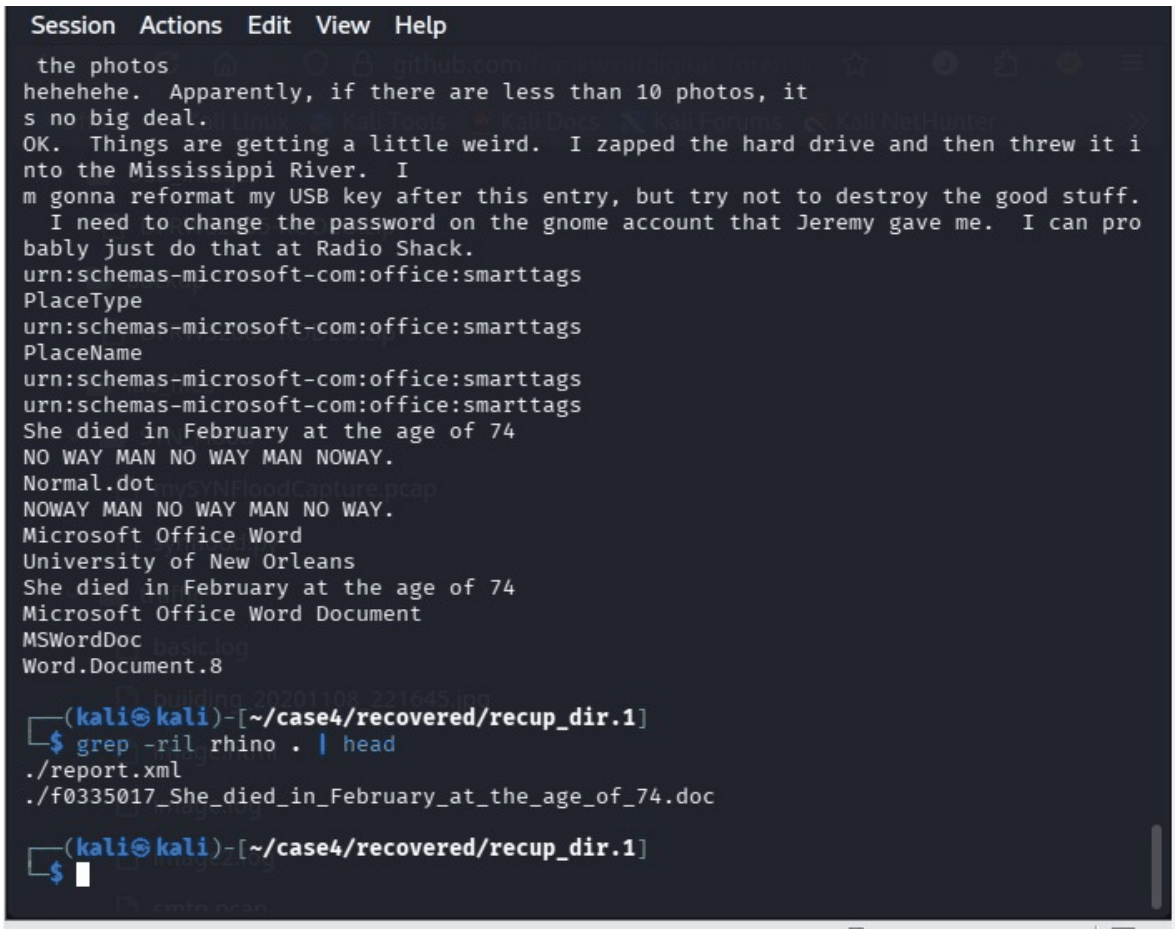


Figure 34. Text recovered from the Word document (end of file) and a search for "rhino" in the first recovery folder.

Interpretation. These are claims written in a file, not verified facts. The author and the writing date cannot be established: the metadata fields can be edited, and the embedded dates (2005) are later than the 2004 dates in the other evidence. I use the document only as a lead. In the correlation section I test two of its claims against independent evidence: the "gnome" account and the reformatting of the USB key.

5.4 Large text files and keyword search

Observation. The two largest carved text files are `f0106905.txt` (116,793,344 bytes) and `f0000545.txt` (52,998,144 bytes). The first begins with a repeating line "CHARLIE" and contains no NUL bytes, and `file` mislabels it as "SIMH tape data" (Figure 35, Figure 36). The second begins with a repeating line "SORRY" (Figure 37). Both sizes divide exactly by the line length (8 and 6 bytes). A case-insensitive search of the carved text files for `password|ftp|gnome|steg|hide|jpg` found nothing (Figure 38).

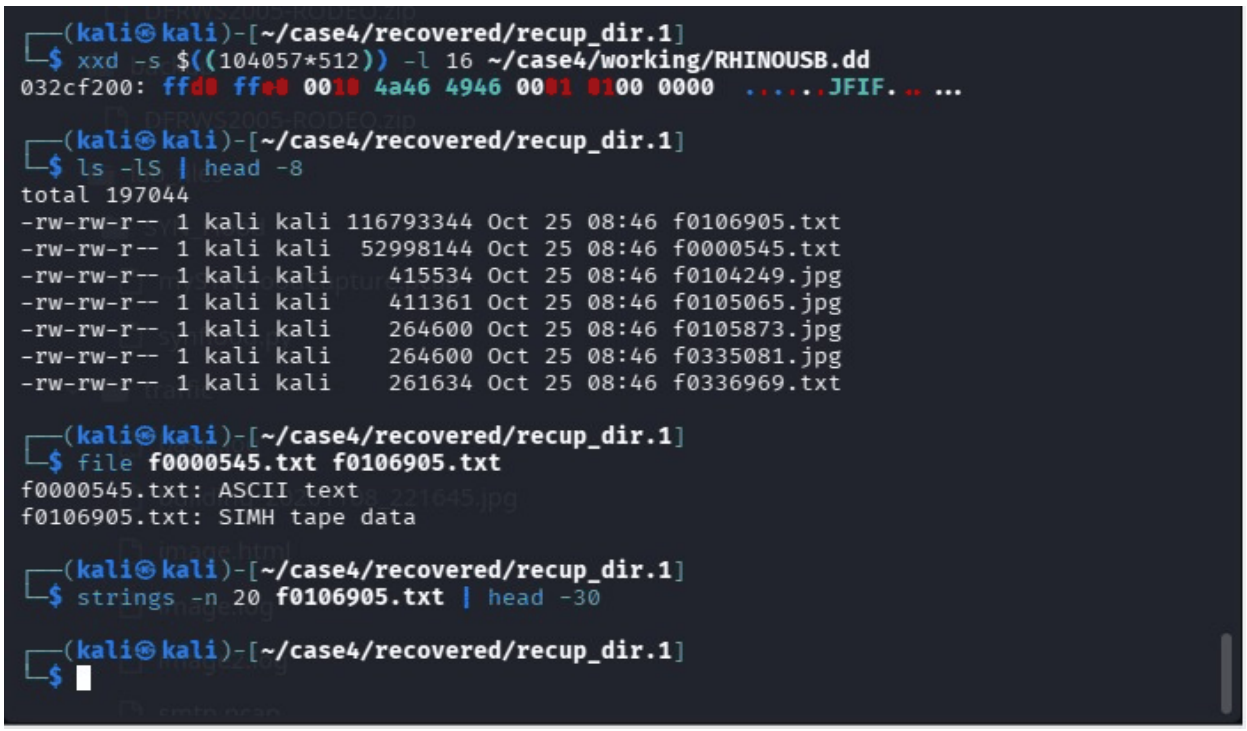


Figure 35. Header bytes at sector 104057; the largest carved text files; file(1) and strings results.

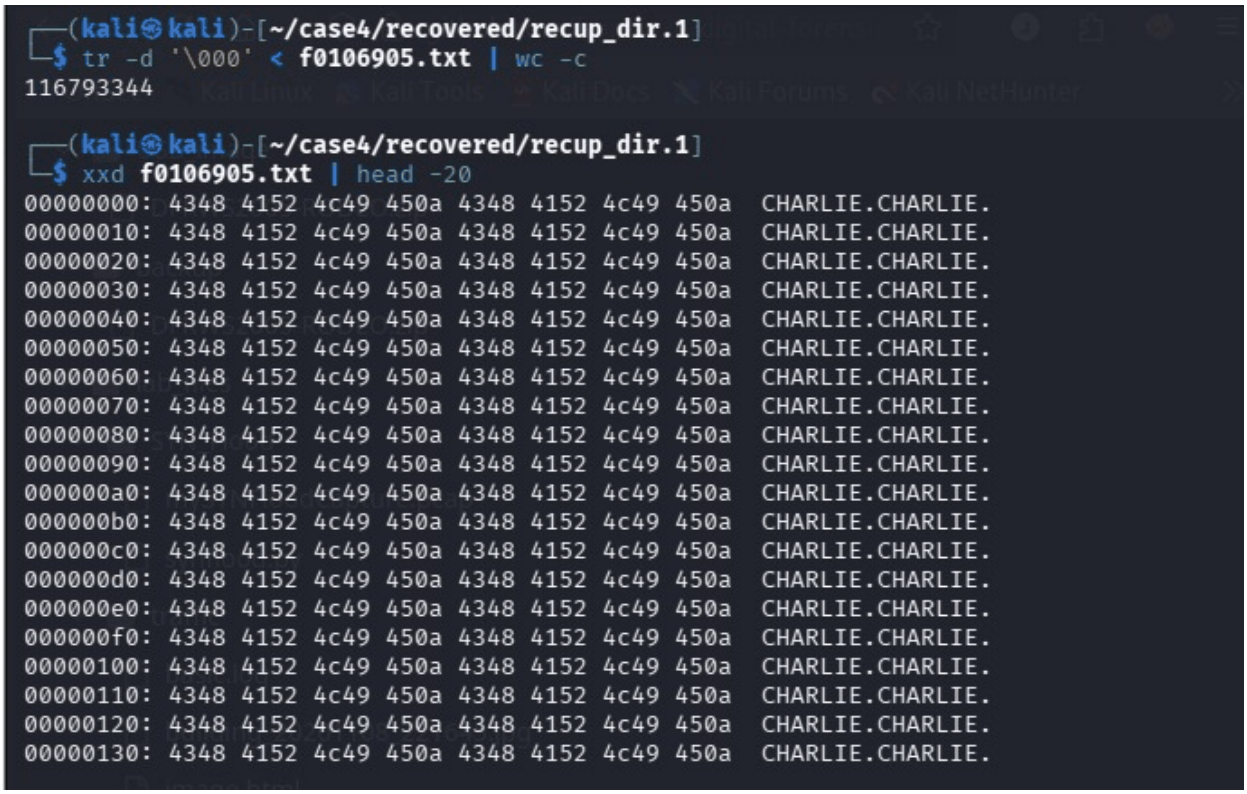


Figure 36. f0106905.txt: non-NUL byte count and a repeating "CHARLIE" line pattern.

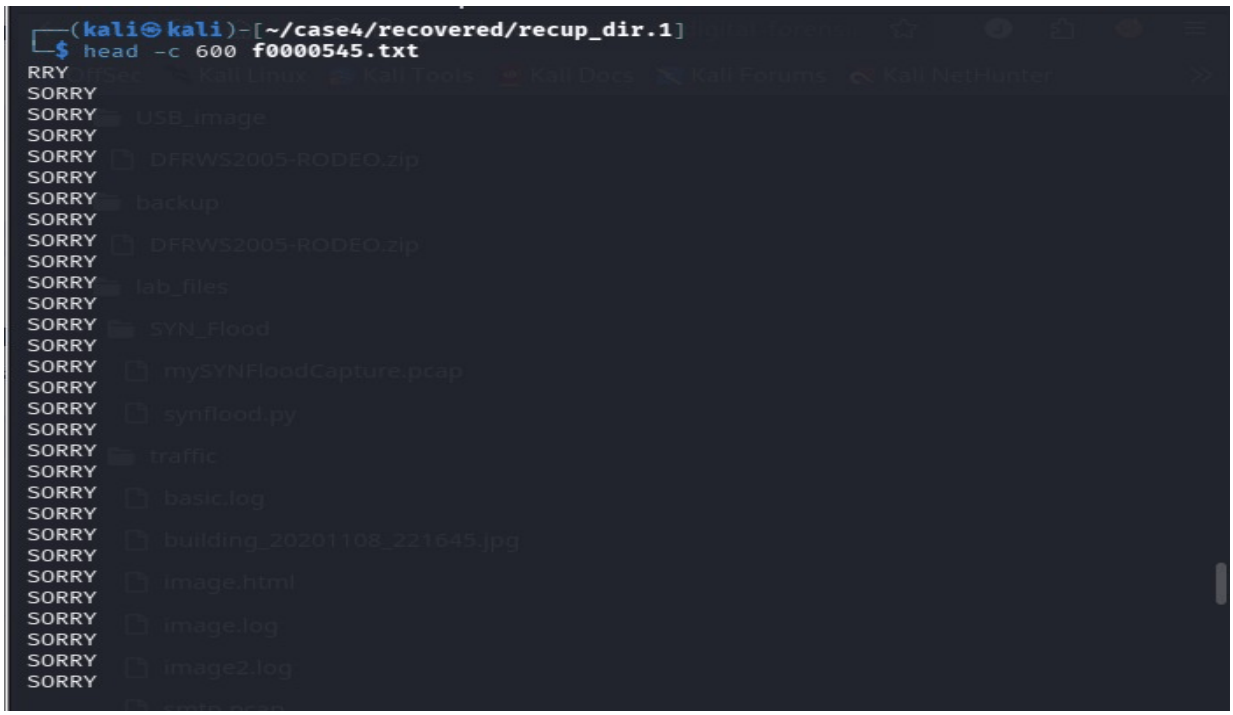


Figure 37. f0000545.txt: a repeating "SORRY" line pattern.

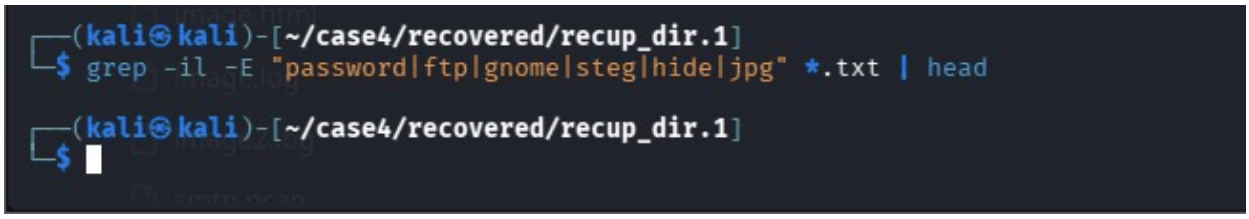


Figure 38. Keyword search of the carved text files: no matches.

Interpretation. On what I viewed, these files are filler, not hidden content. Together they account for about 169.8 MB of the 201.8 MB of non-zero unallocated bytes, if both came from unallocated space. I viewed only the start of each file, so I do not claim that the whole of each file is filler. My keyword search covered ASCII text in the text files only, so a negative result is not proof of absence.

6. Hidden content: steganography (E17 to E20)

6.1 Metadata review of the JPEG files

Observation. I copied the seven recovered JPEG files into a separate folder and verified them against the recorded hashes (seven OK, Figure 39). A targeted ExifTool review of comment, copyright, artist, software, description, user comment and warning fields found two comments and no warnings: `f0104057.jpg` carries "copyright 2000 philg@mit.edu" and `f0106393.jpg` carries "LEAD Technologies Inc. V1.01" (Figure 40).



Figure 39. The seven JPEG carrier copies verified against the recorded hashes (7 OK).



Figure 40. Metadata review of the JPEG files: only two comment fields are present; no warnings.

Interpretation. These read as origin tags from a photographer or an image library. I record them as observations and draw no conclusion from them.

6.2 Tool preparation and provenance

I built stegdetect from source, using commit `46a09e93320cd63b6731e0bc5b22c11c588a0e81` (Figure 41). The first build failed because the top-level compile used the current compiler's default C23 standard, in which `false` is a reserved word and stegdetect uses it as a variable name (Figure 42). Passing the old-C flags to `make` itself fixed it, and `./stegdetect -V` reports version 0.6.1 (Figure 43). For reproducibility, the build command was `linux32 make CFLAGS="-std=gnu89 -fcommon -g -O2 -Wno-error=implicit-function-declaration -Wno-error=int-conversion -Wno-error=incompatible-pointer-types"`.

```
┌──(kali㉿kali)-[~/case4/steg]
└─$ mkdir -p ~/case4/tools && cd ~/case4/tools

┌──(kali㉿kali)-[~/case4/tools]
└─$ git clone https://github.com/poizan42/stegdetect
Cloning into 'stegdetect'...
remote: Enumerating objects: 419, done.
remote: Total 419 (delta 0), reused 0 (delta 0), pack-reused 419 (from 1)
Receiving objects: 100% (419/419), 1.38 MiB | 985.00 KiB/s, done.
Resolving deltas: 100% (59/59), done.

┌──(kali㉿kali)-[~/case4/tools]
└─$ cd stegdetect

┌──(kali㉿kali)-[~/case4/tools/stegdetect]
└─$ git rev-parse HEAD | tee ~/case4/evidence/stegdetect_commit.txt
46a09e93320cd63b6731e0bc5b22c11c588a0e81

┌──(kali㉿kali)-[~/case4/tools/stegdetect]
└─$ sudo apt install -y dh-autoreconf build-essential
[sudo] password for kali:
Sorry, try again.
[sudo] password for kali:
build-essential is already the newest version (12.12).
build-essential set to manually installed.
The following packages were automatically installed and are no longer required:
  libimath-3-1-29t64  libpython3.13-dev  python3.13          rpcsvc-proto
  libmbedcrypto16     libraw23t64        python3.13-dev
  libpostal-data      libvidstab1.1      python3.13-minimal
  libpostal1          libzxing3          python3.13-venv
Use 'sudo apt autoremove' to remove them.
```

Figure 41. stegdetect source cloned and the commit recorded as tool provenance.

```
┌──(kali㉿kali)-[~/case4/tools/stegdetect]
└─$ linux32 make 2>&1 | tail -30
 1049 |             fprintf(stdout, " %s(%d)<[%s][", what,
      |                                    ~^
      |                                     |
      |                                     int
      |                                    %ld
 1050 |                 buflen, is_random(buf, buflen) ? "random" : "nonrandom");
      |                 ~~~~~~
      |                 |
      |                 size_t {aka long unsigned int}
stegdetect.c: In function ‘detect’:
stegdetect.c:1209:37: warning: pointer targets in passing argument 1 of ‘strcmp’ dif
fer in signedness [-Wpointer-sign]
 1209 |                 !strcmp(comments[0], "JPEG Encoder Copyright 1998, James
 R. Weeks and BioElectroMech.")) {
      |                         ~~~~~~~~^~
      |                                 |
      |                                 u_char * {aka unsigned char *}
In file included from stegdetect.c:40:
/usr/include/string.h:171:32: note: expected ‘const char *’ but argument is of type
‘u_char *’ {aka ‘unsigned char *’}
  171 | extern int strcmp (const char *__s1, const char *__s2)
      |                    ~~~~~~~~~~~~^~~~
stegdetect.c:1341:35: warning: variable ‘step’ set but not used [-Wunused-but-set-va
riable]
 1341 |         int i, j, n, off, step;
      |                           ^~~~
stegdetect.c:1145:13: warning: variable ‘a_wasted_var’ set but not used [-Wunused-bu
t-set-variable]
 1145 |         int a_wasted_var;
      |             ^~~~~~~~~~~
```

Figure 42. Build failure: the top-level compile used the default C23 standard.

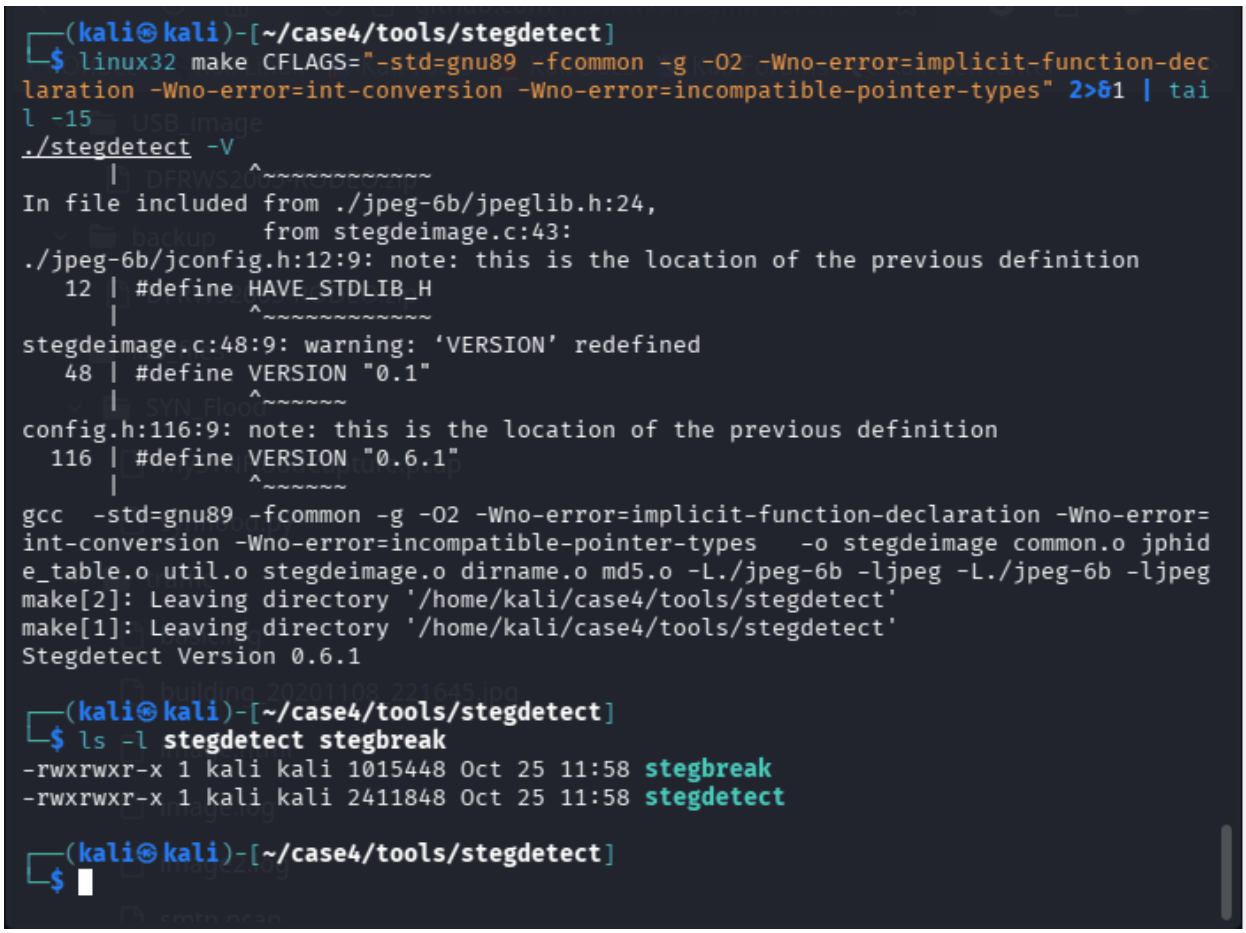


Figure 43. stegdetect 0.6.1 built after passing the old-C flags to make; version confirmed.

6.3 Detection

Observation. stegdetect gave these results at its default sensitivity and at `-s 3` (Figure 44):

File	Default	-s 3
f0104057.jpg	negative	negative
f0104249.jpg	jphide(*)	jphide(***)
f0105065.jpg	skipped (false positive likely)	skipped (false positive likely)
f0105873.jpg (identical to f0335081.jpg)	negative	jphide(*)
f0106393.jpg	negative	negative
f0106409.jpg	negative	jphide(*)
f0335081.jpg	negative	jphide(*)

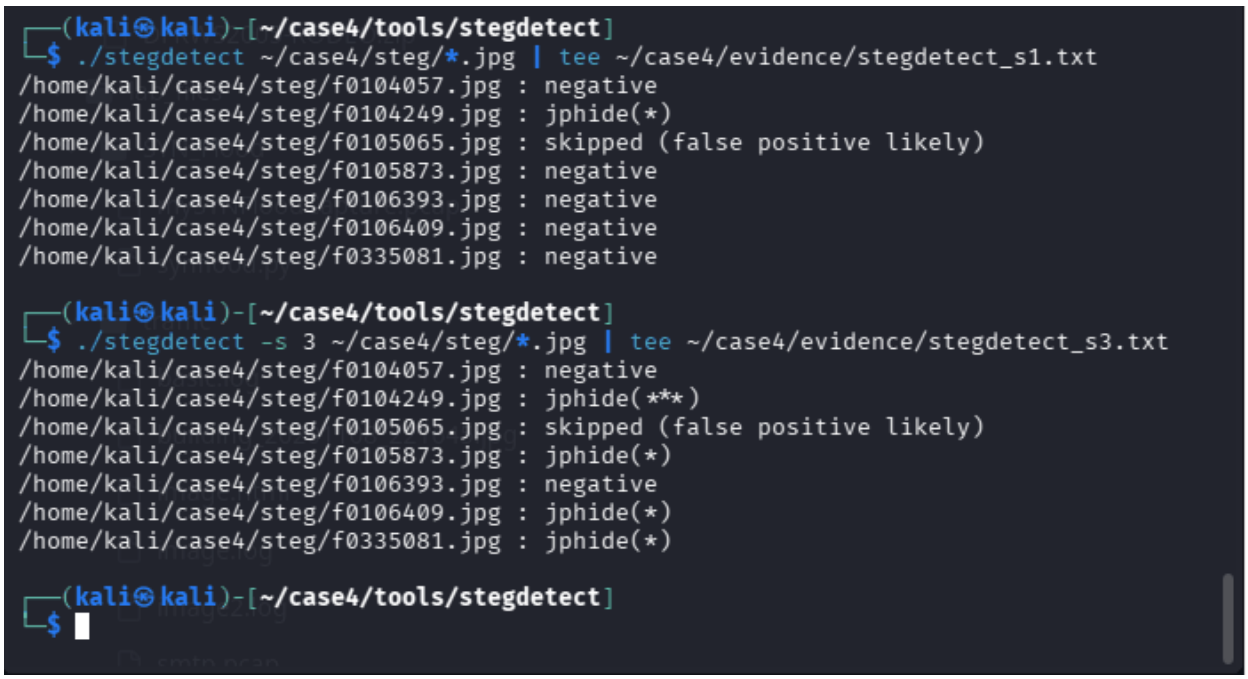


Figure 44. stegdetect results at the default sensitivity and at -s 3.

Interpretation. Only `f0104249.jpg` is flagged at the default setting, and its confidence rises at `-s 3`. The three weaker flags appear only at the higher sensitivity, which also raises false alarms, so I treat them as suspicious but unconfirmed. `f0105873.jpg` and `f0335081.jpg` are one finding, because they are byte-identical. "Skipped" means the tool declined to judge, which is not the same as clean. Detection alone is not proof of hidden content.

6.4 Password recovery (authorised, course use only)

Observation. I ran stegbreak with Kali's `rockyou.txt` wordlist (its SHA-256 is recorded in my evidence folder) against four distinct candidates. The run started at 16:04:46 UTC on 2026-10-25. It found passphrases for `f0105065.jpg` and `f0104249.jpg`, both reported as `jphide[v5]`, and then ended with a **segmentation fault** at 16:51:28 UTC (Figure 45). The passphrases are withheld in this report. I recorded an examiner note and re-ran the other two candidates separately, `f0105873.jpg` and `f0106409.jpg`. Both ended with exit code 139 and without stegbreak's completion line (Figure 46). The end time of the second re-run is about four hours after its start, which probably reflects a suspended virtual machine, so I do not rely on that duration. To show that the tool itself works, I ran stegbreak with a two-word list. It reproduced both hits and ended normally with "Processed 2 files, found 2 embeddings." (Figure 47).



Figure 45. stegbreak dictionary run: start 16:04:46 UTC, two candidate passphrases found (withheld), segmentation fault, end 16:51:28 UTC.

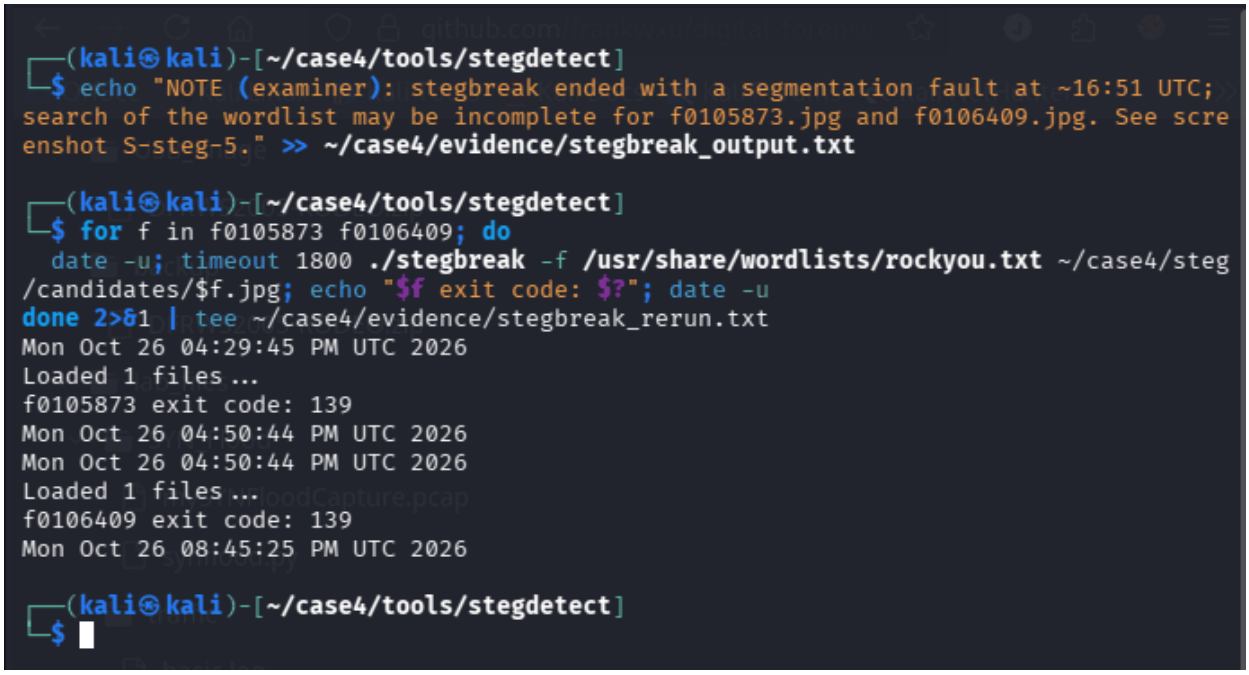


Figure 46. Examiner note and re-runs of the two remaining candidates: both ended with exit code 139 and no completion line.



Figure 47. **Verification run with a two-word list: both hits reproduced (passphrases withheld) and a clean completion line.**

Interpretation. Two carriers have passphrase hits. For `f0105873.jpg` (and its identical twin `f0335081.jpg`) and for `f0106409.jpg`, the search crashed before it finished, so the result is **inconclusive**. It does not mean that no passphrase exists. A hit is only a candidate until an extraction succeeds.

6.5 Extraction

Observation. I obtained the jphs 0.5 package as a tool only and recorded its hashes (Figure 48): the zip is `906e6080dac971acf27b42a1942702dab27ef4e60dbb4dc61077da429c5eeb67`, `jphide.exe` is `a06aba06e0785057101646a7155cff24bf9716824c56b53f78d240f9b7c11c3b` and `jpseek.exe` is `ad2c54554a8e7871062c4b3a68fdfadb105f4d922e8ffc997e8a5884c0125f40`. jpseek is a 32-bit Windows program, so Wine needed its 32-bit support installed. I then ran `jpseek.exe` on each carrier and entered its own passphrase at the prompt (Figure 49). Both extractions produced valid JPEG files (Figure 50, Figure 51):

Extracted file	Carrier	Type and metadata	SHA-256
out_f0104249.jpg	f0104249.jpg	JPEG 384×256; Exif software "ACD Systems Digital Imaging", date 2005:08:08 21:04:10	6dbf775b8999a006ffa74c65127e5add64234184bd2f425b2ad644e8c0d19345
out_f0105065.jpg	f0105065.jpg	JPEG 400×340; camera NIKON E950; software "ACD Systems Digital Imaging", date 2005:08:08 21:05:43	bc9c605372fdf9cc00faf17ae9ce0fdb6d26bb9db28cdff742d9ddbe1470ec16

```
└─$ mkdir -p ~/case4/tools/jphs && cd ~/case4/tools/jphs

┌──(kali㉿kali)-[~/case4/tools/jphs]
└─$ wget -q ftp://ftp.gwdg.de/pub/linux/misc/ppdd/jphs_05.zip

┌──(kali㉿kali)-[~/case4/tools/jphs]
└─$ unzip jphs_05.zip && unzip jphs05.zip
Archive:  jphs_05.zip
 extracting: jphs05.zip
 extracting: jphs05.zip.sig
Archive:  jphs05.zip
  inflating: jphide.exe
  inflating: jpseek.exe
  inflating: Jphswin.exe
  inflating: Readme.txt

┌──(kali㉿kali)-[~/case4/tools/jphs]
└─$ sha256sum jphs_05.zip jphide.exe jpseek.exe | tee ~/case4/evidence/jphs_tool_sha
256.txt
906e6080dac971acf27b42a1942702dab27ef4e60dbb4dc61077da429c5eeb67  jphs_05.zip
a06aba06e0785057101646a7155cff24bf9716824c56b53f78d240f9b7c11c3b  jphide.exe
ad2c54554a8e7871062c4b3a68fdfadb105f4d922e8ffc997e8a5884c0125f40  jpseek.exe

┌──(kali㉿kali)-[~/case4/tools/jphs]
└─$ sudo apt install -y wine
[sudo] password for kali:
The following packages were automatically installed and are no longer required:
  libimath-3-1-29t64  libpython3.13-dev  python3.13          rpcsvc-proto
  libmbedcrypto16     libraw23t64        python3.13-dev
  libpostal-data      libvidstab1.1      python3.13-minimal
  libpostal1          libzxing3          python3.13-venv
Use 'sudo apt autoremove' to remove them.
```

Figure 48. jphs 0.5 package and its executables hashed; Wine installation started.

```
┌──(kali㉿kali)-[~/case4/tools/stegdetect]
└─$ cd ~/case4/steg/extract_work

┌──(kali㉿kali)-[~/case4/steg/extract_work]
└─$ wine jpseek.exe f0104249.jpg out_f0104249.jpg

Welcome to jpseek Rev 0.51
 (c) 1998 Allan Latham <alatham@flexsys-group.com>
This program is freeware.
No charge is made for its use.
Use at your own risk. No liability accepted whatever happens.
Contains cryptogaphy which may be subject to local laws.

Passphrase:
```

Figure 49. jpseek 0.51 started under Wine and waiting for the passphrase.

```
┌──(kali㉿kali)-[~/case4/tools/stegdetect]
└─$ cd ~/case4/steg/extract_work

┌──(kali㉿kali)-[~/case4/steg/extract_work]
└─$ wine jpseek.exe f0104249.jpg out_f0104249.jpg

Welcome to jpseek Rev 0.51
 (c) 1998 Allan Latham <alatham@flexsys-group.com>
This program is freeware.
No charge is made for its use.
Use at your own risk. No liability accepted whatever happens.
Contains cryptogaphy which may be subject to local laws.

Passphrase:

┌──(kali㉿kali)-[~/case4/steg/extract_work]
└─$ file out_f0104249.jpg; xxd out_f0104249.jpg | head -2; sha256sum out_f0104249.jp
g
out_f0104249.jpg: JPEG image data, Exif standard: [TIFF image data, little-endian, d
irentries=5, orientation=upper-left, software=ACD Systems Digital Imaging, datetime=
2005:08:08 21:04:10], baseline, precision 8, 384×256, components 3
00000000: ffd8 ffe1 00e6 4578 6966 0000 4949 2a00  ......Exif..II*.
00000010: 0800 0000 0500 1201 0300 0100 0000 0100  ................
6dbf775b8999a006ffa74c65127e5add64234184bd2f425b2ad644e8c0d19345  out_f0104249.jpg

┌──(kali㉿kali)-[~/case4/steg/extract_work]
└─$
```

Figure 50. Extracted out_f0104249.jpg: valid JPEG (384x256), SHA-256 recorded.

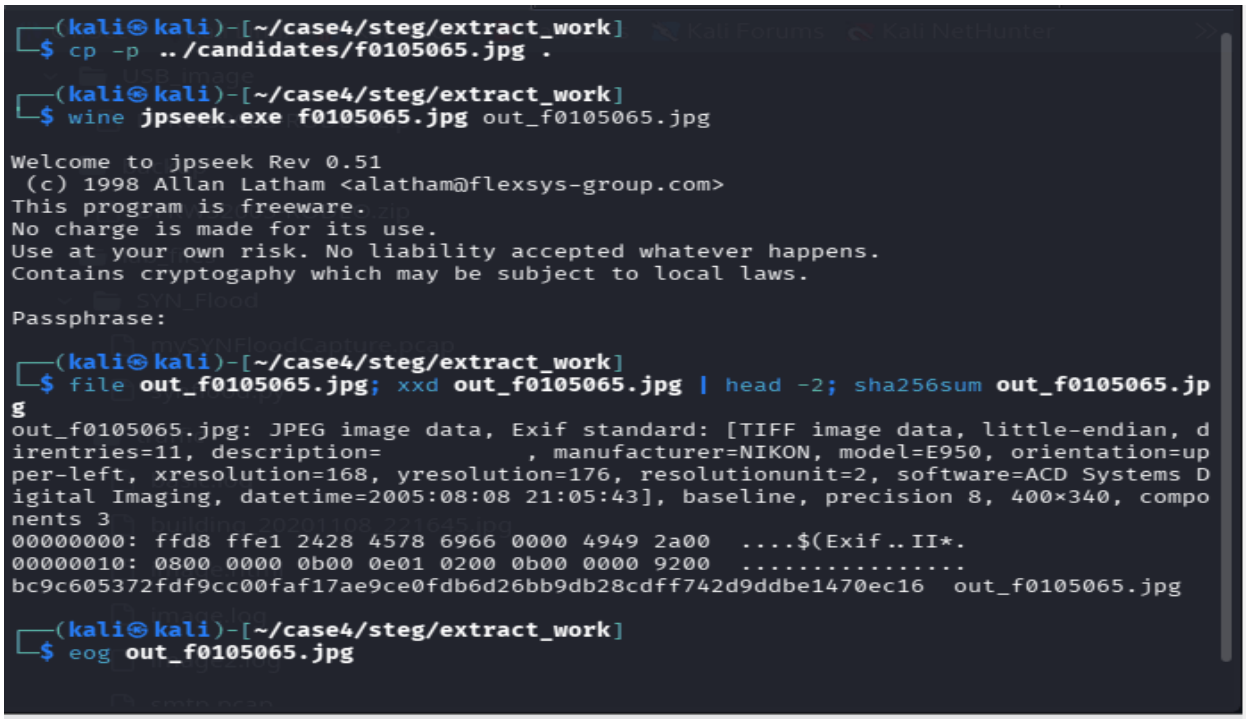


Figure 51. Extracted out_f0105065.jpg: valid JPEG (400x340), SHA-256 recorded.

I compared the SHA-256 of both extracted images with every image recovered from the disk. **Neither matches any of them**, and the seven carrier files still verify as OK (Figure 52).

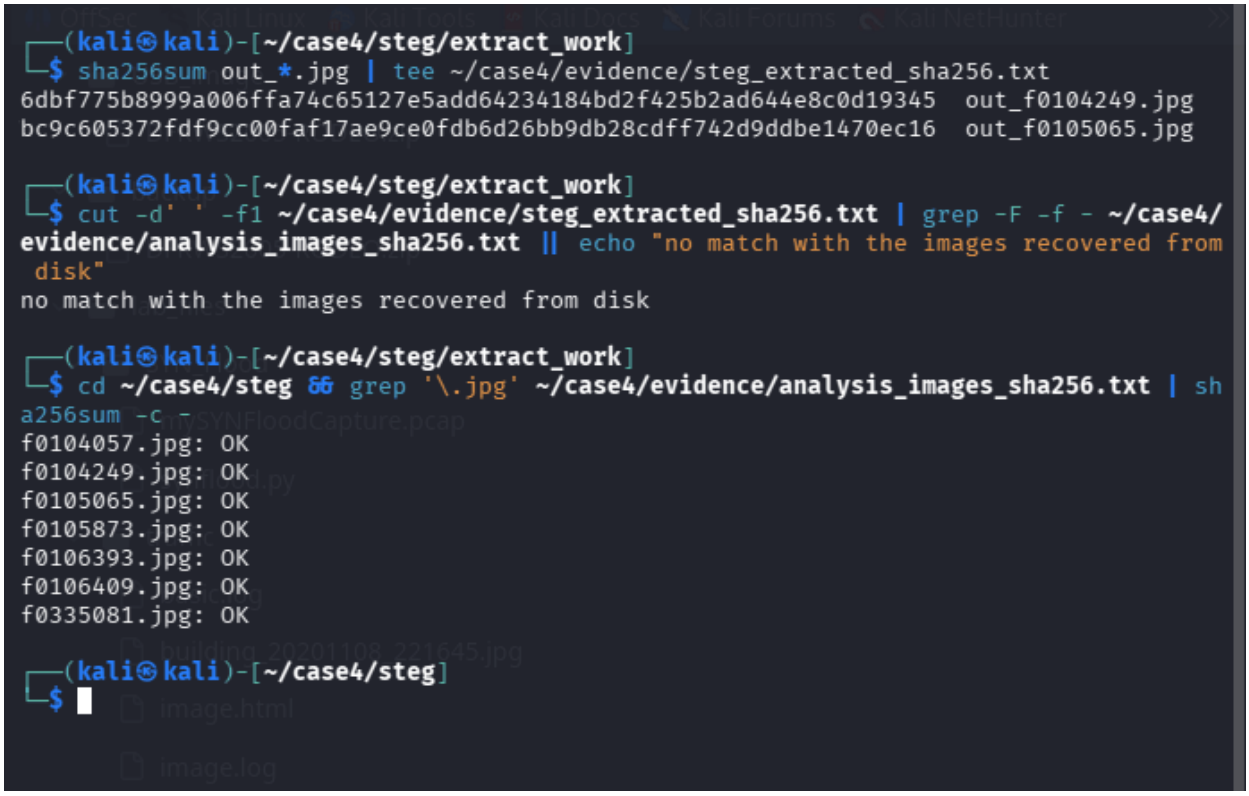


Figure 52. SHA-256 of both extracted images; no match with images recovered from disk; the seven carrier files still verify OK.

Interpretation. A valid, self-consistent JPEG came out of each carrier under a recovered passphrase. That is strong evidence that `f0104249.jpg` and `f0105065.jpg` carried hidden images. Evidence **weakening** a firm conclusion is: the extra stegdetect flags are low-confidence and appear only at `-s 3`; `f0105065.jpg` was "skipped" by stegdetect; the password search crashed for two candidates; and stegdetect is statistical. The embedded dates (2005) are later than the 2004 evidence dates and cannot place any event. The extraction shows that content was hidden, not who hid it or when.

7. FTP capture: rhino.log (E21 to E22)

7.1 The three FTP sessions

Observation. The Wireshark filter `ftp.request.command == "USER"` finds three FTP logins in `rhino.log`, at frames 1532, 1625 and 5633. All come from 137.30.122.253 to 137.30.120.40 as the account **gnome** (Figure 53). FTP sends credentials in clear text, so the password is blacked out in every figure and withheld from the text. A `tshark` table of the PORT, STOR, 150 and 226 messages gives the transfers for sessions 1 and 2 (Figure 54). Following control stream 69 shows the full dialogue of session 1, including the server's own count of bytes transferred (Figure 55, Figure 56). Session 3 is control stream 305 (Figure 57).

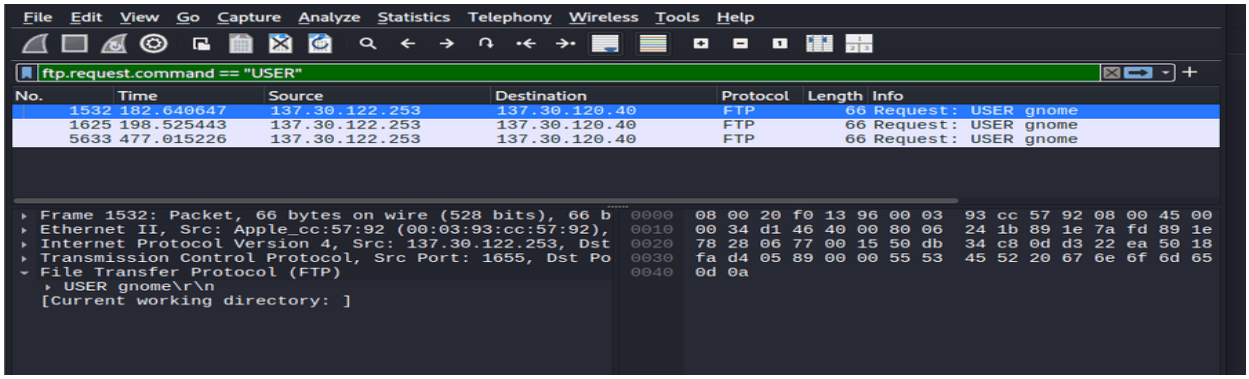


Figure 53. Wireshark: three FTP logins (USER) in rhino.log at frames 1532, 1625 and 5633.

```
┌──(kali㉿kali)-[~/case4/ftp]
└─$ mkdir -p ~/case4/evidence

┌──(kali㉿kali)-[~/case4/ftp]
└─$ tshark -r ~/case4/working/rhino.log \
  -Y 'ftp.request.command in {"STOR","RETR","PASV","PORT"} || ftp.response.code i
n {150,226}' \
  -T fields -E header=y -E separator=/t \
  -e frame.number -e frame.time_utc -e ip.src -e ip.dst -e tcp.stream \
  -e ftp.request.command -e ftp.request.arg -e ftp.response.code -e ftp.response.
arg \
  | tee ~/case4/evidence/ftp_transfers.tsv
frame.number    frame.time_utc  ip.src  ip.dst  tcp.stream      ftp.request.comma
nd      ftp.request.arg ftp.response.code       ftp.response.arg
1544    2004-04-26T22:21:49.952880000Z  137.30.122.253  137.30.120.40   69      P
ORT     137,30,122,253,6,121
1546    2004-04-26T22:21:49.954047000Z  137.30.122.253  137.30.120.40   69      S
TOR     rhino1.jpg
1550    2004-04-26T22:21:49.991431000Z  137.30.120.40   137.30.122.253  69      1
50      Opening BINARY mode data connection for rhino1.jpg.
1612    2004-04-26T22:21:50.179677000Z  137.30.120.40   137.30.122.253  69      2
26      Transfer complete.
1647    2004-04-26T22:22:08.903584000Z  137.30.122.253  137.30.120.40   72      P
ORT     137,30,122,253,6,124
1649    2004-04-26T22:22:08.905569000Z  137.30.122.253  137.30.120.40   72      S
TOR     rhino3.jpg
1653    2004-04-26T22:22:08.937533000Z  137.30.120.40   137.30.122.253  72      1
50      Opening ASCII mode data connection for rhino3.jpg.
1737    2004-04-26T22:22:09.039917000Z  137.30.120.40   137.30.122.253  72      2
26      WARNING! 321 bare linefeeds received in ASCII mode
```

Figure 54. tshark transfer table for sessions 1 and 2: PORT, STOR, 150 and 226 with UTC times, endpoints and streams.

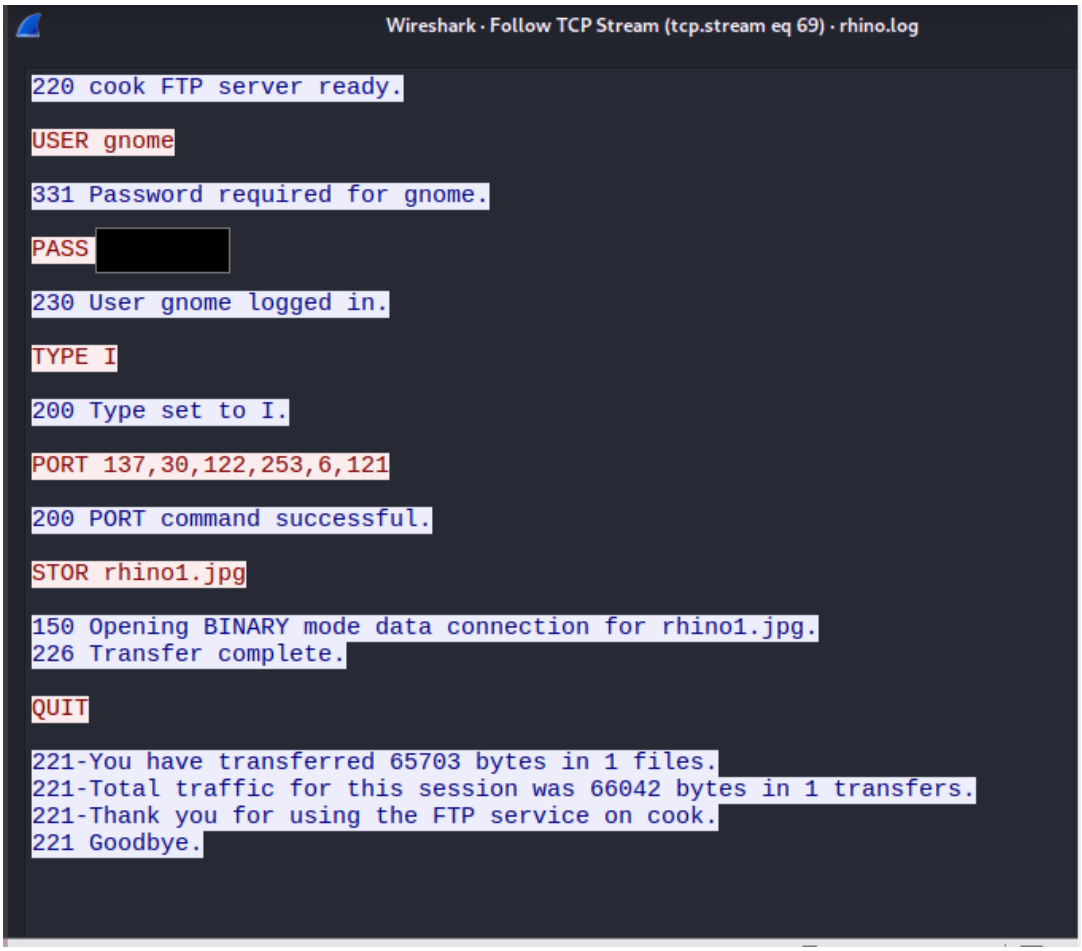


Figure 55. Follow TCP Stream 69 (control connection): login, PORT, STOR rhino1.jpg, 150, 226 and the server byte count (password redacted).

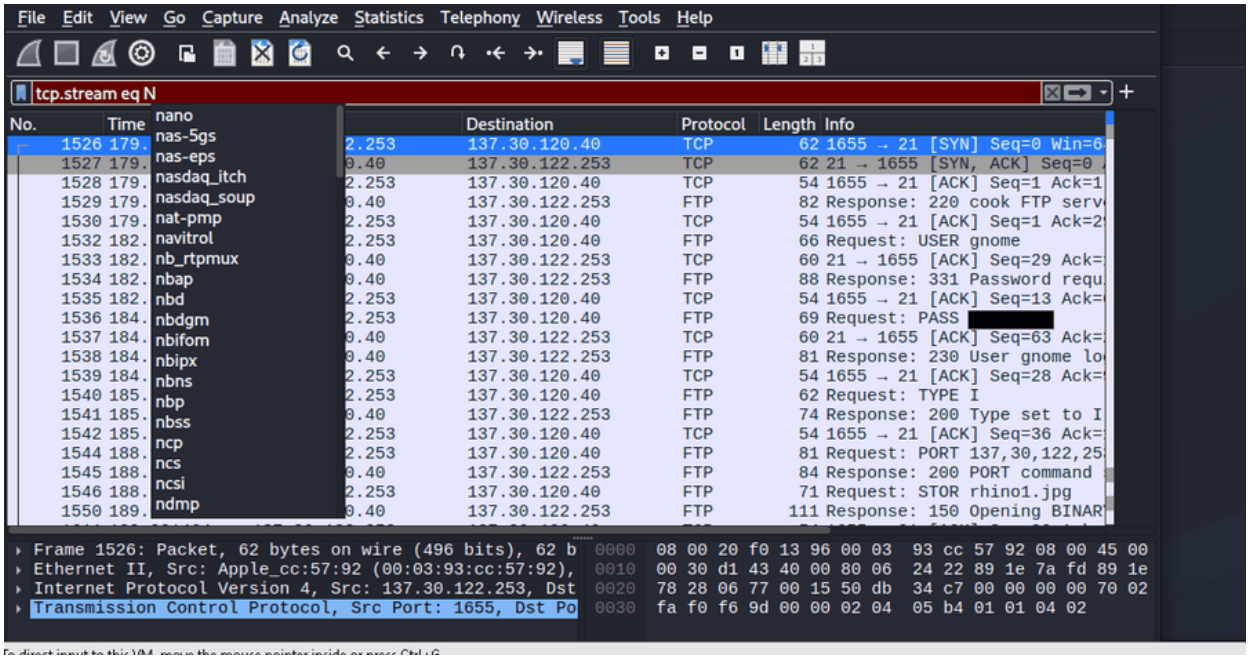


Figure 56. Wireshark packet list for stream 69 (password redacted).

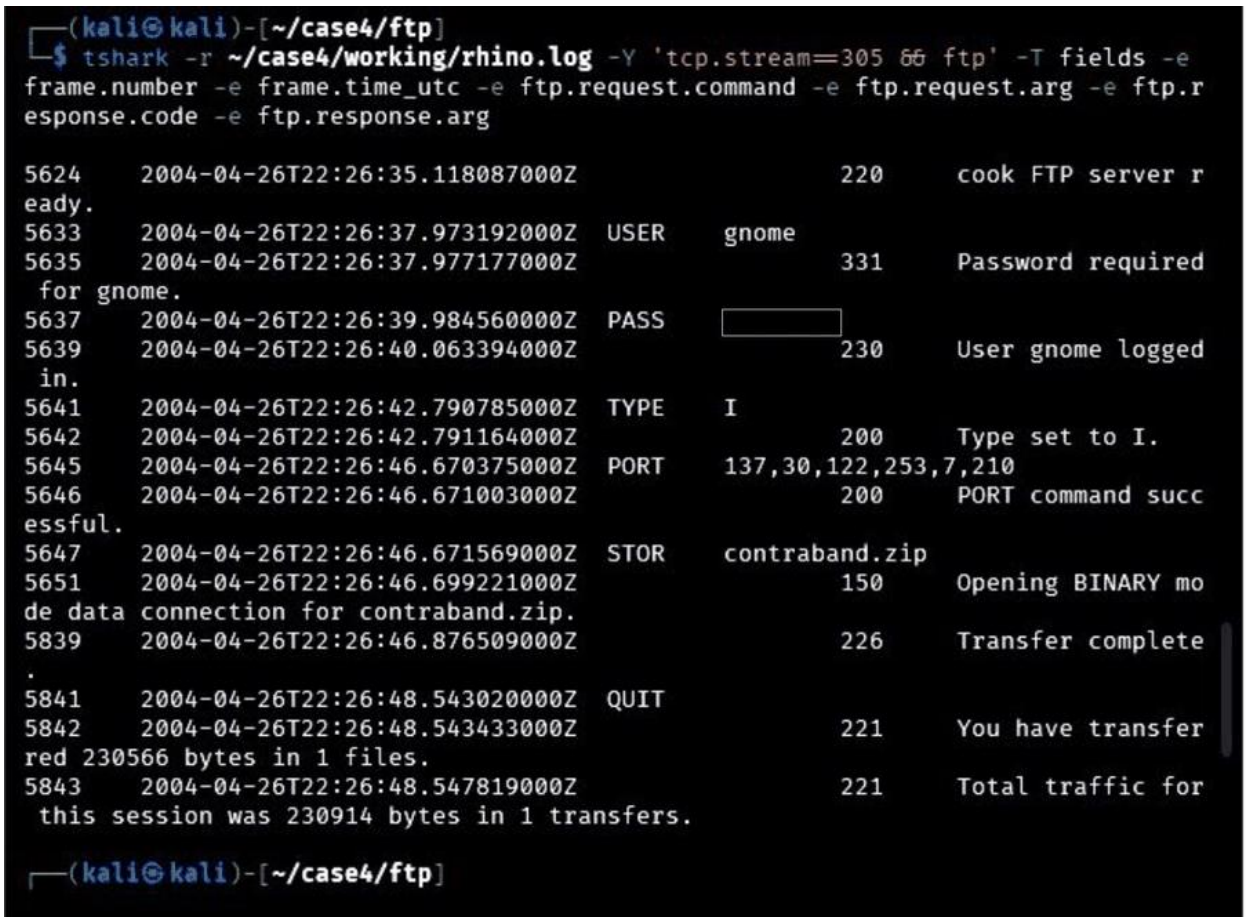


Figure 57. Session 3 (stream 305): login, PORT, STOR contraband.zip, 150, 226 and the server byte count (password redacted).

Session	Control stream / client data port	File uploaded	STOR frame and time (UTC)	150 reply (UTC)	226 reply (UTC)	Bytes
1	69 / 1657	rhino1.jpg	1546 2004-04-26 22:21:49.954047	1550 22:21:49.991431 (BINARY mode)	1612 22:21:50.179677	65,703 (server count equals file size)
2	72 / 1660	rhino3.jpg	1649 2004-04-26 22:22:08.905569	1653 22:22:08.937533 (ASCII mode)	1737 22:22:09.039917, with the warning "321 bare linefeeds received in ASCII mode"	96,899 (file size)
3	305 (data stream 307, client port 2002)	contraband.zip	5647 2004-04-26 22:26:46.671569	5651 22:26:46.699221 (BINARY mode)	5839 22:26:46.876509	230,566 (server count equals file size)

Interpretation. In every session the client sent a PORT command with its own address (137.30.122.253) and then a STOR command. STOR means the **client uploads a file to the server**, so these are three uploads, not downloads. Each ended with `226 Transfer complete`. For sessions 1 and 3, the server's byte count equals the size of the file I rebuilt. The ASCII-mode warning on session 2 matters: the server may have converted line endings, so the copy stored on the server could differ from the bytes the client sent. I rebuilt the bytes that the client sent.

7.2 Validation of the uploaded images

Observation. `rhino1.jpg` is 65,703 bytes and `rhino3.jpg` is 96,899 bytes. `file` reports JPEG data of 501×360 and 720×597. Both begin `FFD8FFE0` and end `FFD9`, and `jpeginfo -c` reports both OK (Figure 58, Figure 59).

```
└─$ sha256sum rhino1.jpg rhino3.jpg
0f30a8777d19a4fe4b2fafd5154b6c4dfafb673202db29971e876e978da4e05d  rhino1.jpg
a85aea8e47d9f5425456f3bafb9c67201c8c2b2b2cec80154c83102e802240c5  rhino3.jpg

┌──(kali㉿kali)-[~/case4/ftp]
└─$ ls -l rhino1.jpg rhino3.jpg
-rw-rw-r-- 1 kali kali 65703 Oct 27 13:59 rhino1.jpg
-rw-rw-r-- 1 kali kali 96899 Oct 27 14:00 rhino3.jpg

┌──(kali㉿kali)-[~/case4/ftp]
└─$ file rhino1.jpg rhino3.jpg
rhino1.jpg: JPEG image data, JFIF standard 1.02, resolution (DPI), density 72×72,
 segment length 16, comment: "File written by Adobe Photoshop\250 4.0", baseline,
 precision 8, 501×360, components 3
rhino3.jpg: JPEG image data, JFIF standard 1.01, resolution (DPI), density 72×72,
 segment length 16, baseline, precision 8, 720×597, components 3

┌──(kali㉿kali)-[~/case4/ftp]
└─$ xxd rhino1.jpg | head -2; xxd rhino1.jpg | tail -2
00000000: ffd8 ffe0 0010 4a46 4946 0001 0201 0048  ......JFIF.....H
00000010: 0048 0000 ffed 1598 5068 6f74 6f73 686f  .H......Photosho
00010090: 2d3f 23fd 6639 4557 f552 2f56 4fb3 5f87  -?#.f9EW.R/VO._.
000100a0: c332 a834 dbff d9                        .2.4...

┌──(kali㉿kali)-[~/case4/ftp]
└─$ xxd rhino3.jpg | head -2; xxd rhino3.jpg | tail -2
00000000: ffd8 ffe0 0010 4a46 4946 0001 0101 0048  ......JFIF.....H
00000010: 0048 0000 ffdb 0043 0005 0304 0404 0305  .H.....C........
00017a70: 91fe 3451 5df5 69c6 dcd6 d4e4 84e4 e76b  ..4Q].i........k
00017a80: 9fff d9                                  ...
```

Figure 58. rhino1.jpg and rhino3.jpg: SHA-256, sizes, file type, JPEG header and trailer bytes.

```
┌──(kali㉿kali)-[~/case4/ftp]
└─$ jpeginfo -c rhino1.jpg rhino3.jpg
rhino1.jpg                    501 x  360 24bit N JFIF,IPTC,Adobe,COM        6
5703 OK
rhino3.jpg                    720 x  597 24bit N JFIF                      9
6899 OK

┌──(kali㉿kali)-[~/case4/ftp]
└─$   tshark -r ~/case4/working/rhino.log -Y 'frame.number==5633' -T fields -e tc
p.stream
305

┌──(kali㉿kali)-[~/case4/ftp]
└─$    tshark -r ~/case4/working/rhino.log -Y 'ftp.request.command in {"STOR","RE
TR","TYPE","PORT"} && frame.number > 5600' -T fields -e frame.number -e frame.tim
e_utc -e ftp.request.command -e ftp.request.arg
5641    2004-04-26T22:26:42.790785000Z  TYPE    I
5645    2004-04-26T22:26:46.670375000Z  PORT    137,30,122,253,7,210
5647    2004-04-26T22:26:46.671569000Z  STOR    contraband.zip

┌──(kali㉿kali)-[~/case4/ftp]
└─$
```

Figure 59. jpeginfo -c validation of both JPEGs; stream number of the third session (frame 5633 is in stream 305); its PORT and STOR commands.

Interpretation. Both are complete, structurally valid JPEG files. Their hashes appear in the master table, and they do not match any image from the disk, the steganography or the HTTP captures.

8. Protected archive (E23 to E25)

8.1 Rebuilding and validating contraband.zip

Observation. My first reconstruction attempt produced an empty file, because a placeholder was left in the `tshark` command. Its SHA-256 is the well-known hash of empty input, and `unzip` and `zipinfo` could not find a zip directory in it (Figure 60). I kept the record and corrected the command. Rebuilding from data stream 307 gave a file of **230,566 bytes**, SHA-256 `b7091a0b4295a561bf76969073ede126a5ca75899bb115447e67272f95d5a29f`, which `file` recognises as a Zip archive (deflate method, entry time 2004-04-26 17:00:12, uncompressed size 230,665) (Figure 61). The file starts with the local-header signature PK `03 04`, and its flag field has bit 0 set, which marks an encrypted entry. The end-of-central-directory record sits at offset 230,544, exactly where the central directory predicts it, and the archive has one entry, `rhino2.jpg` (Figure 62).

```
┌──(kali㉿kali)-[~/case4/ftp]
└─$ sha256sum contraband.zip; ls -l contraband.zip; file contraband.zip
e3b0c44298fc1c149afbf4c8996fb92427ae41e4649b934ca495991b7852b855  contraband.zip
-rw-rw-r-- 1 kali kali 0 Oct 28 13:44 contraband.zip
contraband.zip: empty

┌──(kali㉿kali)-[~/case4/ftp]
└─$ xxd contraband.zip | head -3; xxd contraband.zip | tail -3

┌──(kali㉿kali)-[~/case4/ftp]
└─$ zipinfo -v contraband.zip | head -50
[contraband.zip]
Archive:  contraband.zip
  End-of-central-directory signature not found.  Either this file is not
  a zipfile, or it constitutes one disk of a multi-part archive.  In the
  latter case the central directory and zipfile comment will be found on
  the last disk(s) of this archive.
zipinfo:  cannot find zipfile directory in one of contraband.zip or
          contraband.zip.zip, and cannot find contraband.zip.ZIP, period.
```

Figure 60. *First reconstruction attempt produced an empty file (a placeholder was left in the command); it is recorded and discarded.*

```
┌──(kali㉿kali)-[~/case4/ftp]
└─$ tshark -r ~/case4/working/rhino.log -q -z follow,tcp,raw,307 \
  | grep -E '^\s*[0-9a-f]+$' | tr -d ' \t' | xxd -r -p > contraband.zip

┌──(kali㉿kali)-[~/case4/ftp]
└─$ sha256sum contraband.zip | tee ~/case4/evidence/contraband_sha256.txt
b7091a0b4295a561bf76969073ede126a5ca75899bb115447e67272f95d5a29f  contraband.zip

┌──(kali㉿kali)-[~/case4/ftp]
└─$ ls -l contraband.zip; file contraband.zip
-rw-rw-r-- 1 kali kali 230566 Oct 28 14:21 contraband.zip
contraband.zip: Zip archive data, made by v2.0, extract using at least v2.0, last
 modified Apr 26 2004 17:00:12, uncompressed size 230665, method=deflate

┌──(kali㉿kali)-[~/case4/ftp]
└─$ ls -l contraband.zip; file contraband.zip
-rw-rw-r-- 1 kali kali 230566 Oct 28 14:21 contraband.zip
contraband.zip: Zip archive data, made by v2.0, extract using at least v2.0, last
 modified Apr 26 2004 17:00:12, uncompressed size 230665, method=deflate
```

Figure 61. **Archive rebuilt from data stream 307 with tshark: 230,566 bytes, SHA-256 recorded, recognised as a Zip archive.**

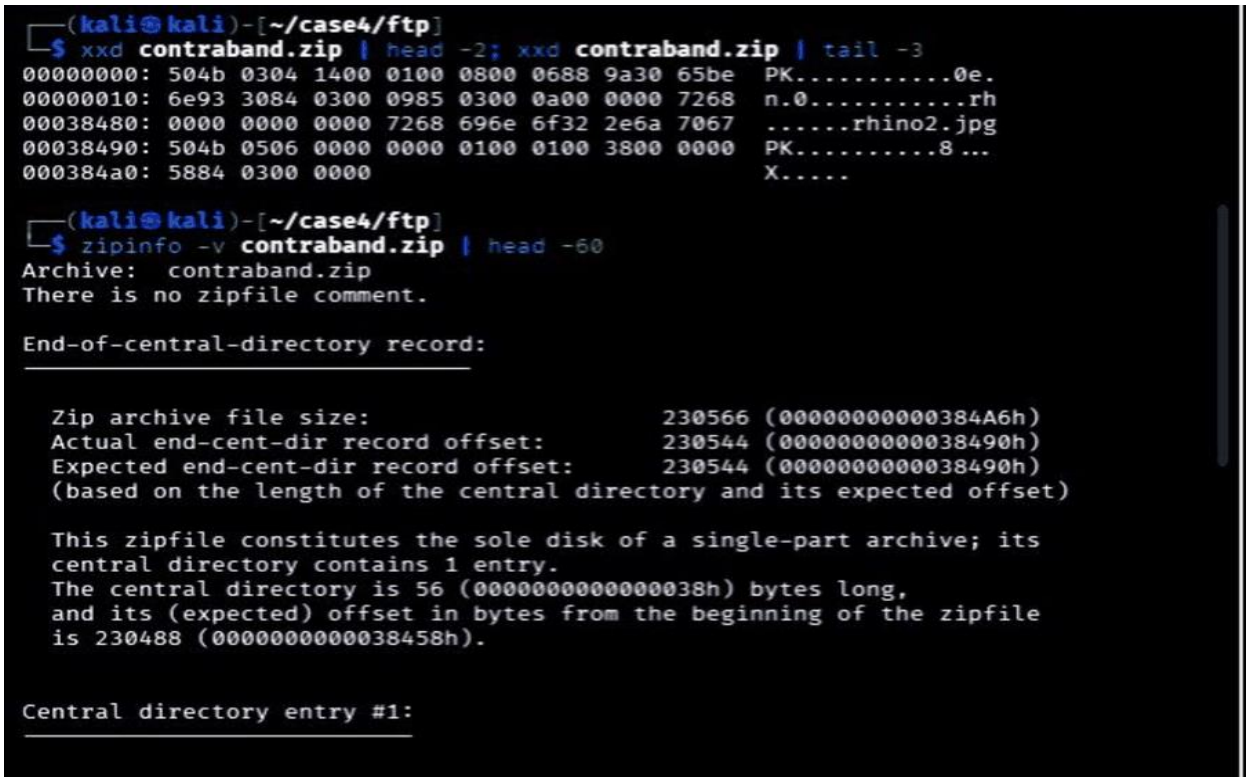


Figure 62. Zip structure: local-header signature at the start; end-of-central-directory record at the expected offset (230,544); one central-directory entry.

Interpretation. The reconstructed size equals the server's reported 230,566 bytes, and the internal structure is consistent, so the archive is complete and intact. The uncompressed size of the entry, 230,665 bytes, is also exactly the size of the carved `f0106409.jpg`.

8.2 Password recovery and extraction (authorised, course use only)

Observation. I ran `fcrackzip` in dictionary mode (`-u -D`) against `contraband.zip` using Kali's `rockyou.txt` wordlist (the same list, and the same SHA-256, used for stegbreak). The run started at 11:03:04 UTC and completed at 11:03:05 UTC, about 13 seconds, and reported a password (Figure 63). The password is withheld in this report.

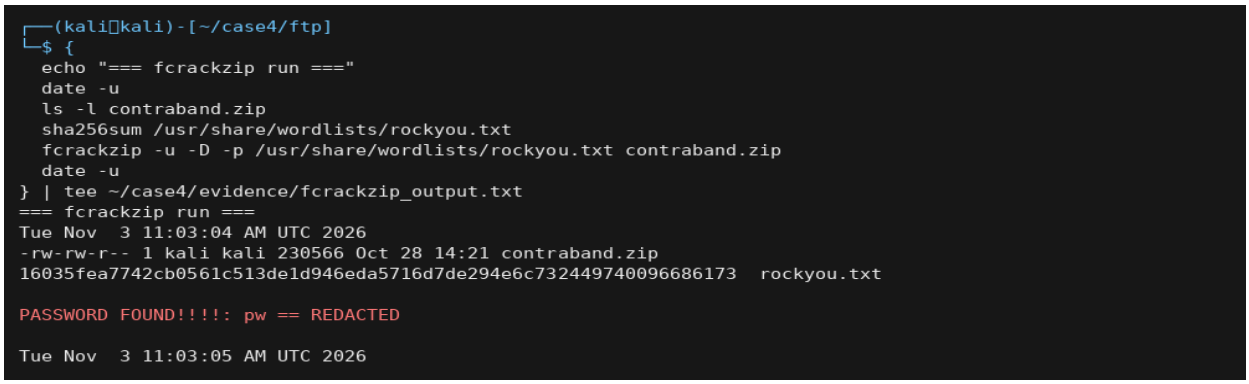


Figure 63. Authorised, course-only fcrackzip run against contraband.zip using the Kali rockyou.txt wordlist (hash recorded); password withheld; run completed in 13 seconds (11:03:04 to 11:03:05 UTC).

Observation. With the recovered password, `unzip` extracted one file, `rhino2.jpg`, into my evidence folder (Figure 64). ExifTool reports a JPEG of 1024×685 at 170 dpi, with a modification time of 2004-04-26 17:00:12 that comes from the zip entry (Figure 65). That time has no time zone.

```
┌──(kali㉿kali)-[~/case4/evidence/contraband_extracted]
└─$ cd ~/case4/ftp

┌──(kali㉿kali)-[~/case4/ftp]
└─$ ls -l contraband.zip
-rw-rw-r-- 1 kali kali 230566 Oct 28 14:21 contraband.zip

┌──(kali㉿kali)-[~/case4/ftp]
└─$ unzip -P '[redacted]' contraband.zip -d ~/case4/evidence/contraband_extracted
Archive:  contraband.zip
  inflating: /home/kali/case4/evidence/contraband_extracted/rhino2.jpg

┌──(kali㉿kali)-[~/case4/ftp]
└─$ unzip -P '[redacted]' contraband.zip -d ~/case4/evidence/contraband_extracted
Archive:  contraband.zip
replace /home/kali/case4/evidence/contraband_extracted/rhino2.jpg? [y]es, [n]o, [
A]ll, [N]one, [r]ename: [redacted]
error:  invalid response [[redacted]]
replace /home/kali/case4/evidence/contraband_extracted/rhino2.jpg? [y]es, [n]o, [
A]ll, [N]one, [r]ename: y
  inflating: /home/kali/case4/evidence/contraband_extracted/rhino2.jpg

┌──(kali㉿kali)-[~/case4/ftp]
└─$ cd ~/case4/evidence/contraband_extracted
```

Figure 64. Extraction of the course archive in the lab (password withheld): rhino2.jpg inflated.

```
┌──(kali㉿kali)-[~/case4/evidence/contraband_extracted]
└─$ exiftool rhino2.jpg
ExifTool Version Number         : 13.50
File Name                       : rhino2.jpg
Directory                       : .
File Size                       : 231 kB
File Modification Date/Time     : 2004:04:26 17:00:12-04:00
File Access Date/Time           : 2026:10:28 16:43:52-04:00
File Inode Change Date/Time     : 2026:10:28 16:43:18-04:00
File Permissions                : -rw-rw-r--
File Type                       : JPEG
File Type Extension             : jpg
MIME Type                       : image/jpeg
JFIF Version                    : 1.01
Resolution Unit                 : inches
X Resolution                    : 170
Y Resolution                    : 170
Image Width                     : 1024
Image Height                    : 685
Encoding Process                : Baseline DCT, Huffman coding
Bits Per Sample                 : 8
Color Components                : 3
Y Cb Cr Sub Sampling            : YCbCr4:2:0 (2 2)
Image Size                      : 1024×685
Megapixels                      : 0.701

┌──(kali㉿kali)-[~/case4/evidence/contraband_extracted]
└─$
```

Figure 65. ExifTool of the extracted rhino2.jpg: JPEG, 1024x685, 170 dpi.

8.3 Match with the file carved from the USB image

Observation. The SHA-256 of the extracted `rhino2.jpg` is `b4f6bbb8d846d6505c9b5b87f115656fdf980a484aa441078a838a3ffc2ac705`. The same value appears for `f0106409.jpg` in the analysis folder and in both recovery folders (Figure 66). I recorded this in a cross-match note (Figure 67).



Figure 66. The SHA-256 prefix b4f6bbb8 appears for f0106409.jpg (both recovery folders) and for rhino2.jpg.



Figure 67. Examiner cross-match note (password withheld).

Interpretation. The same image content is present as a carved file in the unallocated space of the USB image **and** as a file that was uploaded by FTP on 2004-04-26 at about 22:26:46Z. The match shows identical content in two independent evidence sources. It does **not** show which copy came first, in which direction the file moved, or that the person who used the USB device performed the upload.

9. HTTP captures (E26 to E27)

9.1 rhino2.log

Observation. `rhino2.log` holds 13 HTTP requests, all GET requests from client 137.30.123.234 (Figure 68, Figure 69). Three go to Google (64.233.167.104) and ten to `www.cs.uno.edu` (137.30.120.37):

Frame	Time (UTC, 2004-04-28)	Stream	Host	URI
4	21:08:30.270250	0	www.google.com	/
11	21:08:30.955099	0	www.google.com	/images/logo.gif
28	21:08:35.467929	1	www.cs.uno.edu	/~gnome
32	21:08:35.734906	1	www.cs.uno.edu	/~gnome/
36	21:08:36.080760	1	www.cs.uno.edu	/icons/blank.gif
39	21:08:36.341364	1	www.cs.uno.edu	/icons/image2.gif
43	21:08:36.344384	2	www.cs.uno.edu	/icons/back.gif
49	21:08:38.073111	1	www.cs.uno.edu	/~gnome/rhino4.jpg
217	21:08:44.189294	2	www.cs.uno.edu	/~gnome/rhino5.gif
333	21:08:58.067074	4	www.google.com	/
343	21:09:14.099217	5	www.cs.uno.edu	/~venkata
347	21:09:14.422408	5	www.cs.uno.edu	/~venkata/
359	21:09:57.094465	6	www.cs.uno.edu	/~venkata/2025/index.html

The responses give status, type and length (Figure 70). The image responses are `200 image/jpeg` of **153,191 bytes** (frame 215, 21:08:38.275Z) and `200 image/gif` of **85,137 bytes** (frame 312, 21:08:44.378Z). I exported all 13 objects with `tshark` (Figure 71) and hashed them (Figure 72). The exported `rhino4.jpg` and `rhino5.gif` have exactly the Content-Length sizes. `rhino4.jpg` starts `FFD8FFE0` and ends FFD9, and is a JPEG of 1531×2286 with the comment "U-Lead Systems, Inc.". `rhino5.gif` starts `GIF89a`, ends 3B, and is 400×275. The other objects are web-page assets (small icons, a logo, directory listings and the Google front page, which was fetched twice with an identical hash). A comparison of every exported object's hash with the disk, steganography and FTP images found **no match**, with exit code 1 (Figure 73).

```
┌──(kali㉿kali)-[~/case4/ftp]
└─$ tshark -r ~/case4/working/rhino2.log -Y http.request -T fields -e frame.numbe
r -e frame.time_utc -e ip.src -e ip.dst -e http.request.method -e http.host -e ht
tp.request.uri
4       2004-04-28T21:08:30.270250000Z  137.30.123.234  64.233.167.104  GET     w
ww.google.com   /
11      2004-04-28T21:08:30.955099000Z  137.30.123.234  64.233.167.104  GET     w
ww.google.com   /images/logo.gif
28      2004-04-28T21:08:35.467929000Z  137.30.123.234  137.30.120.37   GET     w
ww.cs.uno.edu   /~gnome
32      2004-04-28T21:08:35.734906000Z  137.30.123.234  137.30.120.37   GET     w
ww.cs.uno.edu   /~gnome/
36      2004-04-28T21:08:36.080760000Z  137.30.123.234  137.30.120.37   GET     w
ww.cs.uno.edu   /icons/blank.gif
39      2004-04-28T21:08:36.341364000Z  137.30.123.234  137.30.120.37   GET     w
ww.cs.uno.edu   /icons/image2.gif
43      2004-04-28T21:08:36.344384000Z  137.30.123.234  137.30.120.37   GET     w
ww.cs.uno.edu   /icons/back.gif
49      2004-04-28T21:08:38.073111000Z  137.30.123.234  137.30.120.37   GET     w
ww.cs.uno.edu   /~gnome/rhino4.jpg
217     2004-04-28T21:08:44.189294000Z  137.30.123.234  137.30.120.37   GET     w
ww.cs.uno.edu   /~gnome/rhino5.gif
333     2004-04-28T21:08:58.067074000Z  137.30.123.234  64.233.167.104  GET     w
ww.google.com   /
343     2004-04-28T21:09:14.099217000Z  137.30.123.234  137.30.120.37   GET     w
ww.cs.uno.edu   /~venkata
347     2004-04-28T21:09:14.422408000Z  137.30.123.234  137.30.120.37   GET     w
ww.cs.uno.edu   /~venkata/
359     2004-04-28T21:09:57.094465000Z  137.30.123.234  137.30.120.37   GET     w
ww.cs.uno.edu   /~venkata/2025/index.html
```

Figure 68. rhino2.log: HTTP requests with frame, UTC time, endpoints, host and URI.

```
┌──(kali㉿kali)-[~]
└─$ cd ~/case4/evidence/http_objects

┌──(kali㉿kali)-[~/case4/evidence/http_objects]
└─$ tshark -r ~/case4/working/rhino2.log -Y http.request -T fields -e frame.numbe
d -e http.host -e http.request.uri | tee ~/case4/evidence/http2_requests.tsv
4       2004-04-28T21:08:30.270250000Z  137.30.123.234  64.233.167.104  0       G
11      2004-04-28T21:08:30.955099000Z  137.30.123.234  64.233.167.104  0       G
28      2004-04-28T21:08:35.467929000Z  137.30.123.234  137.30.120.37   1       G
32      2004-04-28T21:08:35.734906000Z  137.30.123.234  137.30.120.37   1       G
36      2004-04-28T21:08:36.080760000Z  137.30.123.234  137.30.120.37   1       G
39      2004-04-28T21:08:36.341364000Z  137.30.123.234  137.30.120.37   1       G
43      2004-04-28T21:08:36.344384000Z  137.30.123.234  137.30.120.37   2       G
49      2004-04-28T21:08:38.073111000Z  137.30.123.234  137.30.120.37   1       G
217     2004-04-28T21:08:44.189294000Z  137.30.123.234  137.30.120.37   2       G
333     2004-04-28T21:08:58.067074000Z  137.30.123.234  64.233.167.104  4       G
343     2004-04-28T21:09:14.099217000Z  137.30.123.234  137.30.120.37   5       G
347     2004-04-28T21:09:14.422408000Z  137.30.123.234  137.30.120.37   5       G
359     2004-04-28T21:09:57.094465000Z  137.30.123.234  137.30.120.37   6       G

┌──(kali㉿kali)-[~/case4/evidence/http_objects]
└─$
```

Figure 69. rhino2.log: request list with TCP stream numbers saved to a file.

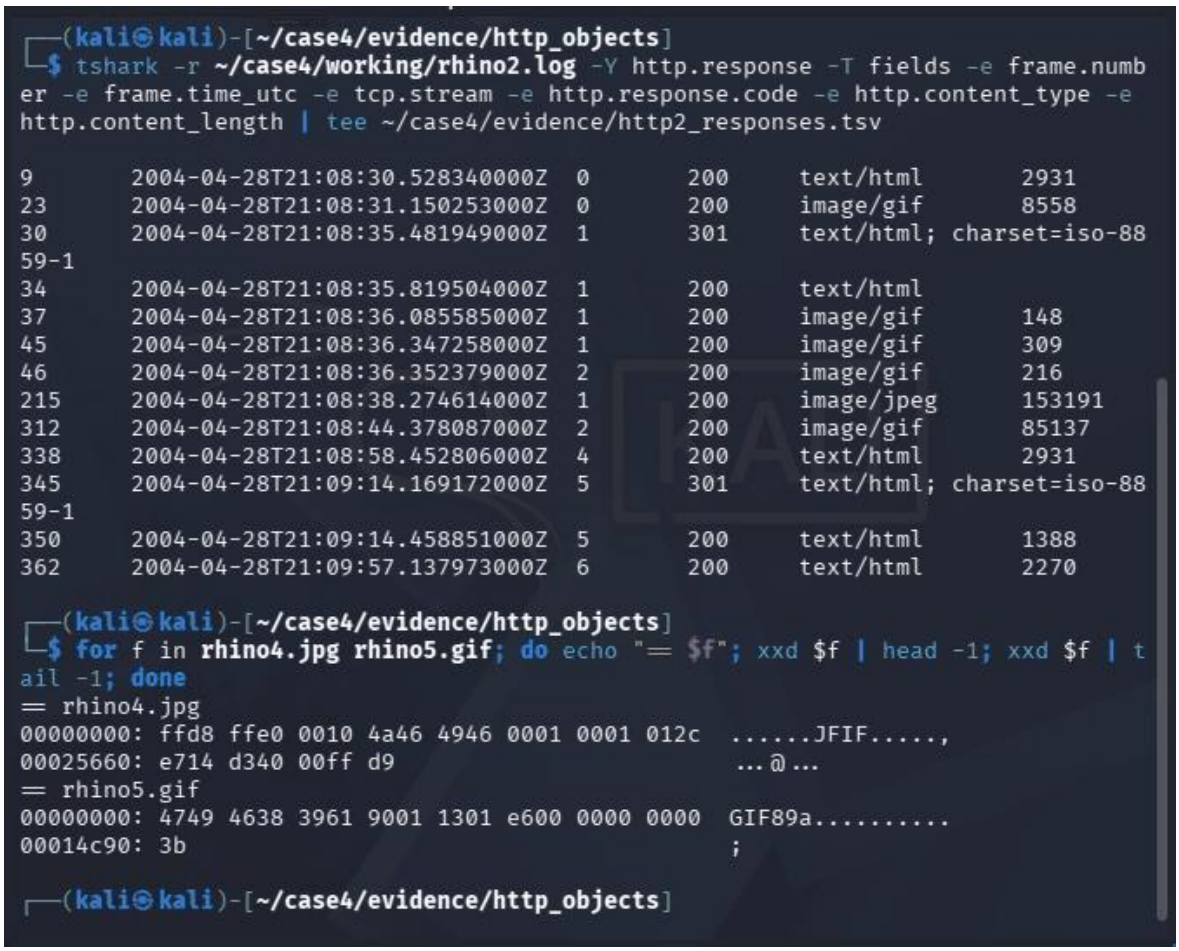


Figure 70. rhino2.log: responses (status, content type, length) and header/trailer bytes of rhino4.jpg and rhino5.gif.

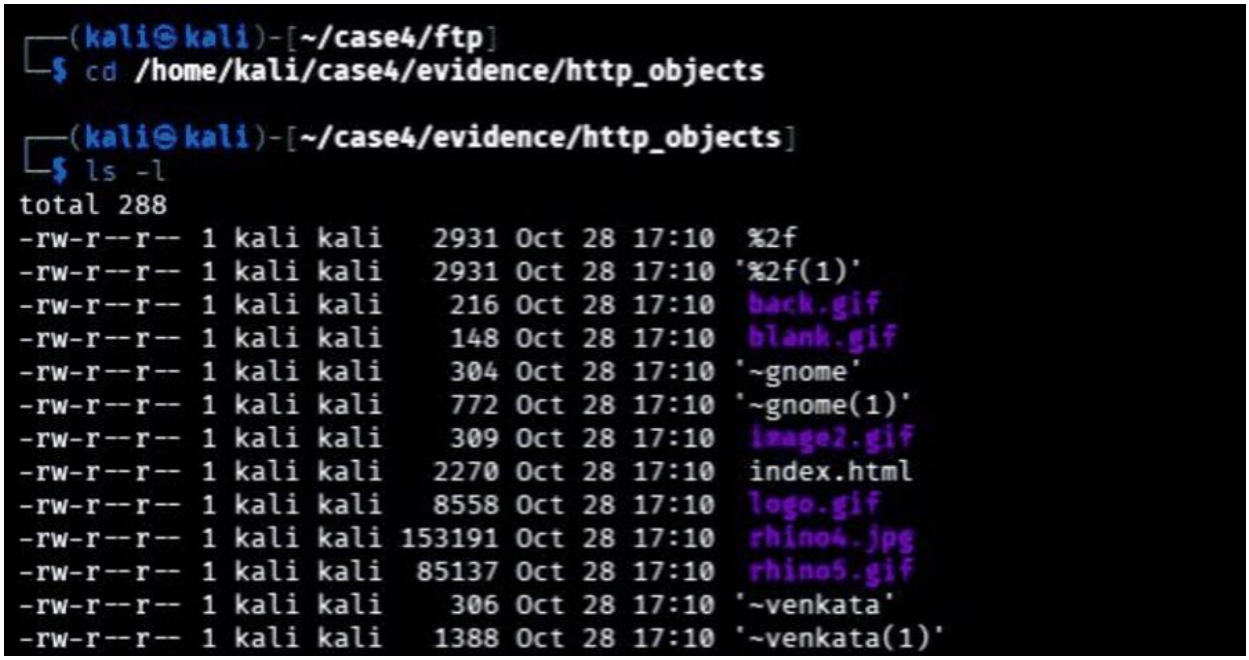


Figure 71. rhino2.log: 13 objects exported with tshark and their sizes.

```
┌──(kali㉿kali)-[~/case4/evidence/http_objects]
└─$ sha256sum * | tee ~/case4/evidence/http_objects_sha256.txt
4a80ca48bc940ae8fdd84ca656661beb6e435ee3879286e9a8bd034ec3410a41  %2f
4a80ca48bc940ae8fdd84ca656661beb6e435ee3879286e9a8bd034ec3410a41  %2f(1)
0e9ced1019385b1101fdaa7c07d01d63aae771176a44d3df81d47b115b5128aa  back.gif
3cb0e54babf019703fe671a32fcc3947aab9079ec2871cf0f9639245cc12d878  blank.gif
91caf30513889d010504a336601c4ea6d1bd1a16638e3b73367a8d766f0b805f  ~gnome
724175606883816dd06fe845aaaa25c83fae37f28bcff26aed75e4a5c13a5efc  ~gnome(1)
1e09d5e4e03c57ba24c23b84cc4af3ce66cb44259849d929b911711c25d25c5e  image2.gif
d1ef300899429f26fd99205e4f46c9904aae0d841f7b406b38a63384e0db0e37  index.html
b89fe394c4ed380e4a4fac663ac6193dc04219464054bad9033765fc6a1e0311  logo.gif
e1c01da391669adf3559d2d87bf2157b37e27a936a1ae66c77b61e6b5b7bc4a6  rhino4.jpg
f36ecc12967d5622cdc70e73d2468fab28a65db944d3157256a7212afa05b056  rhino5.gif
e8fb7cb876122ed4bd110a62a0d5793db4d92ff6e63bba35097cd572c276f44f  ~venkata
f8e891c8e5745044a87efece1254c8a65151fbcf24eaf4e8e157db26805d16d0  ~venkata(1)

┌──(kali㉿kali)-[~/case4/evidence/http_objects]
└─$ file rhino4.jpg rhino5.gif
rhino4.jpg: JPEG image data, JFIF standard 1.00, resolution (DPI), density 300×30
0, segment length 16, comment: "U-Lead Systems, Inc.", baseline, precision 8, 153
1×2286, components 3
rhino5.gif: GIF image data, version 89a, 400 x 275
```

Figure 72. rhino2.log: SHA-256 of the exported objects; file output for rhino4.jpg and rhino5.gif.

```
┌──(kali㉿kali)-[~/case4/evidence/http_objects]
└─$ sha256sum ~/case4/ftp/rhino1.jpg ~/case4/ftp/rhino3.jpg
0f30a8777d19a4fe4b2fafd5154b6c4dfafb673202db29971e876e978da4e05d  /home/kali/cas
e4/ftp/rhino1.jpg
a85aea8e47d9f5425456f3bafb9c67201c8c2b2b2cec80154c83102e802240c5  /home/kali/cas
e4/ftp/rhino3.jpg

┌──(kali㉿kali)-[~/case4/evidence/http_objects]
└─$ cut -d' ' -f1 ~/case4/evidence/http_objects_sha256.txt | grep -f - \
  ~/case4/evidence/analysis_images_sha256.txt \
  ~/case4/evidence/recovered_sha256.txt \
  ~/case4/evidence/steg_extracted_sha256.txt \
  ~/case4/evidence/rhino2_extracted_sha256.txt \
  2>/dev/null
echo "exit code: $?"
exit code: 1

┌──(kali㉿kali)-[~/case4/evidence/http_objects]
└─$
```

Figure 73. SHA-256 of the FTP images rhino1.jpg and rhino3.jpg; hash comparison of the HTTP objects against other evidence (exit code 1: no match).

Interpretation. The client retrieved two image files, `rhino4.jpg` and `rhino5.gif`, from the /~gnome/ directory on `www.cs.uno.edu`, about six seconds apart. Because the response size equals Content-Length and the file ends correctly, both objects are complete. The traffic shows that the files were delivered to the client. It does not show whether they were saved or viewed.

9.2 rhino3.log

Observation. The protocol hierarchy shows 295 frames and 221,350 bytes, of which 27 frames are HTTP, with 4 IMAP frames and 1 MSN Messenger frame (Figure 74). There are 12 requests, again from 137.30.123.234 (Figure 75):

Frame	Time (UTC, 2004-04-28)	Stream	Host (server)	URI
4	21:15:41.262422	0	www.google.com (64.233.167.99)	/
10	21:15:48.234997	0	www.google.com	/search?hl=en&ie=UTF-8&oe=UTF-8&q=rhino.exe
30	21:15:48.774630	0	www.google.com	/nav_first.gif
35	21:15:48.934559	1	www.google.com	/images/logo_sm.gif
38	21:15:49.105241	0	www.google.com	/nav_current.gif
46	21:15:49.435096	0	www.google.com	/nav_page.gif
49	21:15:49.773058	0	www.google.com	/nav_next.gif
59	21:15:55.113910	2	groups.google.com (216.239.51.99)	/groups?q=rhino.exe&hl=en&…&sa=N&tab=wg
73	21:15:55.627839	2	groups.google.com	/intl/en_ALL/images/groups_res.gif
85	21:16:02.002505	2	groups.google.com	/groups?q=rhino.exe&…&selm=MPG.159359f528ff3d2798aaca%40news.ntlworld.com&rnum=2
101	21:16:09.221371	1	www.google.com	/
110	21:16:23.343061	3	www.cs.uno.edu (137.30.120.37)	/~gnome/rhino.exe

The capture holds eight HTTP responses (Figure 76). None of them belongs to the three `groups.google.com` requests (stream 2), so those responses are not in the capture. The response to the last request, frame 274 at 21:16:24.214Z, is `200 application/octet-stream` with a length of **145,920 bytes**. Exporting the objects gives `rhino.exe` at 145,920 bytes, equal to the Content-Length, with SHA-256 `93e70049b60bf5691b43d9e256ca8f459f6bfac832097985bcc6a9040b5ca555` (confirmed again directly in the static-analysis run, Figure 77).

```
┌──(kali㉿kali)-[~/case4/analysis/unique]
└─$ tshark -r ~/case4/working/rhino3.log -q -z io,phs | tee ~/case4/evidence/http
3_protocols.txt

===================================================================
Protocol Hierarchy Statistics
Filter:

frame                                    frames:295 bytes:221350
  eth                                    frames:295 bytes:221350
    ip                                   frames:295 bytes:221350
      tcp                                frames:295 bytes:221350
        http                             frames:27 bytes:19498
          data-text-lines                frames:3 bytes:4080
          image-gif                      frames:4 bytes:4044
          data                           frames:1 bytes:302
        msnms                            frames:1 bytes:60
        imap                             frames:4 bytes:347
===================================================================
```

Figure 74. rhino3.log: protocol hierarchy (HTTP, MSN Messenger and IMAP frames).

```
┌──(kali㉿kali)-[~/case4/analysis/unique]
└─$ tshark -r ~/case4/working/rhino3.log -Y http.request -T fields -e frame.numbe
r -e frame.time_utc -e ip.src -e ip.dst -e tcp.stream -e http.request.method -e h
ttp.host -e http.request.uri | tee ~/case4/evidence/http3_requests.tsv
4       2004-04-28T21:15:41.262422000Z  137.30.123.234  64.233.167.99   0       G
ET      www.google.com  /
10      2004-04-28T21:15:48.234997000Z  137.30.123.234  64.233.167.99   0       G
ET      www.google.com  /search?hl=en&ie=UTF-8&oe=UTF-8&q=rhino.exe
30      2004-04-28T21:15:48.774630000Z  137.30.123.234  64.233.167.99   0       G
ET      www.google.com  /nav_first.gif
35      2004-04-28T21:15:48.934559000Z  137.30.123.234  64.233.167.99   1       G
ET      www.google.com  /images/logo_sm.gif
38      2004-04-28T21:15:49.105241000Z  137.30.123.234  64.233.167.99   0       G
ET      www.google.com  /nav_current.gif
46      2004-04-28T21:15:49.435096000Z  137.30.123.234  64.233.167.99   0       G
ET      www.google.com  /nav_page.gif
49      2004-04-28T21:15:49.773058000Z  137.30.123.234  64.233.167.99   0       G
ET      www.google.com  /nav_next.gif
59      2004-04-28T21:15:55.113910000Z  137.30.123.234  216.239.51.99   2       G
ET      groups.google.com       /groups?q=rhino.exe&hl=en&lr=&ie=UTF-8&oe=UTF-8&s
a=N&tab=wg
73      2004-04-28T21:15:55.627839000Z  137.30.123.234  216.239.51.99   2       G
ET      groups.google.com       /intl/en_ALL/images/groups_res.gif
85      2004-04-28T21:16:02.002505000Z  137.30.123.234  216.239.51.99   2       G
ET      groups.google.com       /groups?q=rhino.exe&hl=en&lr=&ie=UTF-8&oe=UTF-8&s
elm=MPG.159359f528ff3d2798aaca%40news.ntlworld.com&rnum=2
101     2004-04-28T21:16:09.221371000Z  137.30.123.234  64.233.167.99   1       G
ET      www.google.com  /
110     2004-04-28T21:16:23.343061000Z  137.30.123.234  137.30.120.37   3       G
ET      www.cs.uno.edu  /~gnome/rhino.exe

┌──(kali㉿kali)-[~/case4/analysis/unique]
```

Figure 75. rhino3.log: HTTP requests, including the Google searches for "rhino.exe" and the request for /~gnome/rhino.exe.

```
┌──(kali㉿kali)-[~/case4/analysis/unique]
└─$ tshark -r ~/case4/working/rhino3.log -Y http.response -T fields -e frame.numb
er -e frame.time_utc -e tcp.stream -e http.response.code -e http.content_type -e
http.content_length | tee ~/case4/evidence/http3_responses.tsv
8       2004-04-28T21:15:41.369487000Z  0       200     text/html       2931
28      2004-04-28T21:15:48.725700000Z  0       200     text/html
31      2004-04-28T21:15:48.850968000Z  0       200     image/gif       1033
40      2004-04-28T21:15:49.179291000Z  0       200     image/gif       376
47      2004-04-28T21:15:49.518101000Z  0       200     image/gif       373
54      2004-04-28T21:15:50.070417000Z  1       200     image/gif       4707
105     2004-04-28T21:16:09.336902000Z  1       200     text/html       2931
274     2004-04-28T21:16:24.214222000Z  3       200     application/octet-stream1
45920

┌──(kali㉿kali)-[~/case4/analysis/unique]
└─$ mkdir -p ~/case4/evidence/http3_objects

┌──(kali㉿kali)-[~/case4/analysis/unique]
└─$ tshark -r ~/case4/working/rhino3.log -q --export-objects http,$HOME/case4/evi
dence/http3_objects
```

Figure 76. rhino3.log: responses (status, type, length) and the export of HTTP objects.

```
┌──(kali㉿kali)-[~/case4/analysis/unique]
└─$ cd ~/case4/evidence/http3_objects && ls -l && sha256sum * | tee ../http3_obje
cts_sha256.txt && file *
total 208
-rw-r--r-- 1 kali kali   2931 Oct 31 03:58  %2f
-rw-r--r-- 1 kali kali   2931 Oct 31 03:58 '%2f(1)'
-rw-r--r-- 1 kali kali   4707 Oct 31 03:58  logo_sm.gif
-rw-r--r-- 1 kali kali    376 Oct 31 03:58  nav_current.gif
-rw-r--r-- 1 kali kali   1033 Oct 31 03:58  nav_first.gif
-rw-r--r-- 1 kali kali    373 Oct 31 03:58  nav_page.gif
-rw-r--r-- 1 kali kali    290 Oct 31 03:58  object50
-rw-r--r-- 1 kali kali     89 Oct 31 03:58  object68
-rw-r--r-- 1 kali kali   1430 Oct 31 03:58  object83
-rw-r--r-- 1 kali kali      5 Oct 31 03:58  object95
-rw-r--r-- 1 kali kali   1008 Oct 31 03:58  object97
-rw-r--r-- 1 kali kali 145920 Oct 31 03:58  rhino.exe
-rw-r--r-- 1 kali kali  15160 Oct 31 03:58 'search%3fhl=en&ie=UTF-8&oe=UTF-8&q=rh
ino.exe'
4a80ca48bc940ae8fdd84ca656661beb6e435ee3879286e9a8bd034ec3410a41  %2f
4a80ca48bc940ae8fdd84ca656661beb6e435ee3879286e9a8bd034ec3410a41  %2f(1)
a604ce5820ea50a05f20508afc5be7693cdb6ab1bea34bdebc7fa971d17dcdc4  logo_sm.gif
88ddae9d4c3e23761d8015ebc7c5a4132992290d7d8231f38360a49d9d7da8f1  nav_current.gif
fc169c1aa8486b7d2db4ea938f81c4a4a8bba5e2c0e6e0595917383fb0304047  nav_first.gif
b37ba987481fc6064061e3d9c687d002fcba91313da07712048923d0ce1e8a81  nav_page.gif
764b07fb3d3406eb1acb227df92c2cd61af3505e784249df85b19a62e704fdd1  object50
7478e5e50c40e32118356bc1c26ca1f9b03eb88cd73f5aab64d6338fb1a5a85b  object68
269fe915a75c5d59d79e6d5abdfa8f5f7b2ec494c6d58e0b367c911cc0342d5c  object83
933b971c6388d594a23fa1559825db5bec8ade2db1240aa8fc9d0c684949e8c9  object95
674ba2027f9fc3e33c935df3dc5d3b0bd6ab34d8a725b0c58565d27c7394ac42  object97
93e70049b60bf5691b43d9e256ca8f459f6bfac832097985bcc6a9040b5ca555  rhino.exe
b778fbfc48966b37c616e8b9a7f333cd18612a398ed627a88a0db324d226f5c1  search%3fhl=en&
ie=UTF-8&oe=UTF-8&q=rhino.exe
```

Figure 77. rhino3.log: exported objects, sizes and SHA-256 values (rhino.exe is 145,920 bytes).

Static checks only. `file` identifies the object as a **PE32 executable for MS Windows 4.00 (console), Intel i386, 3 sections**, not a renamed image or a disguised file type. It begins with the bytes 4D5A ("MZ", the standard DOS/PE signature) and ends in zero padding, consistent with a complete file. `strings` shows it links against standard Windows system DLLs (`KERNEL32.dll`, `USER32.dll`, `ADVAPI32.dll`, `RPCRT4.dll`, `ole32.dll`, `SETUPAPI.dll`) and references `mscoree.dll`, the .NET/CLR loader, along with generic C-runtime error strings. I recorded its SHA-256 as `93e70049b60bf5691b43d9e256ca8f459f6bfac832097985bcc6a9040b5ca555`

(Figure 78). I did **not** run rhino.exe, did not connect to any address found in it, and did not submit its hash to any online service, as the brief requires.

```
┌──(kali㉿kali)-[~/case4/evidence/http3_objects]
└─$ {
  echo "=== rhino.exe static analysis ==="
  date -u
  ls -l rhino.exe
  file rhino.exe
  sha256sum rhino.exe
  echo "--- header ---"
  xxd rhino.exe | head -2
  echo "--- trailer ---"
  xxd rhino.exe | tail -2
  echo "--- strings (first 30) ---"
  strings -n 8 rhino.exe | head -30
} | tee ~/case4/evidence/rhino_exe_static_analysis.txt
=== rhino.exe static analysis ===
Tue Nov  3 11:03:54 AM UTC 2026
-rw-r--r-- 1 kali kali 145920 Oct 31 03:58 rhino.exe
rhino.exe: PE32 executable for MS Windows 4.00 (console), Intel i386, 3 sections
93e70049b60bf5691b43d9e256ca8f459f6bfac832097985bcc6a9040b5ca555  rhino.exe
--- header ---
00000000: 4d5a 9000 0300 0000 0400 0000 ffff 0000  MZ..............
00000010: b800 0000 0000 0000 4000 0000 0000 0000  ........@.......
--- trailer ---
000239e0: 0000 0000 0000 0000 0000 0000 0000 0000  ................
000239f0: 0000 0000 0000 0000 0000 0000 0000 0000  ................
--- strings (first 30) ---
!This program cannot be run in DOS mode.
ADVAPI32.dll  KERNEL32.dll  NTDLL.DLL  RPCRT4.dll
ole32.dll  SETUPAPI.dll  USER32.dll
SetThreadUILanguage
LC_NUMERIC  LC_MONETARY  LC_CTYPE  LC_COLLATE
CorExitProcess
mscoree.dll
runtime error / TLOSS error / SING error / DOMAIN error
- unable to initialize heap
- not enough space for lowio / stdio initialization
- pure virtual function call
- unable to open console device
- unexpected heap / multithread lock error

NOTE: file was not executed; no network connection attempted;
hash was not submitted to any online scanner (per brief restrictions).
```

Figure 78. Static analysis of rhino.exe: confirmed PE32 Windows executable, SHA-256 recorded, header/trailer bytes shown, and C-runtime/DLL strings; the file was not executed.

Interpretation. From one client, a Google search for "rhino.exe" at 21:15:48Z was followed by a Google Groups search for the same term and, **35 seconds after the first search**, by a request for `/~gnome/rhino.exe`, which returned a complete 145,920-byte file. That sequence is consistent with looking for and then downloading the program. It does not show that the program was run, what it does, or why it was wanted. The few IMAP and MSN Messenger frames fall outside the HTTP question and may contain credentials, so I did not analyse them.

10. Correlation, timeline and conclusion (E28)

10.1 Master image hash table

Observation. I hashed every image from every source into one table and listed the duplicate hashes (Figure 79, Figure 80). The table has **16 entries and 14 unique hashes**. Two pairs are duplicates: `f0105873.jpg` and `f0335081.jpg` (both on the disk), and `f0106409.jpg` (disk) and `rhino2.jpg` (extracted from the FTP archive). The full table is in Appendix B.

```
┌──(kali㉿kali)-[~/case4]
└─$ ( for f in analysis/images/*; do echo "$(sha256sum "$f" | cut -d' ' -f1)  DIS
K  $(basename "$f")"; done
  for f in steg/extract_work/out_*.jpg; do echo "$(sha256sum "$f" | cut -d' ' -f1
)  STEG  $(basename "$f")"; done
  for f in ftp/rhino1.jpg ftp/rhino3.jpg evidence/contraband_extracted/rhino2.jpg
; do echo "$(sha256sum "$f" | cut -d' ' -f1)  FTP   $(basename "$f")"; done
  for f in evidence/http_objects/rhino4.jpg evidence/http_objects/rhino5.gif; do
echo "$(sha256sum "$f" | cut -d' ' -f1)  HTTP  $(basename "$f")"; done ) | tee ev
idence/master_image_hashes.txt
cut -d' ' -f1 evidence/master_image_hashes.txt | sort | uniq -d | grep -F -f - ev
idence/master_image_hashes.txt
2261020a0b31fe10c3ae2872c6283e42993e24e87f1a76822e5840ed1838014e  DISK  f0104057.
jpg
8cdb89c6778d3409a80355bf2b9c644c1bf0c5e31c43047080da9ea5a859de07  DISK  f0104249.
jpg
452075313c93ae313331a86e8cac2f35ea3bdc24700cabf4444e39f5836403ea  DISK  f0105065.
jpg
f92654d9ee17ab6b684b09de01cf0bc4076383c007964946d3f31577447596fb  DISK  f0105873.
jpg
568457ed594c4e98dc503e3e244b1e316376aafb0d1c283d95601d13762a748c  DISK  f0106393.
jpg
b4f6bbb8d846d6505c9b5b67f115656fdf980a484aa441078a838a3ffc2ac705  DISK  f0106409.
jpg
8a67d406ed130c9b5447b03fbf9dd0ac6c384a62b5028d6267e92c4abe230a80  DISK  f0106865.
gif
71e9f6b94e0496cc634d5d643387c7c1405e2df9deccda1482a8c7cc5ba0b199  DISK  f0106889.
gif
f92654d9ee17ab6b684b09de01cf0bc4076383c007964946d3f31577447596fb  DISK  f0335081.
jpg
6dbf775b8999a006ffa74c65127e5add64234184bd2f425b2ad644e8c0d19345  STEG  out_f0104
249.jpg
```

Figure 79. *Master image hash table, first part (disk, steganography extractions).*

```
568457ed594c4e98dc503e3e244b1e316376aafb0d1c283d95601d13762a748c  DISK  f0106393.
jpg
b4f6bbb8d846d6505c9b5b67f115656fdf980a484aa441078a838a3ffc2ac705  DISK  f0106409.
jpg
8a67d406ed130c9b5447b03fbf9dd0ac6c384a62b5028d6267e92c4abe230a80  DISK  f0106865.
gif
71e9f6b94e0496cc634d5d643387c7c1405e2df9deccda1482a8c7cc5ba0b199  DISK  f0106889.
gif
f92654d9ee17ab6b684b09de01cf0bc4076383c007964946d3f31577447596fb  DISK  f0335081.
jpg
6dbf775b8999a006ffa74c65127e5add64234184bd2f425b2ad644e8c0d19345  STEG  out_f0104
249.jpg
bc9c605372fdf9cc00faf17ae9ce0fdb6d26bb9db28cdff742d9ddbe1470ec16  STEG  out_f0105
065.jpg
0f30a8777d19a4fe4b2fafd5154b6c4dfafb673202db29971e876e978da4e05d  FTP   rhino1.jp
g
a85aea8e47d9f5425456f3bafb9c67201c8c2b2b2cec80154c83102e802240c5  FTP   rhino3.jp
g
b4f6bbb8d846d6505c9b5b67f115656fdf980a484aa441078a838a3ffc2ac705  FTP   rhino2.jp
g
e1c01da391669adf3559d2d87bf2157b37e27a936a1ae66c77b61e6b5b7bc4a6  HTTP  rhino4.jp
g
f36ecc12967d5622cdc70e73d2468fab28a65db944d3157256a7212afa05b056  HTTP  rhino5.gi
f
f92654d9ee17ab6b684b09de01cf0bc4076383c007964946d3f31577447596fb  DISK  f0105873.
jpg
b4f6bbb8d846d6505c9b5b67f115656fdf980a484aa441078a838a3ffc2ac705  DISK  f0106409.
jpg
f92654d9ee17ab6b684b09de01cf0bc4076383c007964946d3f31577447596fb  DISK  f0335081.
jpg
b4f6bbb8d846d6505c9b5b67f115656fdf980a484aa441078a838a3ffc2ac705  FTP   rhino2.jp
g
```

Figure 80. *Master image hash table, remainder (FTP, HTTP) and the duplicate-hash listing.*

10.2 Visual review and classification

I built a labelled contact sheet of the 14 unique image files so that I could review all of them together (Figure 83, Figure 84). The thumbnails are pixelated in this report. I reviewed the sheet and confirmed every description below against the images themselves; I also opened `rhino4.jpg` and `rhino5.gif` individually in `eog` to settle the two cases the contact sheet left uncertain (Figure 81, Figure 82).

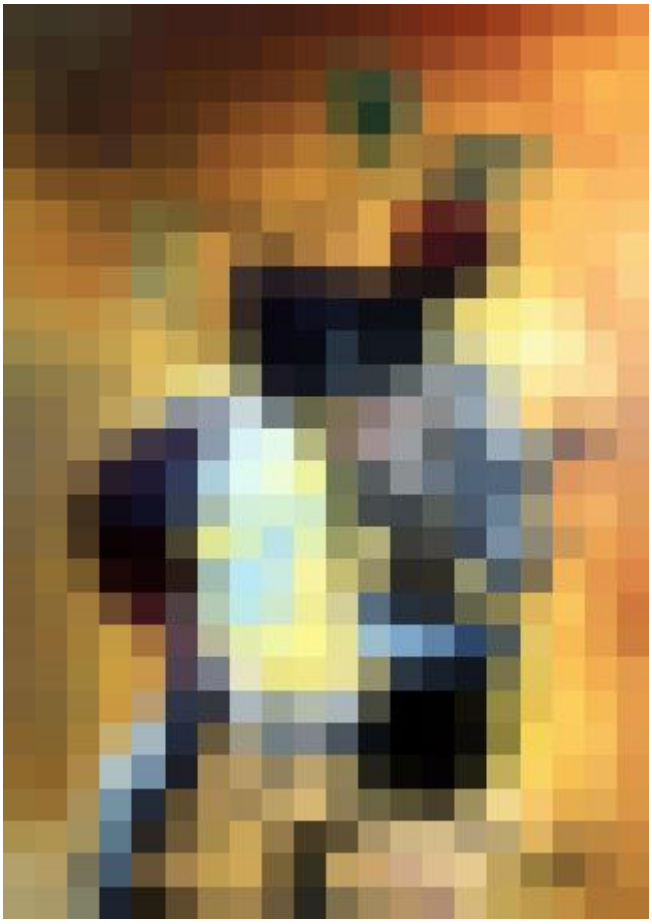


Figure 81. rhino4.jpg viewed directly (pixelated for this report): a surrealist painting containing a rhinoceros figure among other symbolic elements.



Figure 82. rhino5.gif viewed directly (pixelated for this report): a graphite sketch of an origami-style rhinoceros

.

```
┌──(kali㉿kali)-[~/case4/working]
└─$ cd ~/case4 && mkdir -p analysis/unique && cp -p analysis/images/f0104057.jpg
analysis/images/f0104249.jpg analysis/images/f0105065.jpg analysis/images/f010587
3.jpg analysis/images/f0106393.jpg analysis/images/f0106409.jpg analysis/images/f
0106865.gif analysis/images/f0106889.gif steg/extract_work/out_f0104249.jpg steg/
extract_work/out_f0105065.jpg ftp/rhino1.jpg ftp/rhino3.jpg evidence/http_objects
/rhino4.jpg evidence/http_objects/rhino5.gif analysis/unique/

┌──(kali㉿kali)-[~/case4]
└─$ cd analysis/unique && montage -label '%f' *.jpg '*.gif[0]' -tile 5x -geometry
 260×200+6+6 ../all_unique_contact_sheet.png && eog ../all_unique_contact_sheet.p
ng

┌──(kali㉿kali)-[~/case4/analysis/unique]
└─$
```

Figure 83. Command used to build a labelled contact sheet of the unique image files.

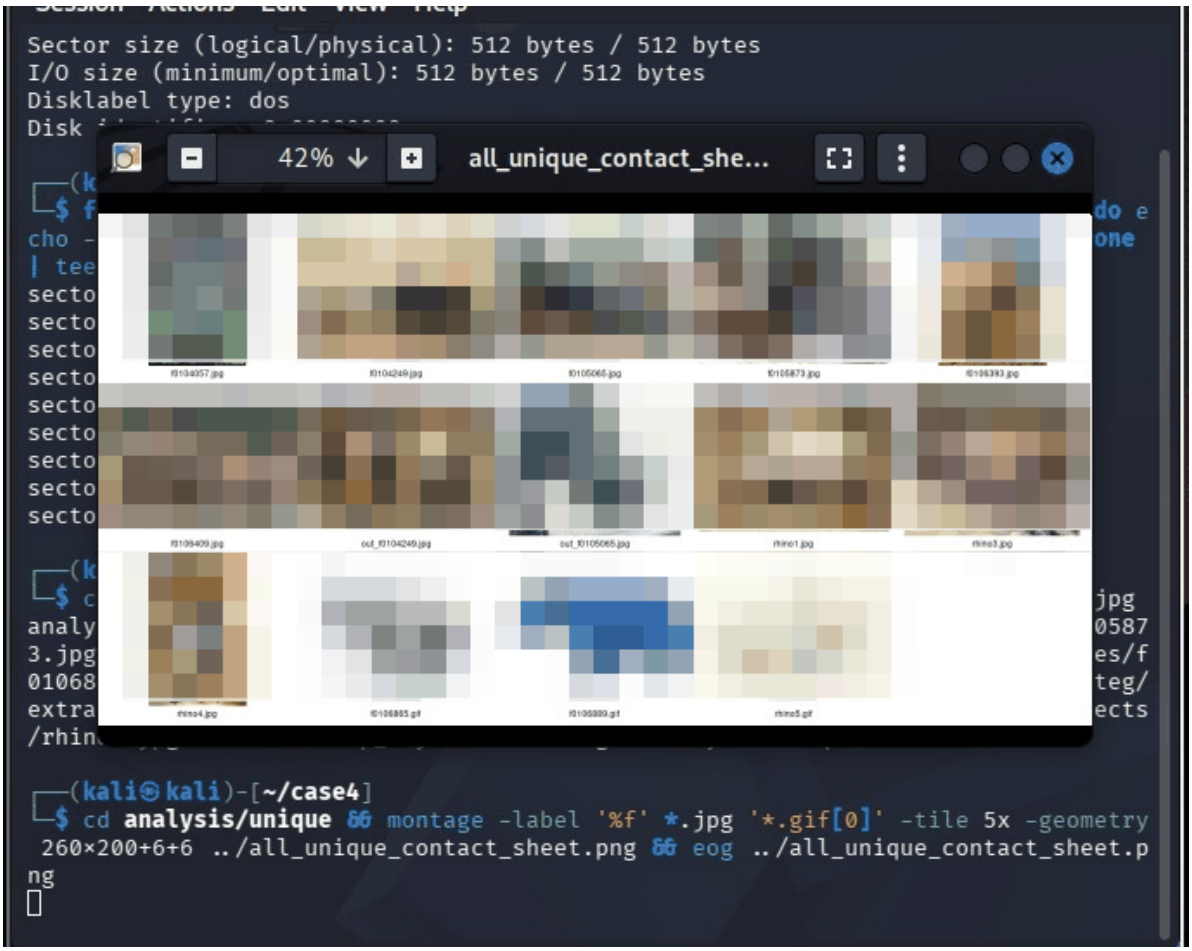


Figure 84. ***Contact sheet of the 14 unique image files. Thumbnails are pixelated in this report; file names remain legible.***

#	File	Source	Appears to show	Rhinoceros?
1	f0104057.jpg	USB (carved)	Close-up of a crocodilian head	No
2	f0104249.jpg	USB (carved); steganography carrier	Crocodilian in water	No
3	f0105065.jpg	USB (carved); steganography carrier	Crocodilian on rocks	No
4	f0105873.jpg = f0335081.jpg	USB (carved, two copies)	Young crocodilian held in a hand with an egg shell	No
5	f0106393.jpg	USB (carved)	A rhinoceros standing	Yes (photograph)
6	f0106409.jpg = FTP rhino2.jpg	USB (carved) and FTP archive	A rhinoceros with a calf	Yes (photograph)
7	f0106865.gif	USB (carved)	Pencil or stipple drawing of a rhinoceros	Yes (drawing)
8	f0106889.gif	USB (carved)	Blue computer-rendered rhinoceros	Yes (rendering)
9	out_f0104249.jpg	USB (hidden in #2)	Three rhinoceroses	Yes (photograph)

10	out_f0105065.jpg	USB (hidden in #3)	Close-up of a rhinoceros head	Yes (photograph)
11	rhino1.jpg	FTP upload	A rhinoceros in grass	Yes (photograph)
12	rhino3.jpg	FTP upload	A rhinoceros with a calf	Yes (photograph)
13	rhino4.jpg	HTTP download	Surrealist painting: a gray rhinoceros standing on a hat, within a larger symbolic collage (tree branch, sunglasses, apple, hand)	Yes (painting)
14	rhino5.gif	HTTP download	Graphite sketch of an origami-style rhinoceros, signed "K."	Yes (drawing)

10.3 Counting unique rhinoceros images

Source	Unique rhinoceros images	Files	Running total
USB image, carved from unallocated space	4	f0106393, f0106409, f0106865.gif, f0106889.gif	4
USB image, hidden by steganography	2	out_f0104249, out_f0105065	6 on the USB image alone
FTP uploads (new beyond the USB image)	2	rhino1.jpg, rhino3.jpg (rhino2.jpg repeats f0106409)	8
HTTP downloads	2	rhino4.jpg (confirmed by direct viewing), rhino5.gif	10 unique rhinoceros images in total

Interpretation. Counting by hash, the combined evidence holds **10 unique rhinoceros images**. I confirmed every one by directly viewing it, including `rhino4.jpg`, which turned out to be a surrealist painting containing a rhinoceros as one element of a larger composition rather than a straightforward photograph. The **USB image alone holds six**, below the threshold of nine. The threshold is reached only when images seen in network traffic are added, and the network traces show transfer or retrieval, not possession on a device. Of the ten, six are photographs, three are drawings or renderings, and one is a painting, so whether non-photographic depictions count as "images" is a question the scenario's law must settle; I do not decide it here. A hash-unique count is also not a picture-unique count, because a re-encoded copy of one picture would have a different hash.

10.4 Timeline

Times are UTC for the network captures and the examination. File-system, zip and document times have no time zone and are shown as stored; I mark them "zone unknown".

Time	Source	Event	Evidence
2004-04-26 17:00:12 (zone unknown)	Zip entry time	rhino2.jpg last modified, as stored in contraband.zip. At UTC-4 or UTC-5 this is 21:00:12Z or 22:00:12Z, before the upload below.	E23, E24
2004-04-26 22:21:49.954Z	rhino.log	FTP upload of rhino1.jpg (65,703 bytes) as account "gnome" from 137.30.122.253 to 137.30.120.40; completed 22:21:50.180Z	E21, E22
2004-04-26 22:22:08.906Z	rhino.log	FTP upload of rhino3.jpg in ASCII mode (96,899 bytes); completed 22:22:09.040Z with a line-feed warning	E21, E22
2004-04-26 22:26:46.672Z	rhino.log	FTP upload of contraband.zip (230,566 bytes); completed 22:26:46.877Z; session closed 22:26:48.543Z	E21, E23
2004-04-28 21:08:38.073Z	rhino2.log	Client 137.30.123.234 requests /~gnome/rhino4.jpg; 153,191-byte reply at 21:08:38.275Z	E26
2004-04-28 21:08:44.189Z	rhino2.log	Same client requests /~gnome/rhino5.gif; 85,137-byte reply at 21:08:44.378Z	E26
2004-04-28 21:15:48.235Z	rhino3.log	Same client searches Google for "rhino.exe"; Google Groups search at 21:15:55.114Z	E27
2004-04-28 21:16:23.343Z	rhino3.log	Same client requests /~gnome/rhino.exe; 145,920-byte reply at 21:16:24.214Z	E27
2004-04-30 18:11:20 and 18:11:24 (zone unknown)	FAT directory	gumbo1.txt and gumbo2.txt written; the latest file-system times on the volume	E08
2004-04-30 21:29:47 (zone unknown)	Package file time	Time stored for RHINOUSB.dd in the course package	E04
2005-08-08 to 2005-08-09	Document and JPEG metadata	Embedded dates in the carved Word document and in both extracted JPEGs; later than the 2004 events, so unreliable for ordering	E14, E20
2026-10-25 10:35:21Z and 10:40:44Z	Examiner	Session start and package saved; analysis followed on 25 to 31 October 2026	E01, E02

Interpretation. The network events fall in a clear order: three FTP uploads on 26 April, then HTTP retrievals on 28 April. The file-system times (30 April) are later, but their time zone is unknown and the two sets of dates come from different kinds of source, so I do not claim that the USB volume was last written after the network activity.

10.5 Correlation of artefacts

Link	Evidence	Strength	What it does not show
USB image to FTP	The same SHA-256 for the carved f0106409.jpg and rhino2.jpg inside the uploaded archive; the archive entry size equals the carved file size (230,665 bytes)	Strong for identical content	Which copy came first, who moved it, or that the USB user performed the upload

FTP to HTTP	The FTP login name "gnome" and the web path /~gnome/ on the university server; rhino-named files in both	Weak to moderate (a name only)	The client addresses and server addresses differ (137.30.122.253 and 137.30.120.40 against 137.30.123.234 and 137.30.120.37), and no image hash is shared
Search to download	From the same client, a Google search for "rhino.exe" 35 seconds before the request for /~gnome/rhino.exe	Moderate (sequence)	That the program was run, what it does, or why it was fetched
Recovered document to disk state	The document says the writer would reformat a USB key; the volume has two small files, no deleted entries and a non-zero unallocated area	Consistent, not proof	Who wrote the document or when; the file-system state alone does not show a reformat
Steganography to other sources	Two hidden rhinoceros images were extracted from crocodilian carriers; they match nothing in the network traffic	Independent content on the USB image	Who hid them or when

10.6 Conclusion and confidence

- **What the evidence supports.** The preserved USB image contains six unique rhinoceros images, four recovered from unallocated space and two hidden by jphide inside crocodilian carrier images. The FTP capture shows three uploads under the account "gnome", one of which contains an image identical to a file on the USB image. The HTTP captures show retrieval of two further rhinoceros-named images and the download of a program named rhino.exe, preceded by a search for that name.
- **Threshold.** Across the USB image and the network traffic combined, there are **10** unique rhinoceros images by hash, all confirmed by direct viewing. On the USB image alone there are six.
- **What the evidence does not support.** It does not establish that one person used the USB device, the FTP client and the HTTP client, that the program was executed, or when the images were deleted from the USB volume.
- **Confidence.** High for the technical facts (hash-verified files, validated headers and trailers, byte counts that agree). Moderate for a link between the FTP and HTTP activity through the "gnome" name. Low for any attribution to a named person.

10.7 Limitations

8. **Attribution.** No artefact ties the USB image, the FTP client (137.30.122.253) and the HTTP client (137.30.123.234) to one person or machine.
9. **Timestamps.** FAT, zip and document times have no time zone, my virtual machine labels them with its own offset, and the embedded 2005 dates conflict with the 2004 evidence.

10. **Incomplete tool runs.** stegbreak crashed, so two candidates are inconclusive. I viewed only part of the slack space and of the two large filler files.
11. **FTP ASCII mode.** The server may have altered rhino3.jpg on receipt, so the stored copy could differ from the bytes I rebuilt.
12. **Partial captures.** The groups.google.com responses are absent from rhino3.log, the Google fetch in rhino2.log shows out-of-order and missing segments, and I did not analyse the IMAP and MSN Messenger frames.
13. **Counting.** A hash-unique count is not a picture-unique count, my classification is a visual judgement, and three of the nine images are drawings or renderings.
14. **Claims in recovered documents** are unverified statements.
15. **rhino.exe** was examined statically only, so its behaviour is unknown.
16. **Possession versus transfer.** Network traces show that content was sent or retrieved, not that it was stored on a device.

Appendix A. Evidence log

ID	Tool or method	Observation	Figures
E01	Shell, `date -u`	Case folders created; examination start 2026-10-25 10:35:21 UTC	Fig. 1
E02	`wget`, `ls`	Package downloaded, 3,614,418 bytes, saved 10:40:44 UTC	Fig. 2
E03	`sha256sum`, `md5sum`, `chmod 444`	Original zip hashed before use and set read-only	Fig. 4
E04	`unzip`, `sha256sum`, `md5sum`	Four files extracted; working-copy hashes recorded and set read-only	Fig. 3, 5, 6
E05	`md5sum -c`	All four working files match the published reference MD5 values	Fig. 7
E06	`file`, `mmls`, `fdisk`	FAT16 volume at sector 0; no partition table; mmls exit code 1	Fig. 8, 9, 10, 11
E07	`img_stat`, `fsstat`	Raw 259,506,176-byte image; FAT16 layout, 4,096-byte clusters	Fig. 12
E08	`fls`, `istat`, `dd`, `xxd`	Two allocated files; no deleted entries; raw root directory agrees	Fig. 13, 14, 15, 18
E09	`icat`	Both allocated files are recipes	Fig. 16, 17
E10	`blkls`	Unallocated space 259,227,136 bytes; 201,794,898 non-zero	Fig. 10
E11	PhotoRec 7.2	Two runs on the read-only copy; 132 carved files per run	Fig. 19, 20, 21, 22
E12	`find`, `sha256sum`, `diff`	266 files hashed; runs identical except report.xml; 133 unique hashes	Fig. 23, 24, 25
E13	`file`, `xxd`, `blkstat`	Nine carved images; headers at start offsets; start sectors not allocated	Fig. 26, 27, 28, 29, 30
E14	`file`, ExifTool, text extraction	Carved Word document: metadata and statements (2005 dates)	Fig. 31, 32, 33, 34
E15	`xxd`, `ls`, `grep`	Two large filler text files; no keyword matches	Fig. 35, 36, 37, 38
E16	ImageMagick `montage`	Contact sheet of 14 unique images	Fig. 83, 84
E17	`sha256sum -c`, ExifTool	Carrier copies verified; two comment fields, no warnings	Fig. 39, 40
E18	stegdetect 0.6.1	Built from commit 46a09e9; results at default and -s 3	Fig. 41, 42, 43, 44
E19	stegbreak, `rockyou.txt`	Two passphrase hits; two candidates inconclusive after crashes; hits reproduced	Fig. 45, 46, 47
E20	jpseek 0.51 under Wine	Two valid JPEGs extracted; no hash match with disk images	Fig. 48, 49, 50, 51, 52
E21	Wireshark, `tshark`	Three FTP uploads as account "gnome"	Fig. 53, 54, 55, 56, 57
E22	`sha256sum`, `file`, `xxd`, `jpeginfo`	rhino1.jpg and rhino3.jpg validated	Fig. 58, 59
E23	`tshark` follow, `zipinfo`	contraband.zip rebuilt; size equals server count; structure intact	Fig. 60, 61, 62

E24	`unzip`, ExifTool	rhino2.jpg extracted (course use)	Fig. 64, 65
E25	`grep`, cross-match note	rhino2.jpg equals carved f0106409.jpg	Fig. 66, 67
E26	`tshark`, export objects	rhino2.log: 13 requests and objects; rhino4.jpg and rhino5.gif validated	Fig. 68, 69, 70, 71, 72, 73
E27	`tshark`, export objects	rhino3.log: search then download of rhino.exe (145,920 bytes)	Fig. 74, 75, 76, 77
E28	Shell script, `sort`, `uniq`	Master table: 16 entries, 14 unique hashes	Fig. 79, 80

Appendix B. Master Image hash table

File	Source	SHA-256
f0104057.jpg	DISK	2261020a0b31fe10c3ae2872c6283e42993e24e87f1a76822e5840ed1838014e
f0104249.jpg	DISK	8cdb89c6778d3409a80355bf2b9c644c1bf0c5e31c43047080da9ea5a859de07
f0105065.jpg	DISK	452075313c93ae313331a86e8cac2f35ea3bdc24700cabf4444e39f5836403ea
f0105873.jpg	DISK	f92654d9ee17ab6b684b09de01cf0bc4076383c007964946d3f31577447596fb
f0106393.jpg	DISK	568457ed594c4e98dc503e3e244b1e316376aafb0d1c283d95601d13762a748c
f0106409.jpg	DISK	b4f6bbb8d846d6505c9b5b87f115656fdf980a484aa441078a838a3ffc2ac705
f0106865.gif	DISK	8a67d406ed130c9b5447b03fbf9dd0ac6c384a62b5028d6267e92c4abe230a80
f0106889.gif	DISK	71e9f6b94e0496cc634d5d643387c7c1405e2df9deccda1482a8c7cc5ba0b199
f0335081.jpg	DISK	f92654d9ee17ab6b684b09de01cf0bc4076383c007964946d3f31577447596fb
out_f0104249.jpg	STEG	6dbf775b8999a006ffa74c65127e5add64234184bd2f425b2ad644e8c0d19345
out_f0105065.jpg	STEG	bc9c605372fdf9cc00faf17ae9ce0fdb6d26bb9db28cdff742d9ddbe1470ec16
rhino1.jpg	FTP	0f30a8777d19a4fe4b2fafd5154b6c4dfafb673202db29971e876e978da4e05d
rhino3.jpg	FTP	a85aea8e47d9f5425456f3bafb9c67201c8c2b2b2cec80154c83102e802240c5
rhino2.jpg (from contraband.zip)	FTP	b4f6bbb8d846d6505c9b5b87f115656fdf980a484aa441078a838a3ffc2ac705
rhino4.jpg	HTTP	e1c01da391669adf3559d2d87bf2157b37e27a936a1ae66c77b61e6b5b7bc4a6
rhino5.gif	HTTP	f36ecc12967d5622cdc70e73d2468fab28a65db944d3157256a7212afa05b056